

具身智能

第3-4讲：机器感知
Junwei Liang
AI Thrust
HKUST (Guangzhou)

Embodied AI

Lecture 3-4: Machine Perception
Junwei Liang
AI Thrust
HKUST (Guangzhou)

课程事务： 第一部分

- 课程网站
 - <https://hkust-aiaa4220.github.io/fall2026/lecs.html>
 - 你可以查看课程安排、作业链接、下载课件、政策等。
 - 课件会在每堂课课前发布

如果你想在课件上做笔记，
现在就可以从课程安排网站
下载今天的课件

Embodied AI

AIAA 4220, Fall 2026

Instructors: [Junwei Liang](#)



香港科技大学 (广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Home		Schedule
Lecture:	Date and Time: 09:00AM - 11:50AM, Monday Location: Rm 233, E1 Websites: Instructure Zoom: 628-334-1826 PW: 12345 (In-person expected) (Recordings will be available after each class)	
Course Description:	This course aims to provide students with key principles and algorithms to build modern Embodied AI systems. Key topics include machine perception, embodied navigation and manipulation algorithms. Through this course, students will learn and practice the foundational principles, techniques, and tools to build new Embodied AI systems.	
Target Audience/ Prerequisites:	This is a course primarily for junior and senior students.	
Course Work:	Grading	Grading will be based on best 5 of 6 in-class quizzes (25%), one paper presentation (25%), one machine perception homework (25%) and one simulation project (25%). • The in-class quizzes are designed to complement the lectures and making sure you understand the key knowledge about the course. • There are 6 in-class quizzes over the semester. We will take your best 5 out of the 6 quizzes as your grade.
	In-Class Quizzes	

新生请查阅
前几讲课件，
了解课程
细节。

Logistics: Part 1

- Course website
 - <https://hkust-aiaa4220.github.io/fall2026/lecs.html>
 - You can see the **course schedule, homework links, download slides, policy**, etc.
 - The slides will be available before each class

You can download the slides now from the
schedule website for today’s lecture if you want
to take note on it

Embodied AI

AIAA 4220, Fall 2026

Instructors: [Junwei Liang](#)



香港科技大学 (广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Home		Schedule
Lecture:	Date and Time: 09:00AM - 11:50AM, Monday Location: Rm 233, E1 Websites: Instructure Zoom: 628-334-1826 PW: 12345 (In-person expected) (Recordings will be available after each class)	
Course Description:	This course aims to provide students with key principles and algorithms to build modern Embodied AI systems. Key topics include machine perception, embodied navigation and manipulation algorithms. Through this course, students will learn and practice the foundational principles, techniques, and tools to build new Embodied AI systems.	
Target Audience/ Prerequisites:	This is a course primarily for junior and senior students.	
Course Work:	Grading	Grading will be based on best 5 of 6 in-class quizzes (25%), one paper presentation (25%), one machine perception homework (25%) and one simulation project (25%). • The in-class quizzes are designed to complement the lectures and making sure you understand the key knowledge about the course. • There are 6 in-class quizzes over the semester. We will take your best 5 out of the 6 quizzes as your grade.
	In-Class Quizzes	

For new students,
check out slides
from last lectures
to see the details.

回顾：什么是具身智能？

- 具身智能是机器学习与机器人学的交叉
- 研究如何利用传感数据训练最优方案，让机器人行动并达成目标
- 感知与决策是关键



Recap - What is Embodied AI?

- Embodied AI is the intersection of **machine learning and robotics**
- The study of **training optimal solutions using sensory data** for robots to act and achieve goals
- **Perception** and decision making are the key



如何开始调研一个研究方向？

1. 总体策略

以数据集为中心的方法

找出驱动该领域的核心基准数据集。

文献综述

查阅使用这些数据集的高被引论文与近期前沿工作。

建立基线

梳理标准基线模型、评价指标与实验协议。

2. 主要会议与期刊

计算机视觉（含具身智能）

CVPR, ICCV, ECCV, SIGGRAPH (3D Vision, NeRF/GS)

机器学习

NeurIPS, ICML, ICLR, AAAI, IJCAI

重要研讨会

CVPR AI City Challenge, Autonomous Driving, Ego4D; NeurIPS ML for Autonomous Driving

★ 事实上的顶级具身智能会议：NeurIPS, RSS, CoRL

3. 资源

综述论文

Awesome Lists

Papers With Code

代码库 / 仓库

How to Start Surveying a Research Direction?

1. General Strategy

Dataset-Centric Approach

Identify primary benchmark datasets driving the specific field .

Literature Review

Check highly-cited and recent state-of-the-art papers using these datasets .

Establish Baselines

Map out standard baseline models, metrics, and experimental protocols .

2. Primary Conferences & Venues

Computer Vision (Inc. Embodied AI)

CVPR, ICCV, ECCV, SIGGRAPH (3D Vision, NeRF/GS)

Machine Learning

NeurIPS, ICML, ICLR, AAAI, IJCAI

(Learning-based) Robotics

ICRA, IROS, RSS, CoRL / RA-L, T-RO, Science Robotics

Key Workshops

CVPR AI City Challenge, Autonomous Driving, Ego4D; NeurIPS ML for Autonomous Driving

★ De Facto Top Embodied AI Conferences: NeurIPS, RSS, CoRL

3. Resources

Survey Papers

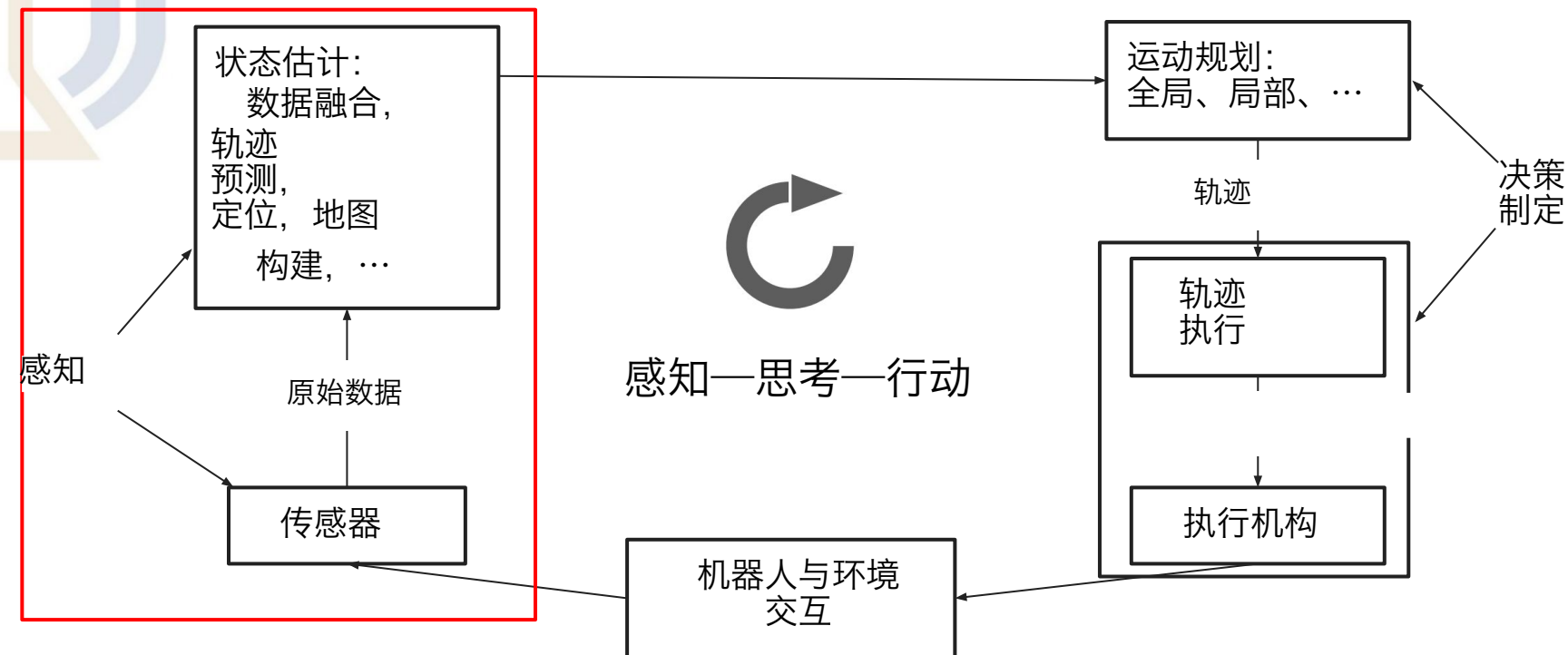
Awesome Lists

Papers With Code

Codex / Repos

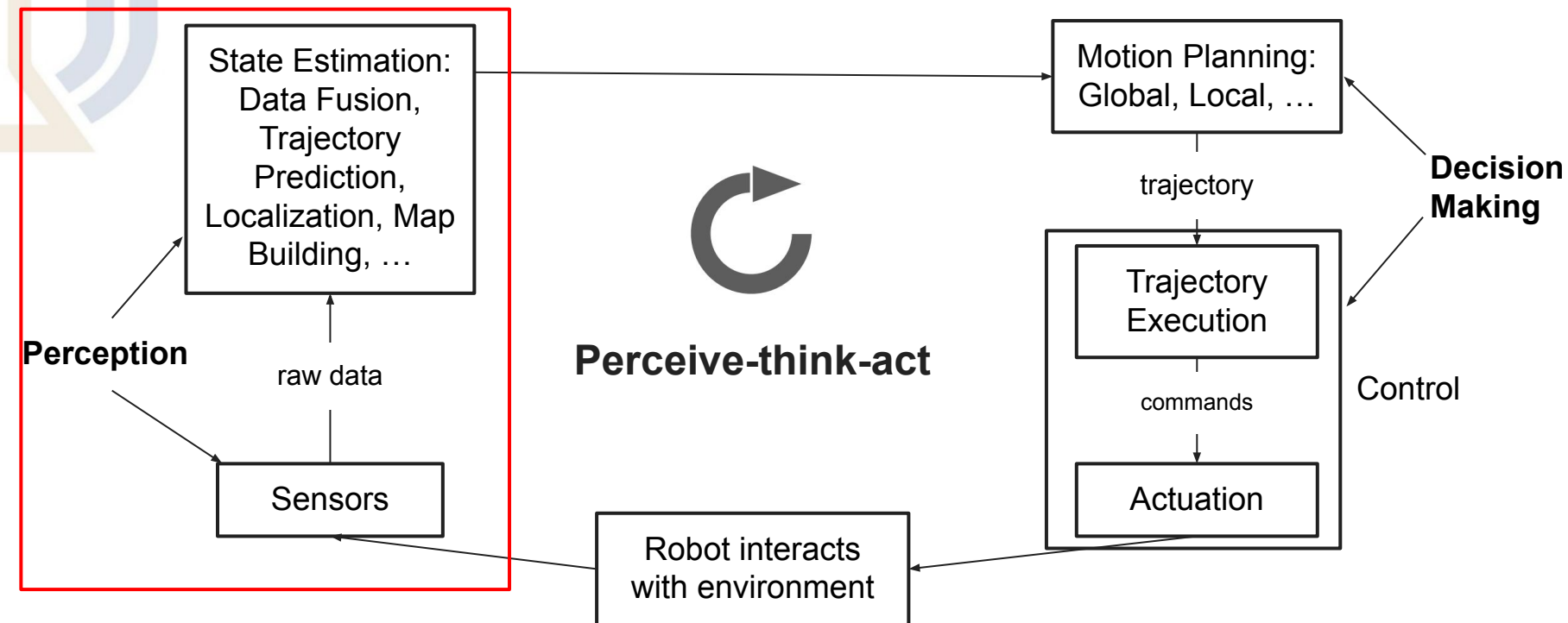
回顾：具身智能范式

- 感知—思考—行动循环



Recap - The Embodied AI Paradigm

- The Perceive-think-act circle



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 关键模型
 - CNN
 - 稠密预测
 - 注意力模型
 - 大规模视觉-语言学习
 - 几何计算机视觉
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

6

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

计算机视觉基础 —— 图像与视频存储

- 数字图像

- 若干由数字组成的矩阵
- 灰度图：单个数字矩阵

- 每个数字表示图像在该位置的强度
- 可理解为该位置的灰色深浅
- 灰度图通常由彩色图像转换得到



What we see

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

What a computer sees

Source: S. Narasimhan

Computer Vision Background - Image and Video Storage

- Digital image

- A set of matrices of numbers

- Grayscale: a single matrix of numbers

- Each number represents the intensity of the image at a specific location in the image
 - Interpreted as the shade of grey at that location
- Grayscale images are typically derived from color images



What we see

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

What a computer sees

Source: S. Narasimhan

计算机视觉基础 —— 图像与视频存储

- 数字图像 —— 灰度图

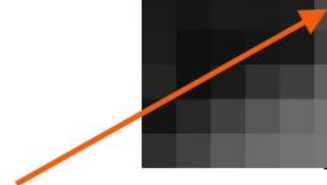
What we see



Only a single number need
be stored per pixel

What the computer "sees"

147	157	158	161	168	164	156	159	158	158
157	166	163	158	154	155	162	170	154	154
78	111	143	162	171	173	163	149	150	150
13	36	60	82	116	145	155	136	151	151
21	18	19	22	46	106	151	152	151	151
27	25	29	28	34	87	141	151	143	143
31	11	19	22	26	72	128	141	133	133
40	15	29	53	72	108	131	124	128	128
22	46	75	99	116	128	133	133	136	136
55	79	103	122	125	130	132	132	136	136



Picture Element (PIXEL)
Position & gray value (scalar)

* Credit: Rita Singh

Computer Vision Background - Image and Video Storage

- Digital image - Grayscale

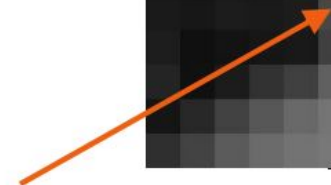
What we see



Only a single number need
be stored per pixel

What the computer "sees"

147	157	158	161	168	164	156	159	158	158
157	166	163	158	154	155	162	170	154	154
78	111	143	162	171	173	163	149	150	150
13	36	60	82	116	145	155	136	151	151
21	18	19	22	46	106	151	152	151	151
27	25	29	28	34	87	141	151	143	143
31	11	19	22	26	72	128	141	133	133
40	15	29	53	72	108	131	124	128	128
22	46	75	99	116	128	133	133	136	136
55	79	103	122	125	130	132	132	136	136

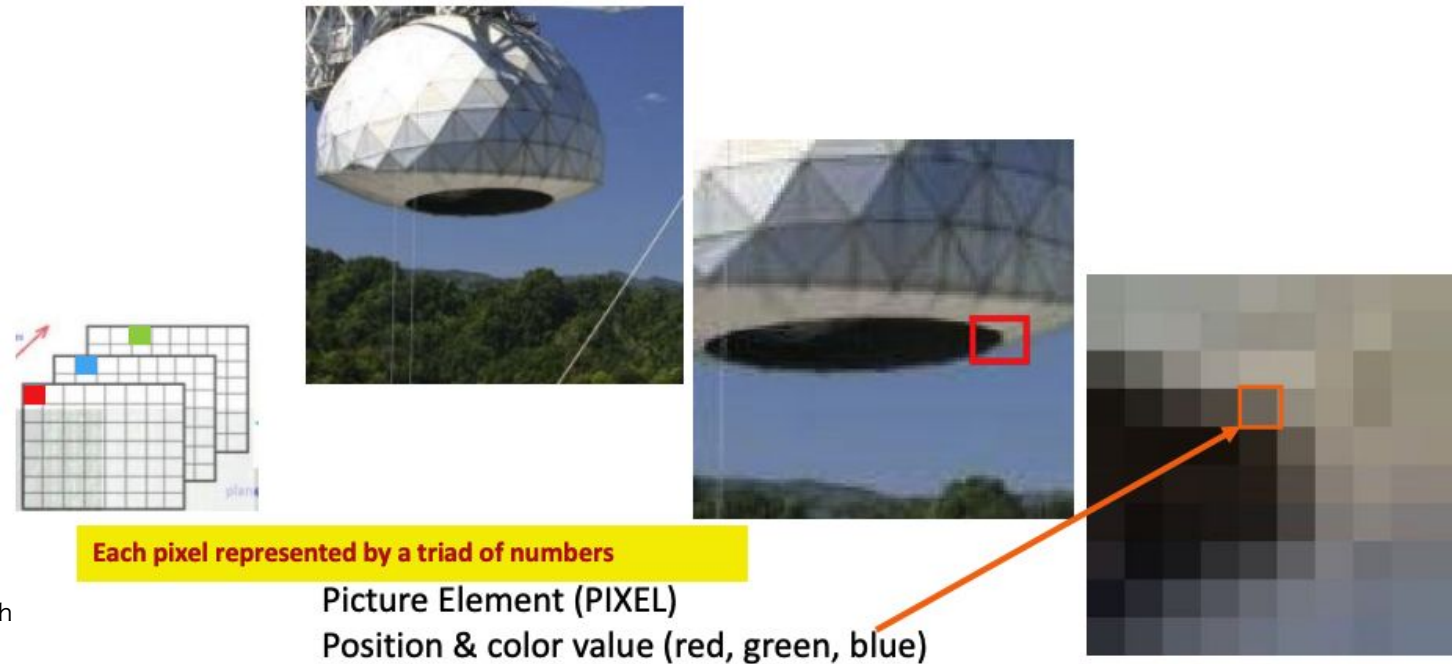


Picture Element (PIXEL)
Position & gray value (scalar)

* Credit: Rita Singh

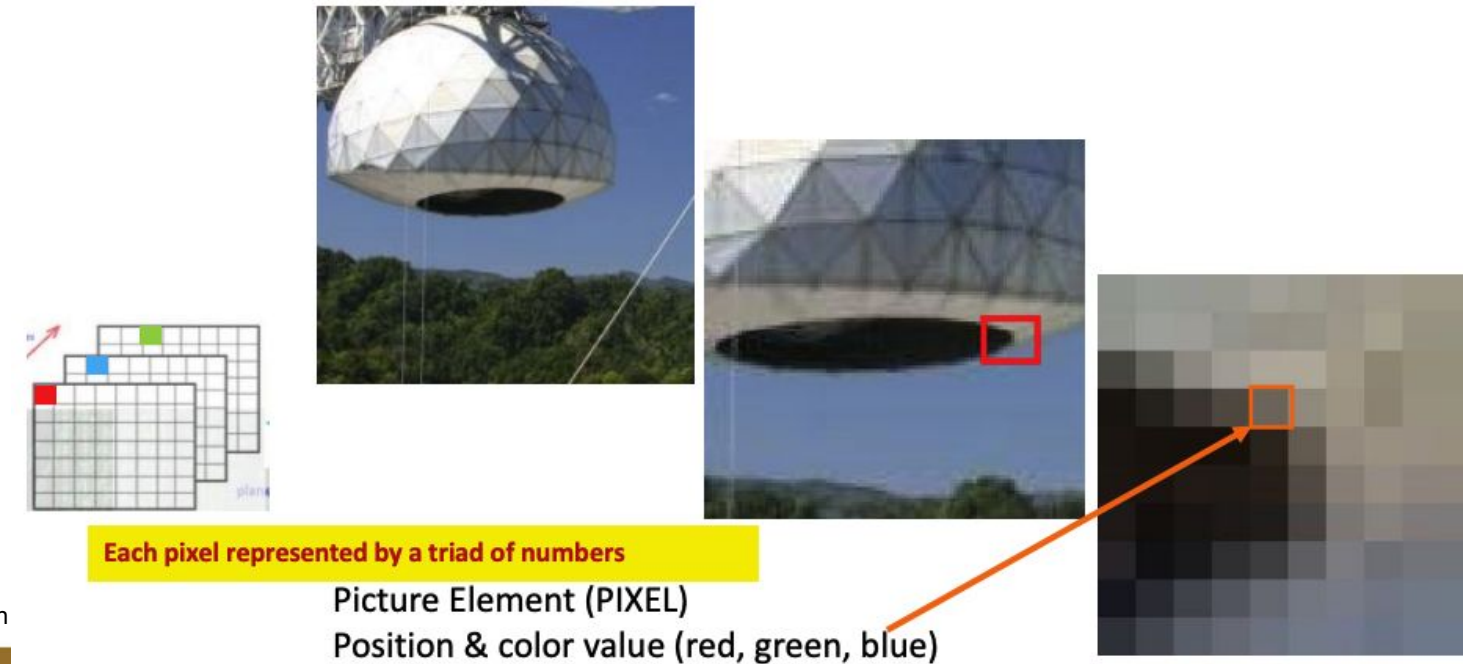
计算机视觉基础 —— 图像与视频存储

- 数字图像 —— 彩色图像



Computer Vision Background - Image and Video Storage

- Digital image - Color images

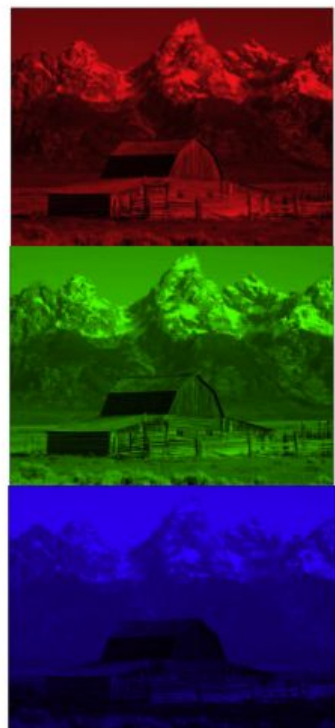


计算机视觉基础 —— 图像与视频存储

- 数字图像 —— 彩色图像 —— RGB 颜色空间



Example from wikipedia



RGB colorspace:

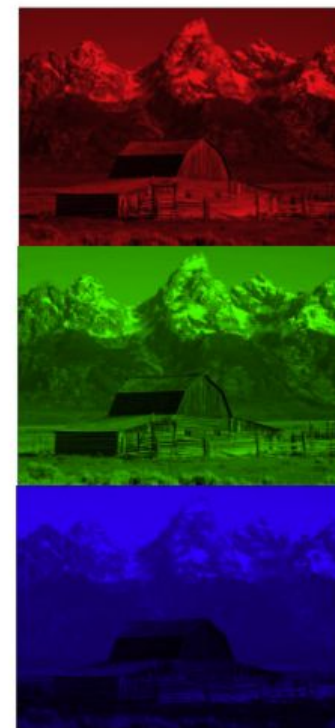
The three matrices represent R, G and B

Computer Vision Background - Image and Video Storage

- Digital image - Color images - RGB colorspace



Example from wikipedia



RGB colorspace:

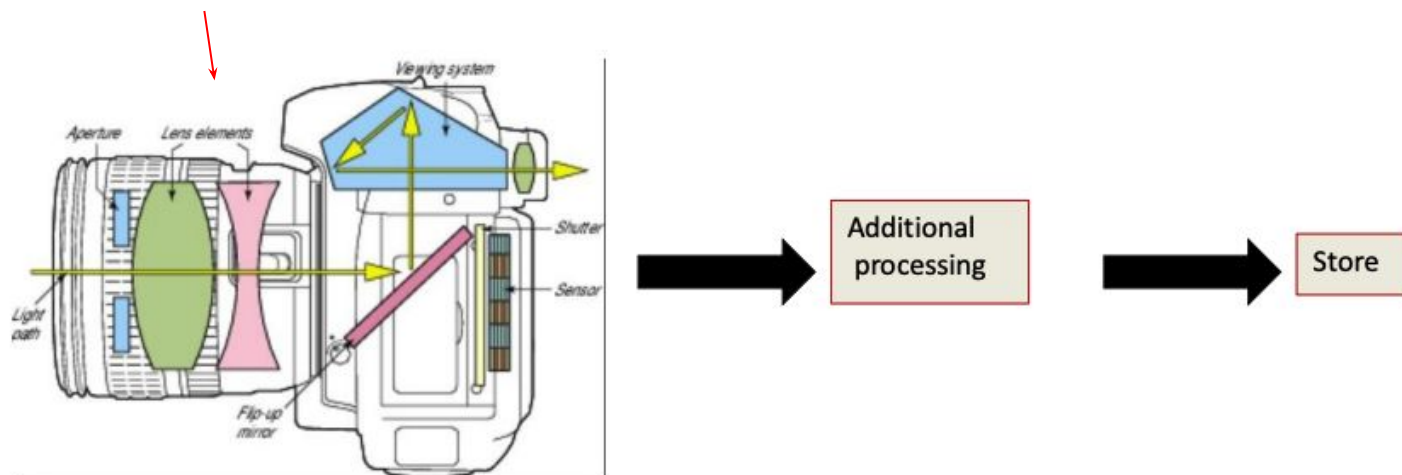
The three matrices represent R, G and B

计算机视觉基础 —— 图像与视频存储

- 相机拍摄之后会发生什么？

- 数字图像并不会以原始形式直接存储
- 图像在存储前会经过多阶段处理

带透镜的针孔相机模型：
上一讲已经讲过

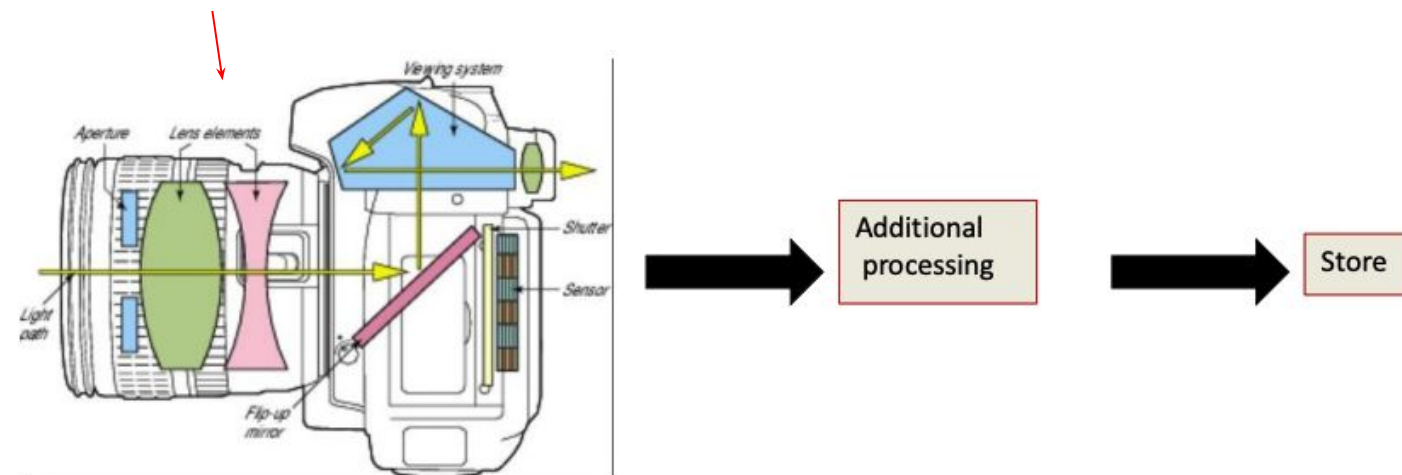


Computer Vision Background - Image and Video Storage

- What happens after camera capture?

- The digital image is not stored in its raw form
- Image is processed in multiple stages before it is stored

Pinhole camera model with lens:
we cover this in last lecture



计算机视觉基础 —— 图像与视频存储

- 相机拍摄之后会发生什么？
 - 数字图像并不会以原始形式直接存储
 - 图像在存储前会经过多阶段处理
- 对颜色做插值
- 伽马校正
- 以及其他处理步骤

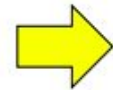
Computer Vision Background - Image and Video Storage

- What happens after camera capture?
 - The digital image is not stored in its raw form
 - Image is processed in multiple stages before it is stored
 - White balancing
 - Demosaicing
 - Interpolation for colors
 - Gamma correction
 - And other processing steps

计算机视觉基础 —— 图像与视频存储

- 白平衡

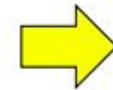
- 相机会自动调整各颜色通道，得到更“中性”的图像
- 更接近在白光下拍摄的效果



Computer Vision Background - Image and Video Storage

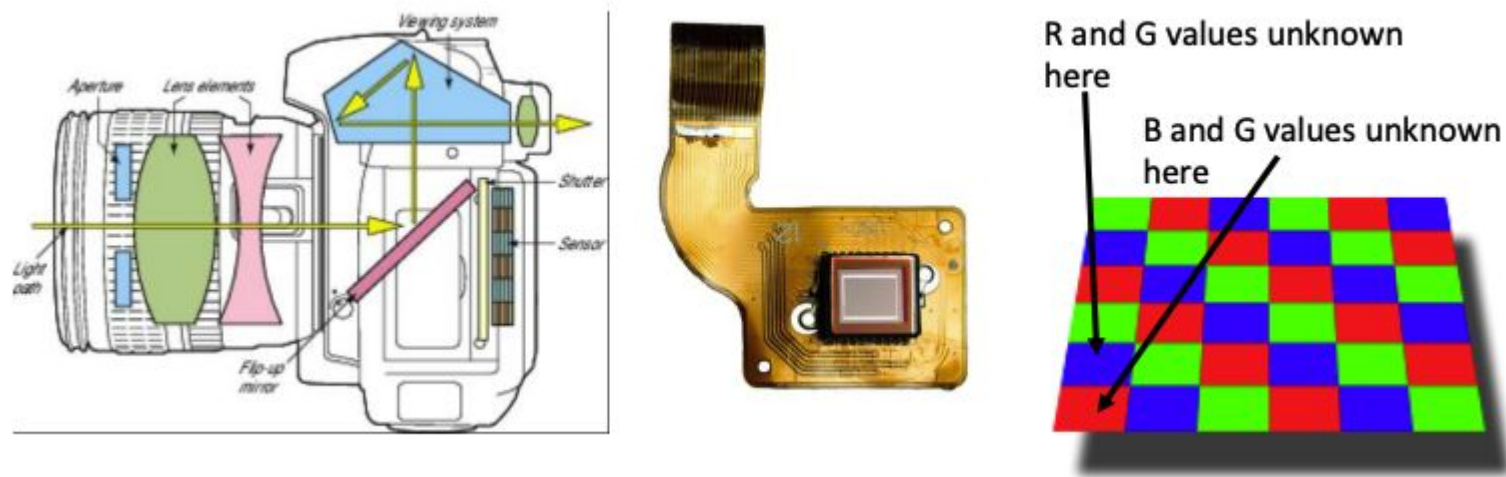
- White balancing

- Camera automatically adjusts individual colors to give you a more “neutral” image
- Closer to an image taken under white light



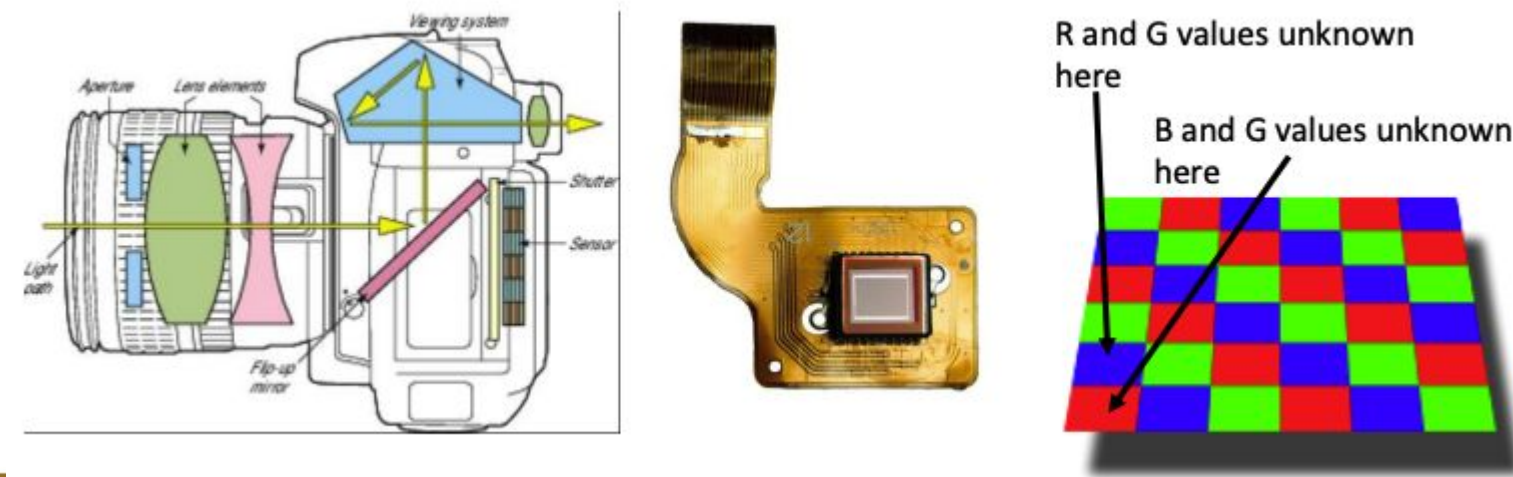
计算机视觉基础 —— 图像与视频存储

- 一块 $N \times M$ 的数码相机并不会为三种颜色各自采集 $N \times M$ 个像素
- R、G、B 传感器交替排列
- 任意像素上只有一种颜色的值是已知的，另外两种未知



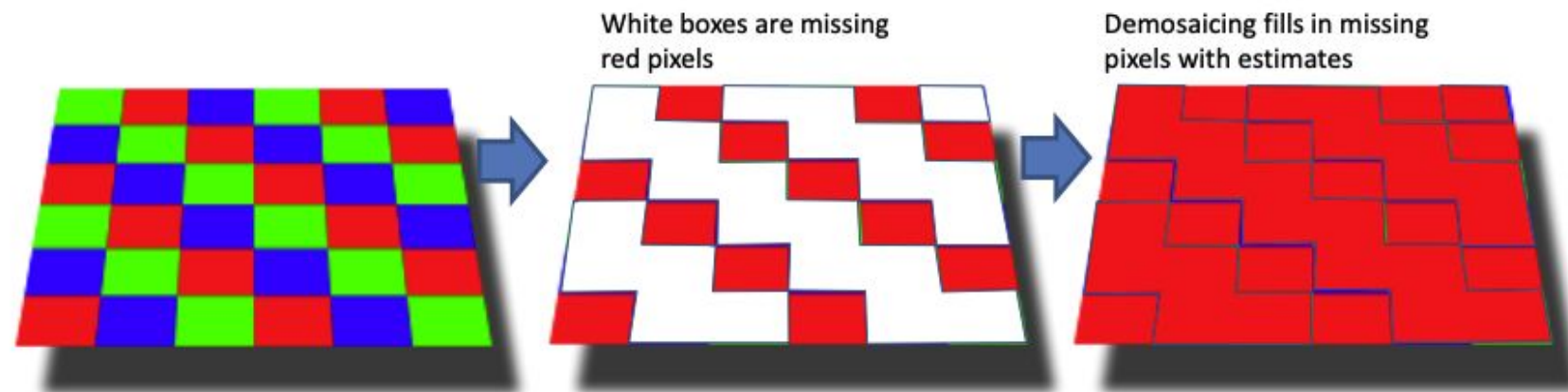
Computer Vision Background - Image and Video Storage

- Demosaicing
 - An $N \times M$ digital camera does not actually capture $N \times M$ pixels for each of the three colors
 - R, G and B sensors alternate
 - In any pixel, the value of only one of the colors is known, the other two are unknown



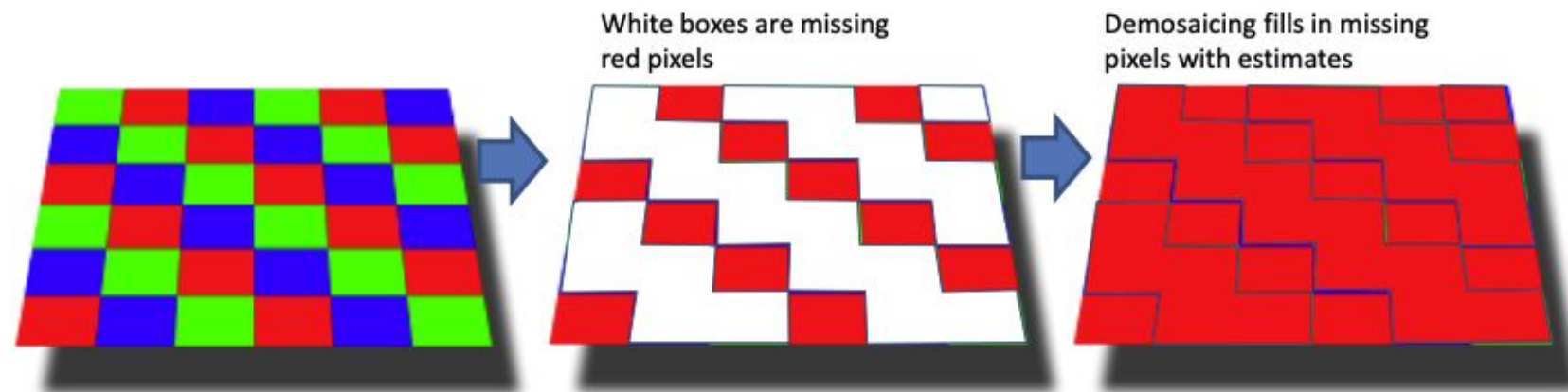
计算机视觉基础 —— 图像与视频存储

- 对每种颜色，缺失的像素必须补齐
 - 通过邻域估计得到
- 由最近的已知像素做线性组合
- 这一过程称为去马赛克
- 其实和超分辨率任务很像！



Computer Vision Background - Image and Video Storage

- Demosaicing
 - For each color, the missing pixels must be filled in
 - Done by estimating them from their neighbors
 - A linear combination of the nearest present pixels
 - This process is called demosaicing
 - Actually similar to the super-resolution task!



计算机视觉基础 —— 图像与视频存储

- 伽马校正

- 各颜色通道的像素值会经过非线性压缩
- 这会降低像素值的动态范围

■ 从而可以用更少的比特存储



Computer Vision Background - Image and Video Storage

- Gamma correction

- Pixel values (of all colors) are compressed through a non-linearity
- This reduces the dynamic range of pixel values
 - Can be stored using fewer bits



计算机视觉基础 —— 图像与视频存储

- 存储：可交换图像文件格式（EXIF）

- 一种标准化的图像格式

- 由多个标准化组织共同约定

- 规定了数字图像记录设备（相机、扫描仪等）所用的图像、声音及其他标签格式

- 标准包含以下格式规范

- JPEG 图像

- TIFF 图像 (Tagged Image File Format)

- RIFF wav 音频文件 (Resource Interchange File Format)

- 附加的元数据

Computer Vision Background - Image and Video Storage

- Storage: The Exchangeable Image Format (EXIF)

- A standardized format for images

- Agreed upon by a number of standardization bodies

- Specifies formats for images, sounds, and other tags used by digital image recorders (cameras, scanners, etc.)

- Standard includes format specifications for

- JPEG images

- TIFF images (Tagged Image File Format)

- RIFF wav for audio files (Resource Interchange File Format)

- Appended metadata

计算机视觉基础 —— 图像与视频存储

- 存储：可交换图像文件格式（EXIF）

○ 示例

■ 注意相机焦距！（以及其他内参）

Tag	Value	from wikipedia	
Manufacturer	CASIO	Date and time (digitized)	2003:08:11 16:45:32
Model	QV-4000	Components configuration	Y Cb Cr –
Orientation (rotation)	top-left [8 possible values ^[20]]	Compressed bits per pixel	4.01
Software	Ver1.01	Exposure bias	0.0
Date and time	2003:08:11 16:45:32	Max. aperture value	2.00
YCbCr positioning	centered	Metering mode	Pattern
Compression	JPEG compression	Flash	Flash did not fire
X resolution	72.00	Focal length	20.1 mm
Y resolution	72.00	MakerNote	432 bytes unknown data
Resolution unit	Inch	FlashPix version	FlashPix version 1.0
Exposure time	1/659 s	Color space	sRGB
F-number	f/4.0	Pixel X dimension	2240
Exposure program	Normal program	Pixel Y dimension	1680
Exif version	Exif version 2.1	File source	DSC
Date and time (original)	2003:08:11 16:45:32	Interoperability index	R98
		Interoperability version	(null)

Computer Vision Background - Image and Video Storage

- Storage: The Exchangeable Image Format (EXIF)

○ Example

■ Note the camera focal length! (and other intrinsic parameters)

Tag	Value	from wikipedia	
Manufacturer	CASIO	Date and time (digitized)	2003:08:11 16:45:32
Model	QV-4000	Components configuration	Y Cb Cr –
Orientation (rotation)	top-left [8 possible values ^[20]]	Compressed bits per pixel	4.01
Software	Ver1.01	Exposure bias	0.0
Date and time	2003:08:11 16:45:32	Max. aperture value	2.00
YCbCr positioning	centered	Metering mode	Pattern
Compression	JPEG compression	Flash	Flash did not fire
X resolution	72.00	Focal length	20.1 mm
Y resolution	72.00	MakerNote	432 bytes unknown data
Resolution unit	Inch	FlashPix version	FlashPix version 1.0
Exposure time	1/659 s	Color space	sRGB
F-number	f/4.0	Pixel X dimension	2240
Exposure program	Normal program	Pixel Y dimension	1680
Exif version	Exif version 2.1	File source	DSC
Date and time (original)	2003:08:11 16:45:32	Interoperability index	R98
		Interoperability version	(null)

计算机视觉基础 —— 图像与视频存储

- 相机拍摄之后会发生什么？

- 数字图像并不会以原始形式直接存储
- 图像在存储前会经过多阶段处理

- 对颜色做插值
 - 伽马校正
 - 以及其他处理步骤

- 图像压缩, JPEG、PNG...

Computer Vision Background - Image and Video Storage

- What happens after camera capture?

- The digital image is not stored in its raw form
- Image is processed in multiple stages before it is stored
 - White balancing
 - Demosaicing
 - Interpolation for colors
 - Gamma correction
 - And other processing steps
 - **Image compression**, JPEG, PNG...

计算机视觉基础 —— 图像与视频存储

- 摄像机

- 和静态相机类似
- 以固定的每秒次数自动连续拍摄
- 帧率由视觉暂留决定
- 通常是 30 张/秒 (帧每秒, FPS)
- 声音同步录制, 并与图像帧时间对齐



- 全局快门、卷帘快门
 - 快门速度 快门/曝光时间
- 30 FPS: 快门速度最多
 $1/30 = 33.3 \text{ ms}$
做网球跟踪时,
需要 $< 1 \text{ ms}$ 的快门速度。

Computer Vision Background - Image and Video Storage

- Video cameras

- Just like still image cameras
- Take multiple shots automatically at a fixed rate per second
- The rate is decided based on persistence of vision
- Usually 30 photos/second (frame-per-second, FPS)
- Sound is recorded alongside and time synchronized with the photo frames



- Global shutter, rolling shutter
 - Shutter speed 快门/曝光时间
- 30 FPS: shutter speed at most
 $1/30 = 33.3 \text{ ms}$
When we do tennis ball tracking,
shutter speed of $< 1 \text{ ms}$ is needed.

计算机视觉基础 —— 图像与视频存储

- 摄像机

- 用于压缩图像以便存储与分发，再解压压缩后的图像以便显示
- 为什么需要？

- 原始高清视频每小时可高达 400GB..

- 图像: JPEG、PNG、..

- 视频: H.264、MPEG-4、...

- 可用 2 个空间维 + 1 个时间维表示
- 编码利用空间域与时间域中的冗余

Computer Vision Background - Image and Video Storage

- Video cameras

- Codec

- scheme to **compress** images for storage and distribution and to **decompress** the compressed images for display
 - Why?
 - Raw HD video can take up to 400GB per hour..
 - Image: JPEG, PNG,...
 - Video: H.264, MPEG-4, ...
 - Can be represented in 2 spatial and 1 temporal dimension
 - Coding exploits **redundancies** in spatial and temporal domains

计算机视觉基础 —— 图像与视频存储

- 摄像机
 - 编解码器 —— 视频编码
 - 空间编码
 - 也叫帧内编码 (intra-coding)
 - 利用一帧内部的冗余 (JPEG 等)
 - 时间编码
 - 也叫预测编码
 - 利用跨帧冗余

Computer Vision Background - Image and Video Storage

- Video cameras
 - Codec - Video coding
 - Spatial coding
 - Also called intra-coding
 - Exploits redundancies within a frame (JPEG, etc.)
 - Temporal coding
 - Also called predictive coding
 - Exploits redundancies across frames

计算机视觉基础 —— 图像与视频存储

- 摄像机

- 编解码器 —— 视频编码

- 三种帧类型

- I 帧 (Intra-coded)、P 帧 (Predicted) 和 B 帧 (Bi-directionally predicted)
 - I 帧用类似 JPEG 的方式编码, 利用空间冗余 (与图像编码相同)。视频的第一帧是 I 帧
 - P 帧、B 帧: 只存储相对参考帧的变化。
 - I 帧会按固定间隔插入, 或根据视频中的运动

插入

- 这些压缩不是无损的 (因此会有伪影)。有时我们的识别模型也会受到影响



Computer Vision Background - Image and Video Storage

- Video cameras

- Codec - Video coding

- 3 types of frames

- **I-frame** (Intra-coded), P-frame (Predicted) and B-frame (Bi-directionally predicted)
 - I-frames are coded using a JPEG like scheme. Exploits spatial redundancies (as in image coding). The first frame of a video is an I-frame
 - P-frames, B-frames: Only changes from the reference are stored.
 - I-frames are inserted at regular intervals or inserted based on the motion in the video

- These compressions are not lossless (hence there are artifacts). Sometimes our recognition model may be affected



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 关键模型
 - CNN
 - 稠密预测
 - 注意力模型
 - 大规模视觉-语言学习
 - 几何计算机视觉
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

24

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

计算机视觉基础 —— 特征

- 在卷积神经网络（CNN）模型出现之前

- 基于启发式的特征工程
 - 局部描述子
 - 全局表示
- 然后再用支持向量机等机器学习方法

■ 在 AIAA 5032 (AI 基础) / AIAA 2205 (AI 导论) 中讲过



Feature
Engineering

ML Methods
Ex. SVMs

Computer Vision Background - Features

- Before convolutional neural network (CNN) model

- Feature engineering based on heuristics
 - Local descriptors
 - Global representation
- Then apply ML methods like support vector machines
 - Covered in AIAA 5032 (Foundations of AI) / AIAA 2205 (Intro to AI)



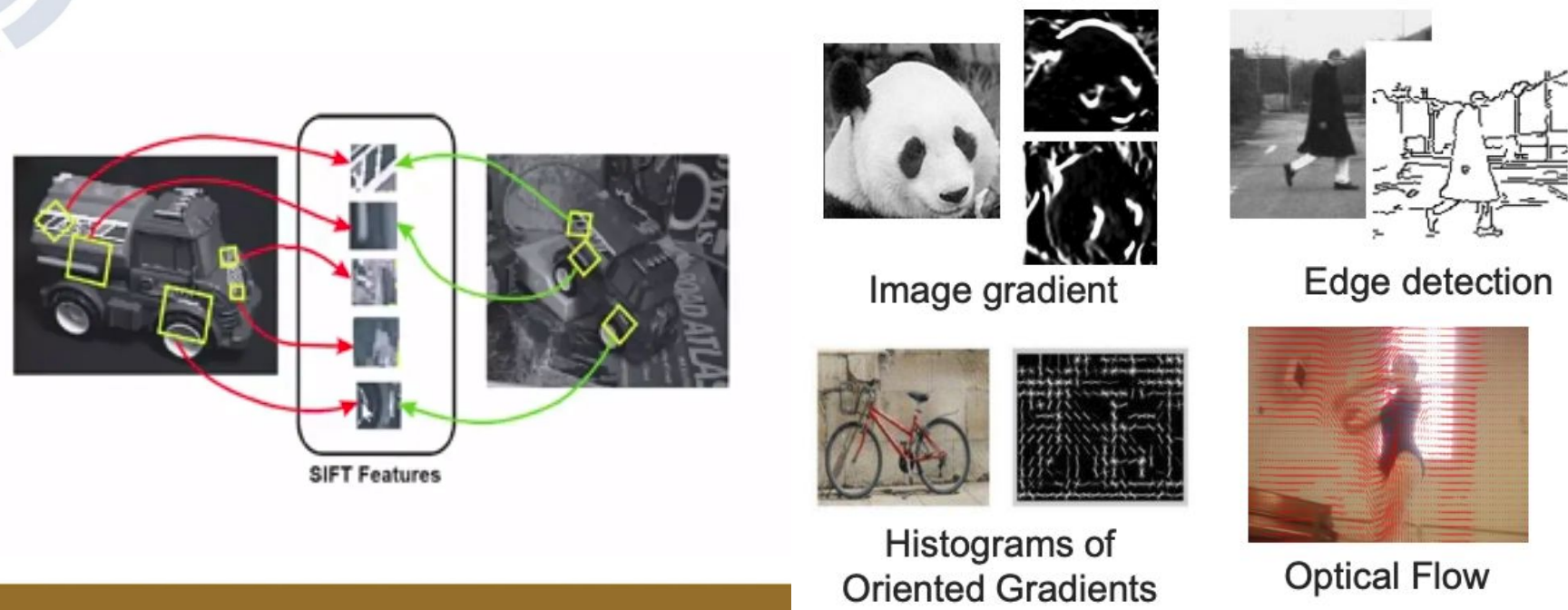
Feature
Engineering

ML Methods
Ex. SVMs

计算机视觉基础 —— 特征

- 局部描述子

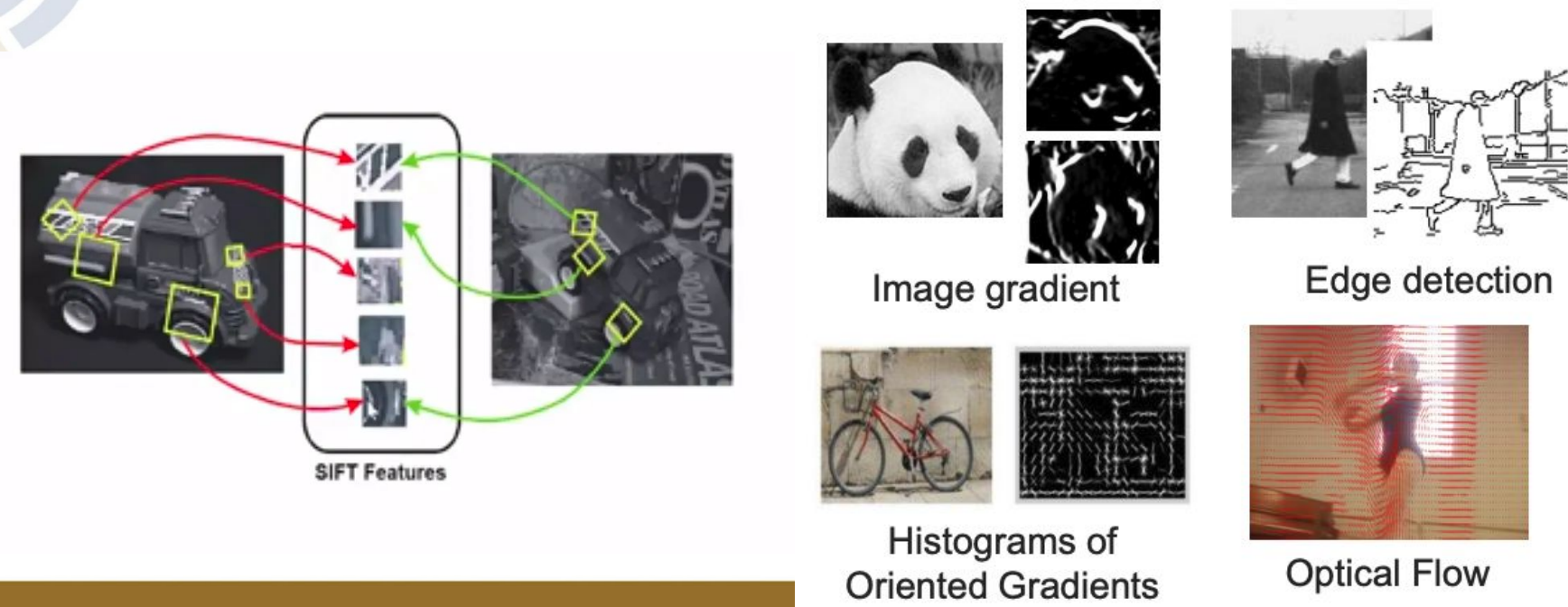
- 方向梯度直方图 (HOG)、尺度不变特征变换 (SIFT) 等



Computer Vision Background - Features

- Local descriptors

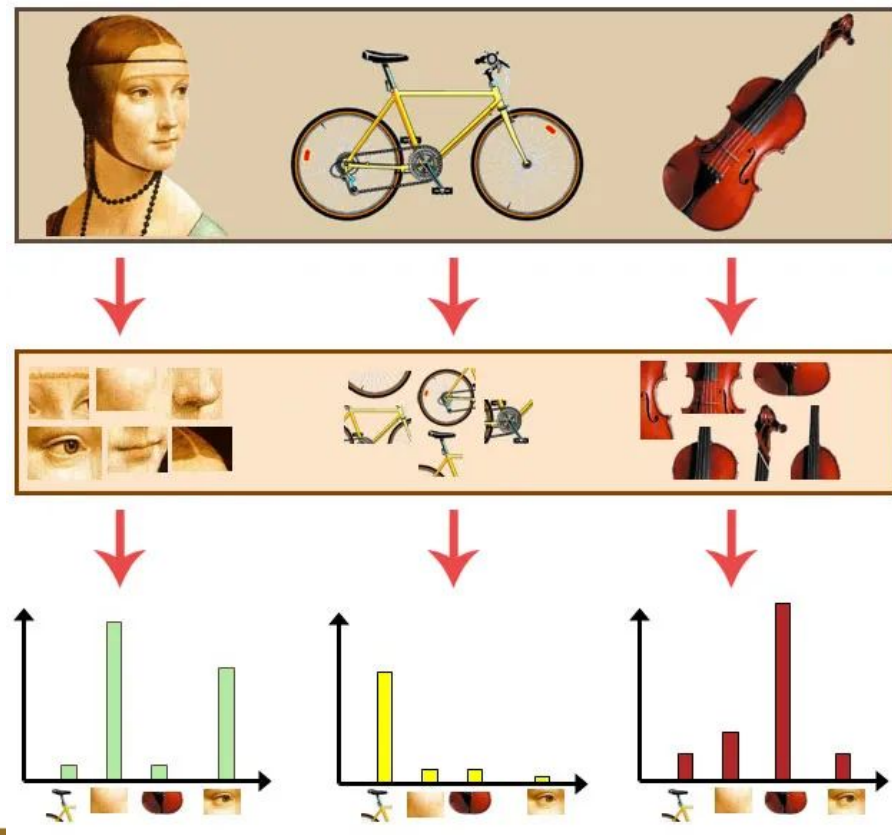
- histogram of oriented gradients (HOG), Scale-Invariant Feature Transform (SIFT), etc.



计算机视觉基础 —— 特征

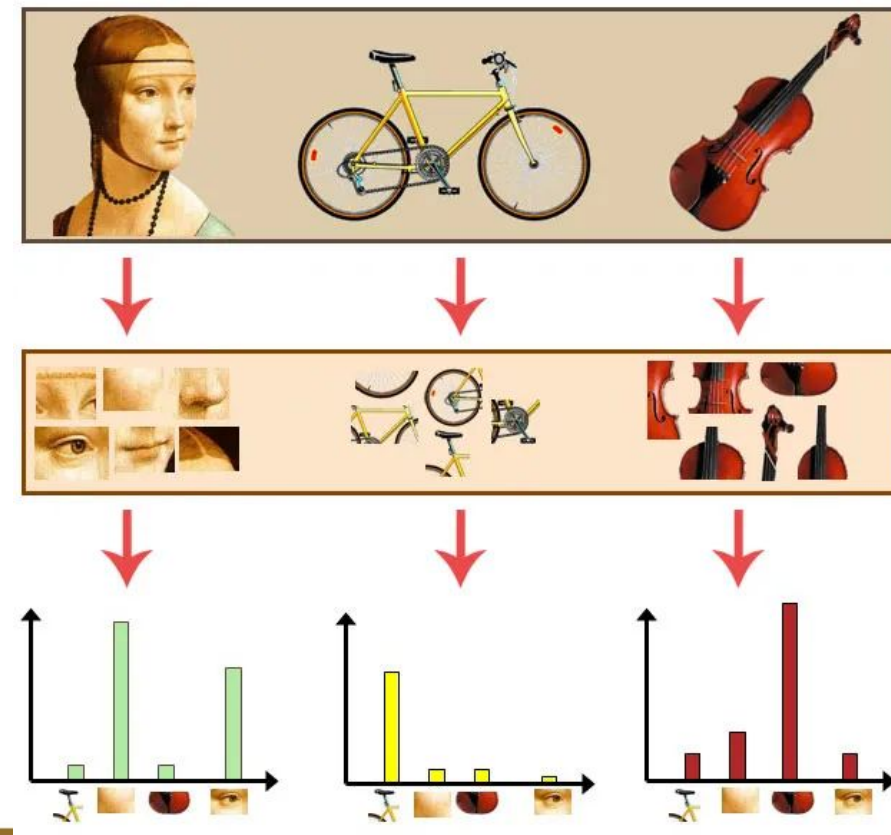
• 全局表示

- 局部描述子
- 聚类
 - K-means
- 得到固定长度的向量表示, 适用于任意尺寸的图片/视频



Computer Vision Background - Features

- Global representation
 - Bag-of-visual-words
 - Local descriptors
 - Clustering
 - K-means
 - Got fixed length vector representation for any size image/video

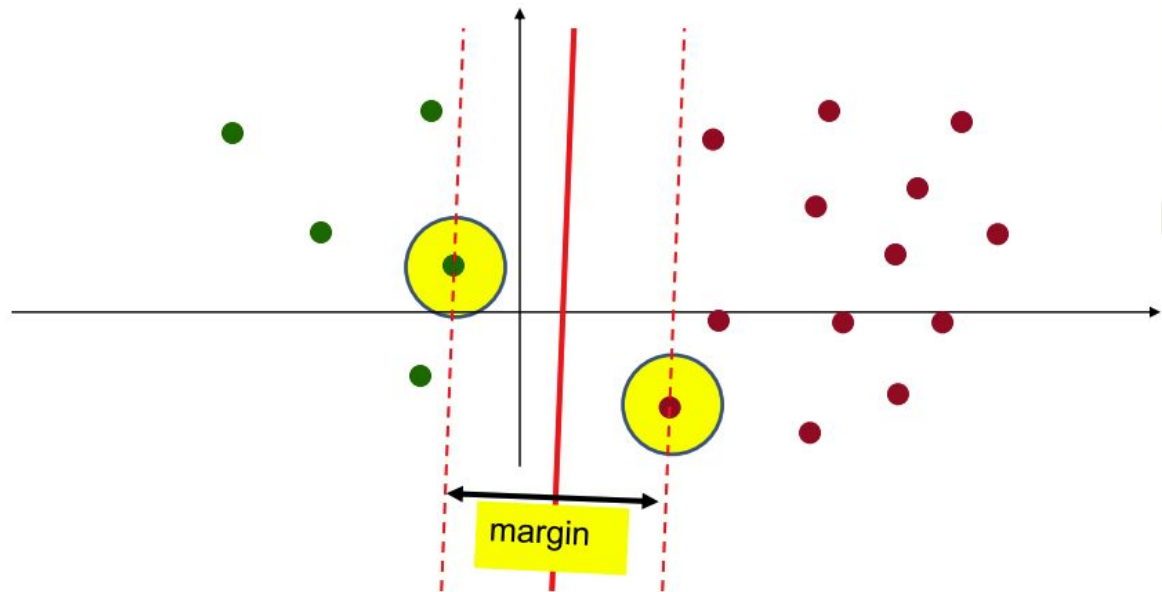


计算机视觉基础 —— 特征

- 全局表示

- 局部描述子
- 聚类

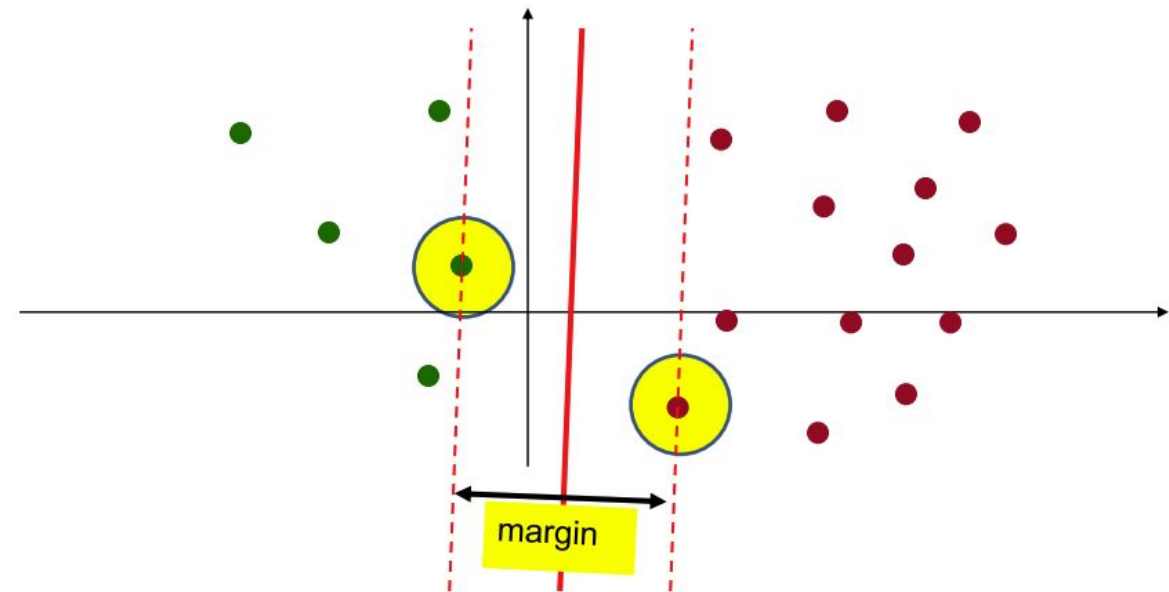
- K-means
- 得到固定长度的向量表示, 适用于任意尺寸的图像/视频
- 然后再用机器学习方法, 如支持向量机 (SVM)



Computer Vision Background - Features

- Global representation

- Bag-of-visual-words
 - Local descriptors
 - Clustering
 - K-means
 - Got fixed length vector representation for any size image/video
- Then apply ML methods like support vector machines (SVM)



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 关键模型
 - CNN
 - 稠密预测
 - 注意力模型
 - 大规模视觉-语言学习
 - 几何计算机视觉
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

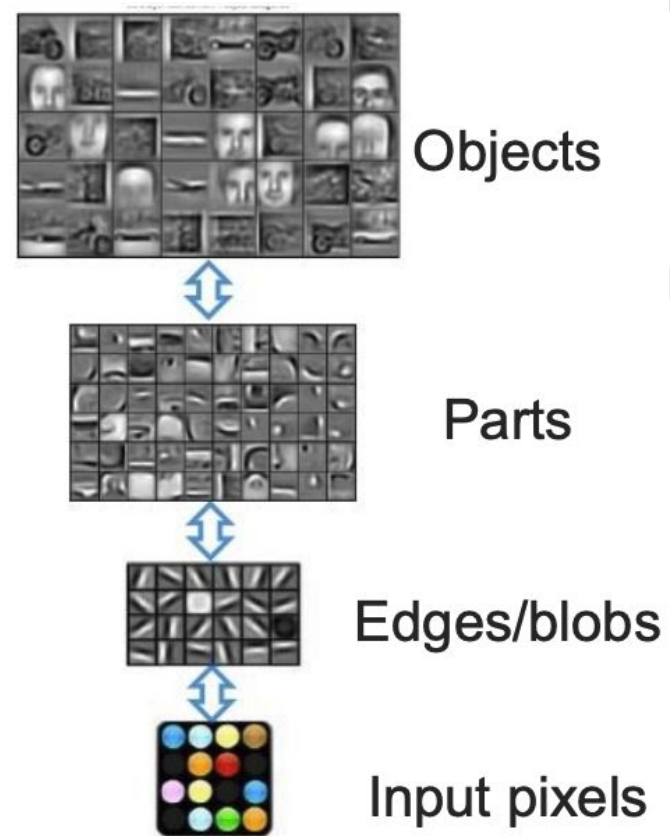
29

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

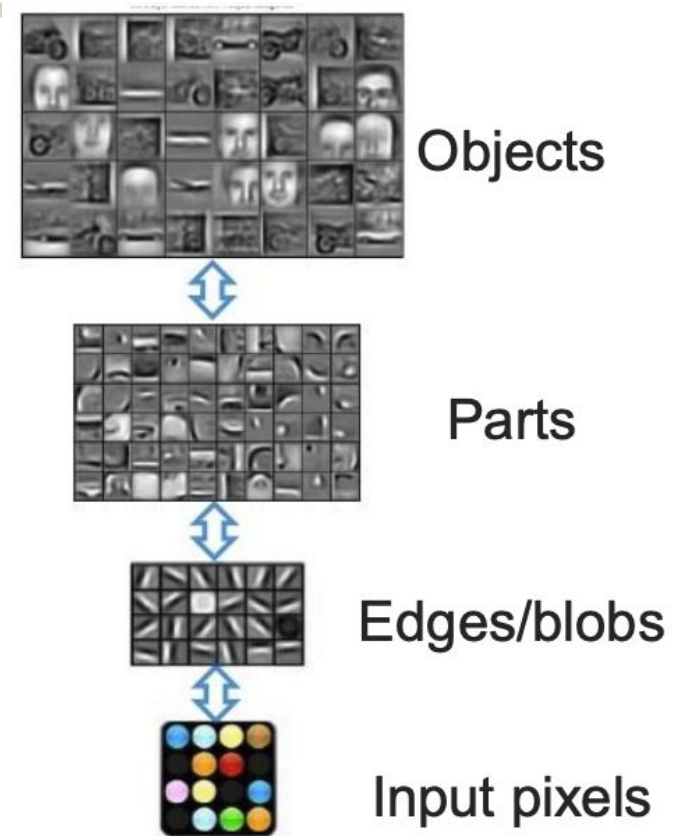
卷积神经网络

- 设计目标
 - 构建更抽象、分层的视觉表示
- 主要优势:
 - 灵感来自视觉皮层（人眼）
 - 促进视觉抽象
 - 利用平移不变性
 - 卷积核/模板是学习得到的
 - 参数量少于 MLP



Convolutional Neural Networks

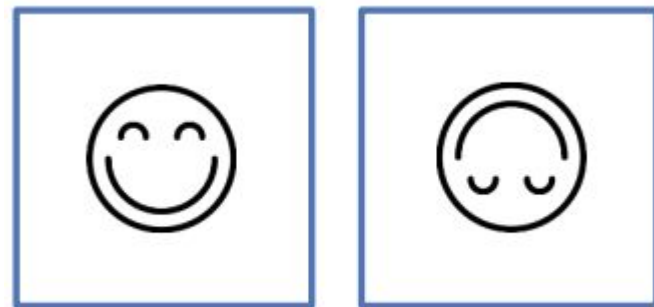
- Design goal
 - building more abstract, hierarchical visual representations
- Key advantages:
 - Inspired from visual cortex (human eye)
 - Encourages visual abstraction
 - Exploits translation invariance
 - Kernels/templates are learned
 - Fewer parameters than MLP



卷积神经网络

- 平移不变性

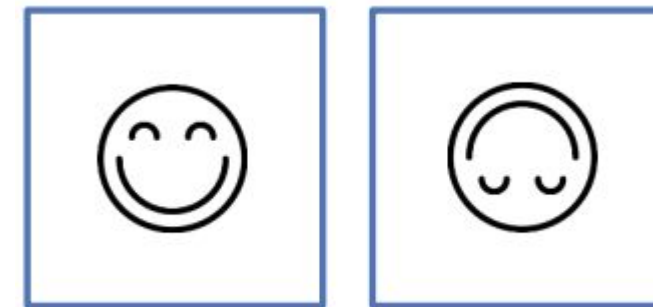
- 两个数据点 —— 哪个朝上？
 - MLP 很容易学会这个任务



Convolutional Neural Networks

- Translation Invariance

- 2 data points - which one is up?
 - MLP can easily learn this task



卷积神经网络

- 平移不变性

- 两个数据点 —— 哪个朝上?

- MLP 很容易学会这个任务

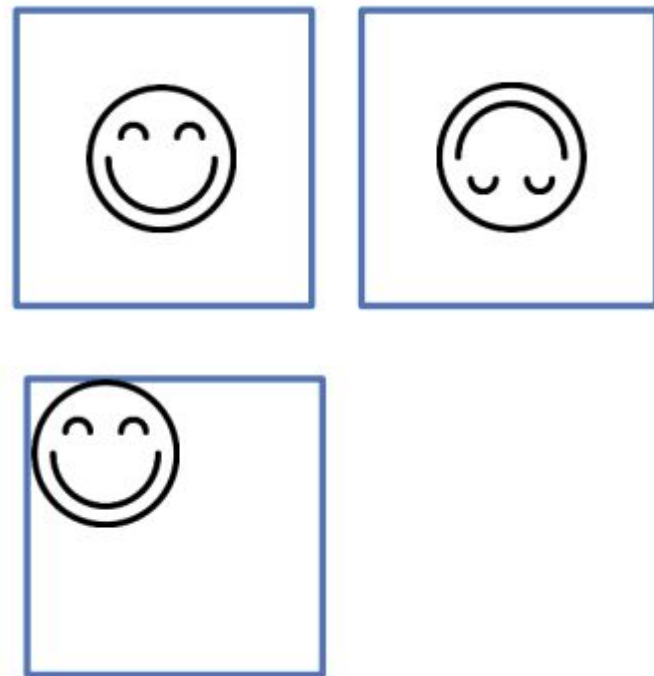
- 如果脸稍微平移了呢?

- MLP 可能识别失败

- 不具备平移不变性

- CNN 具有平移不变性

- 基于卷积核的滑动窗口计算



Convolutional Neural Networks

- Translation Invariance

- 2 data points - which one is up?

- MLP can easily learn this task

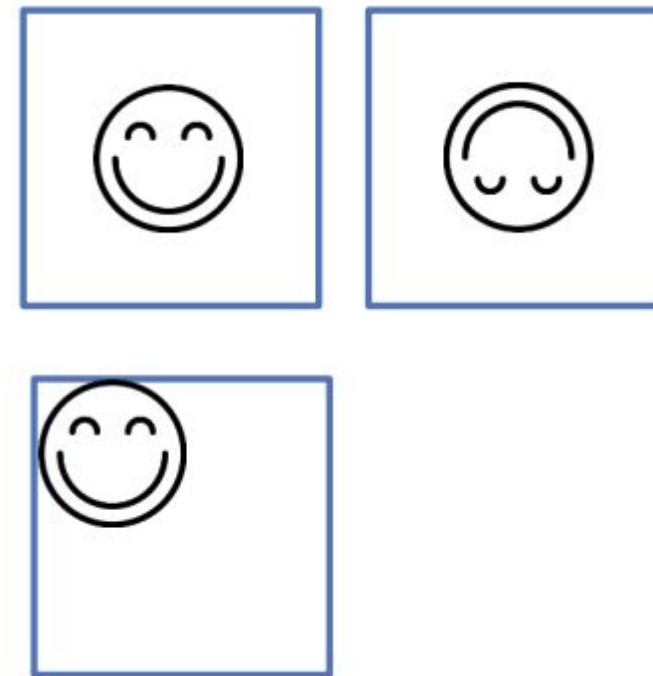
- What if the face is slightly translated?

- MLP may fail to recognize this

- Not translation invariant

- CNN has translation invariance properties

- Kernel-based, sliding-window computation

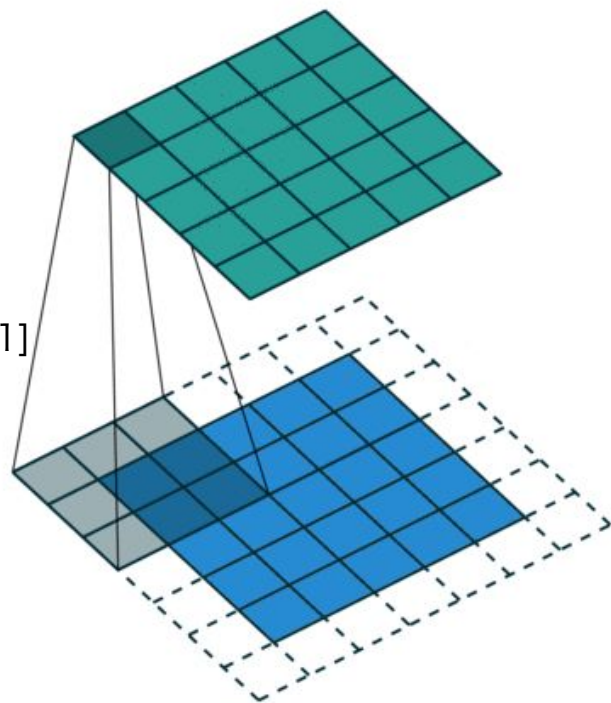


卷积实践

- 一维 CNN:

- 信号长度为 N
- 卷积核长度为 K
- 输出长度为 $N - K + 1$
- 例子

- 输入大小 (3), 卷积核/滤波器大小 (2), 输出大小 (2)
- $f = [1, 2, 1]$, $g = [1, -1]$, $f * g = [1-2, 2-1] = [-1, 1]$



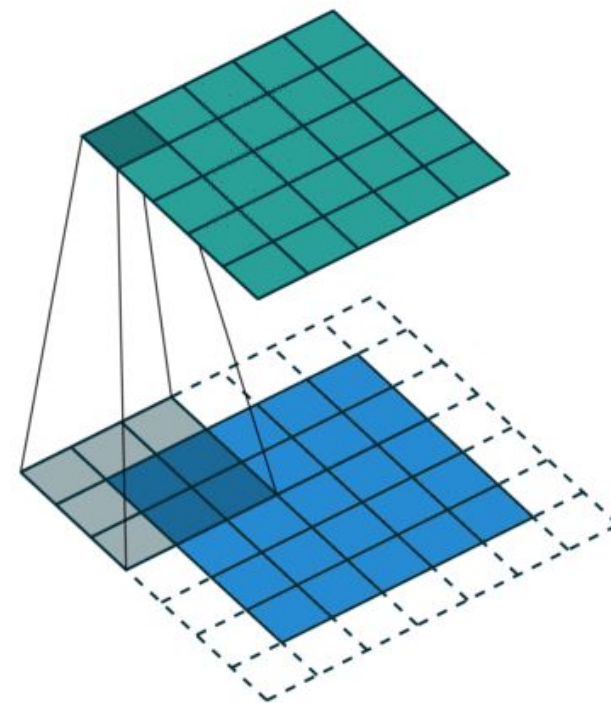
Convolution In Practise

- 1-dimensional CNN:

- We have a signal of length N
- Kernel of length K
- Output will be length $N - K + 1$

- Example

- Input size (3), kernel/filter size (2), output size (2)
- $f = [1, 2, 1]$, $g = [1, -1]$, $f * g = [1-2, 2-1] = [-1, 1]$



卷积实践

- 如果希望输出尺寸不同，可以给信号加 padding:
 - 只需在两端补 0
 - 例子

$$\bigcirc \boxtimes = [1, 2, 1], \boxtimes' = [0, 1, 2, 1], \boxtimes = [1, \boxtimes 1], \boxtimes' \boxtimes \boxtimes = [-1, 1-2, 2-1,] = [-1, -1, 1]$$

Convolution In Practise

- If we want output to be different sizes, we can **add padding** to the signal:
 - Just add 0s at the beginning and end
 - Example
 - $f = [1, 2, 1], f' = [0, 1, 2, 1], g = [1, -1], f * g = [-1, 1-2, 2-1,] = [-1, -1, 1]$

卷积实践

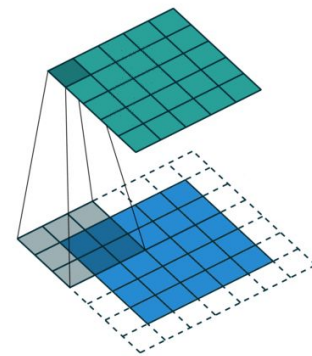
- 如果希望输出尺寸不同，可以给信号加 padding:
 - 或者
 - 做步长卷积（也称膨胀卷积）：滤波器会跳过若干像素或采样点

- 信号长度为 N
- 卷积核长度为 K
- 输出长度为 $(N \div K) \times \text{stride} + 1$
- 步长为 2 的例子
- 输入大小 (6)，卷积核 (2)，输出 (3)
- $f = [0, 0, 1, 2, 1, 0]$, $g = [1, 1]$, $f * g = [0, -1, 1]$

Convolution In Practise

- If we want output to be different sizes, we can add padding to the signal:
 - Or
 - We can perform **strided (aka, dilated) convolution**: the filter jumps over pixels or samples
 - We have a signal of length N
 - Kernel of length K
 - Output will be length $(N - K) / \text{stride} + 1$
 - Example with stride 2
 - Input of size (6), kernel (2), output (3)
 - $f = [0, 0, 1, 2, 1, 0]$, $g = [1, -1]$, $f * g = [0, -1, 1]$

二维卷积 —— 示例



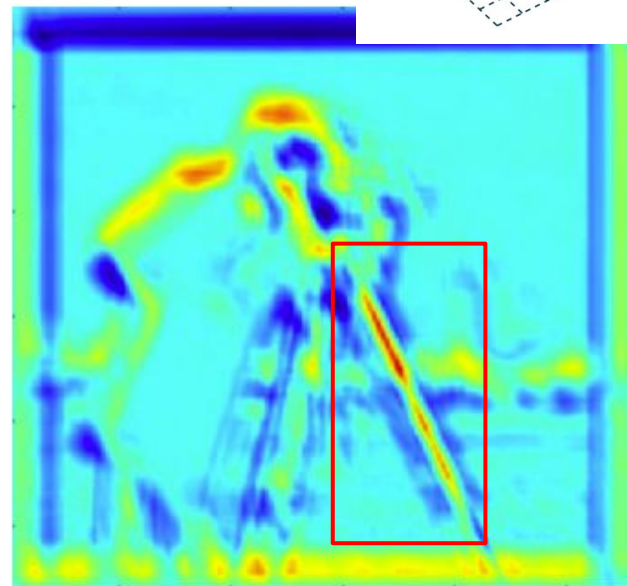
Input image

*



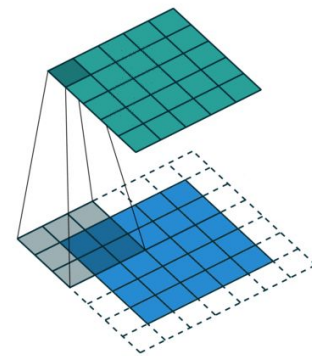
Convolution
kernel

=



Response map

Convolution In 2D - Example



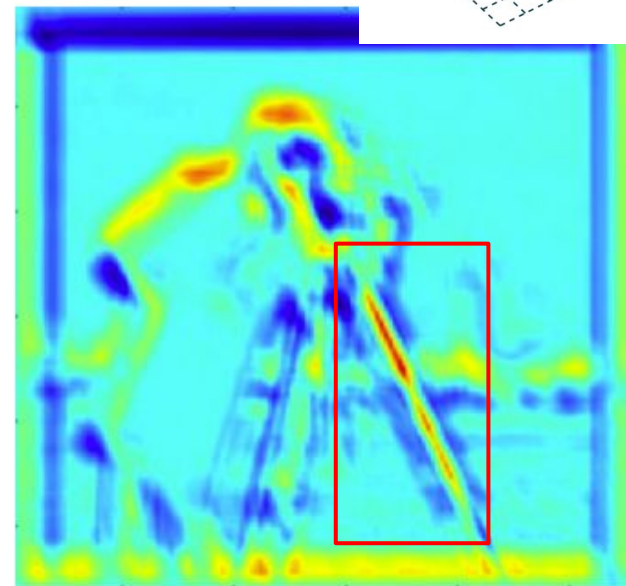
Input image

*



Convolution
kernel

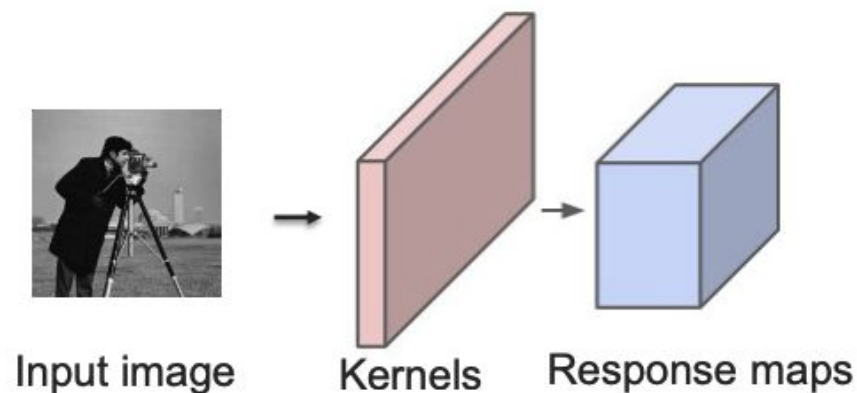
=



Response map

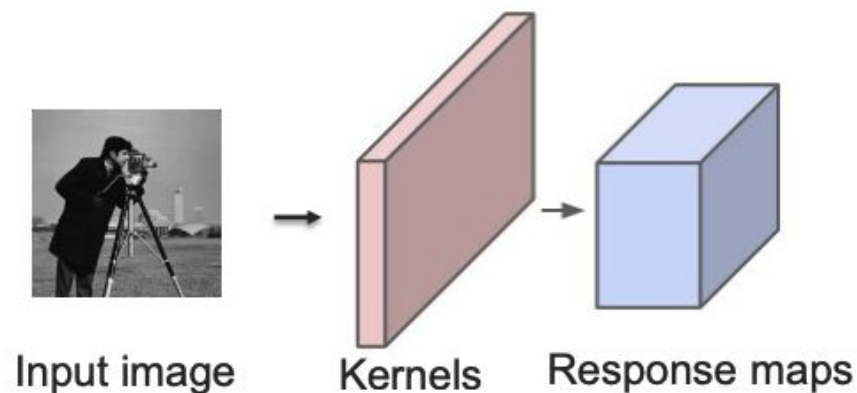
卷积神经层

- 可以扩展到二维（甚至三维！）
 - 只需保证正确的像素与正确的权重相连
- 也可以扩展到多通道二维
 - 例如 RGB 图像
- 还可以扩展到多个卷积核/滤波器
 - 输出不再是单张图，而是一个张量（三维矩阵）



Convolutional Neural Layer

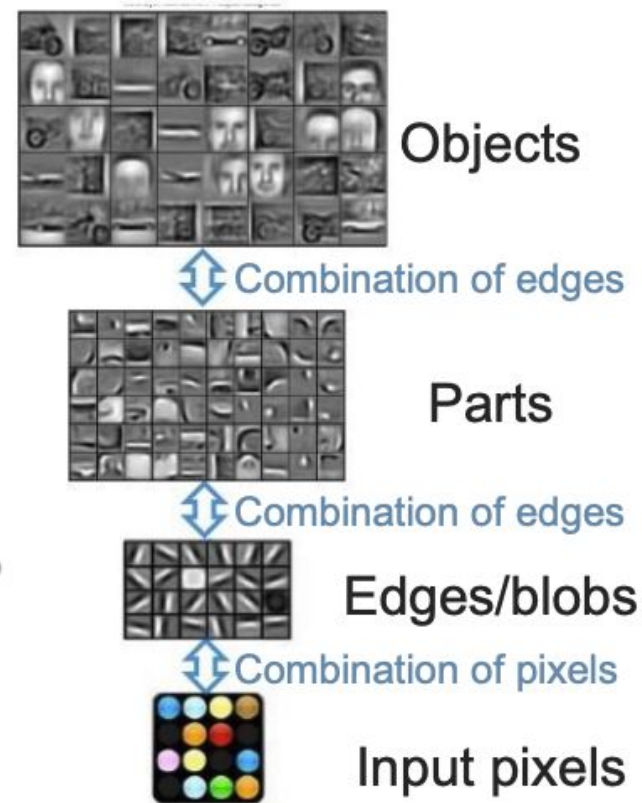
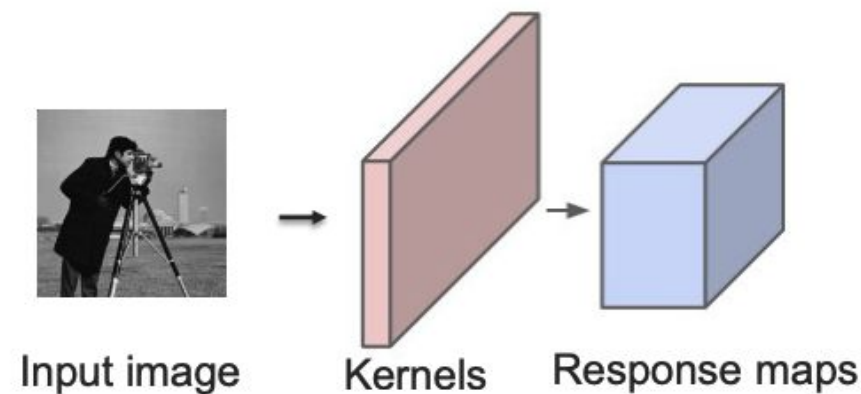
- Can expand this to 2D (or even 3D!)
 - Just need to make sure to link the right pixel with the right weight
- Can expand to multi-channel 2D
 - e.g., for RGB images
- Can expand to multiple kernels/filters
 - Output is not a single image anymore, but a tensor (a 3D matrix)



卷积神经网络

- 多层卷积

- 让网络学习局部组合, 从而增加复杂度

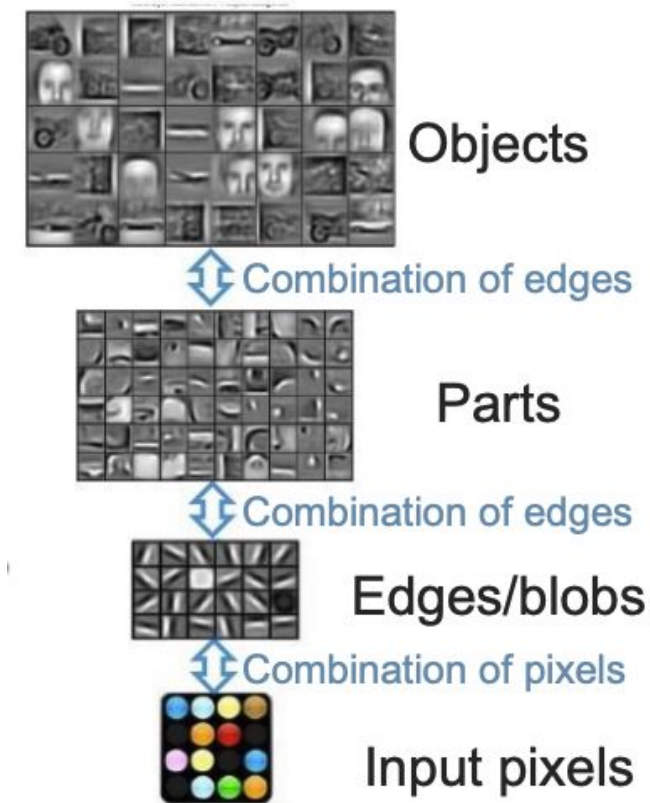
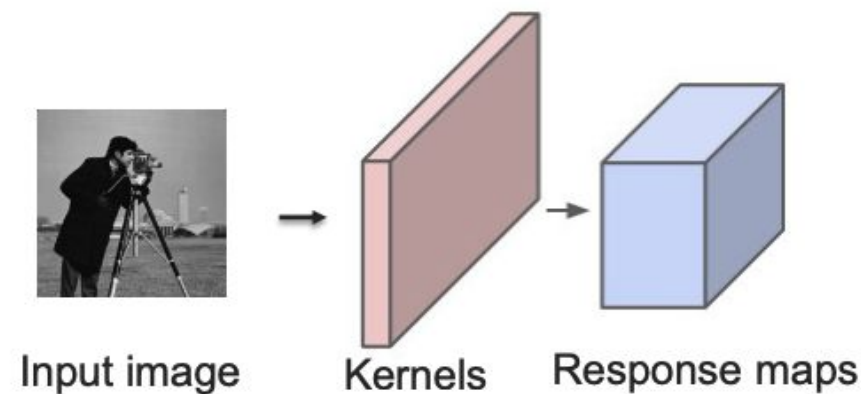


38

Convolutional Neural Network

- Multiple convolutional layers

- Allows the network to learn combinations of sub-parts, to increase complexity

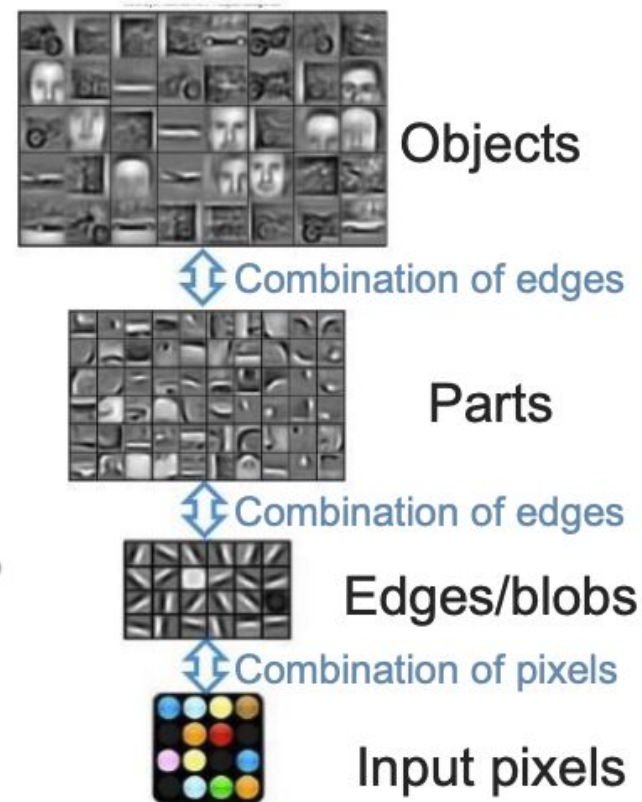
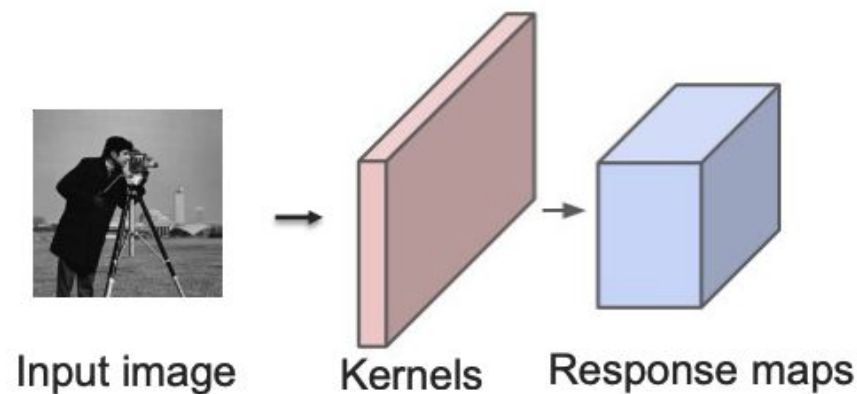


38

卷积神经网络

- 多层卷积
- 让网络学习局部组合, 从而增加复杂度
- 但如何促进抽象和汇总?

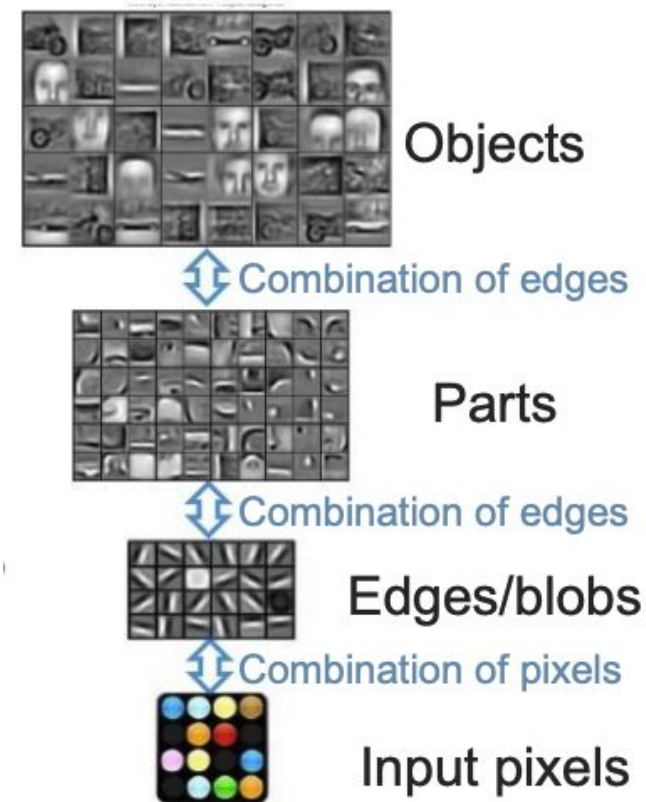
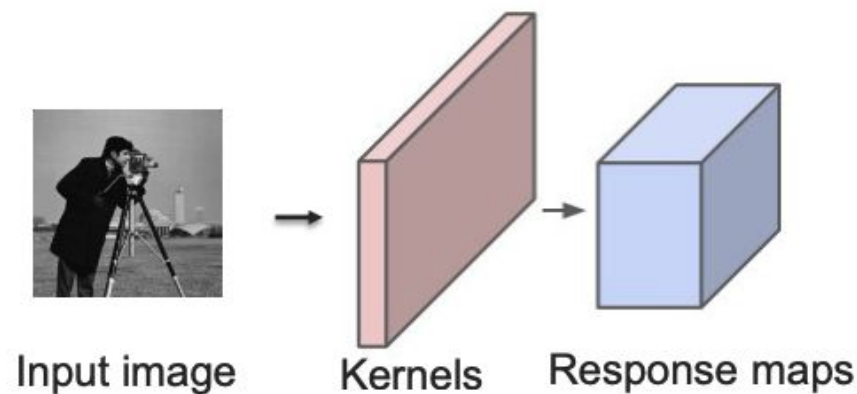
○ 池化层!



39

Convolutional Neural Network

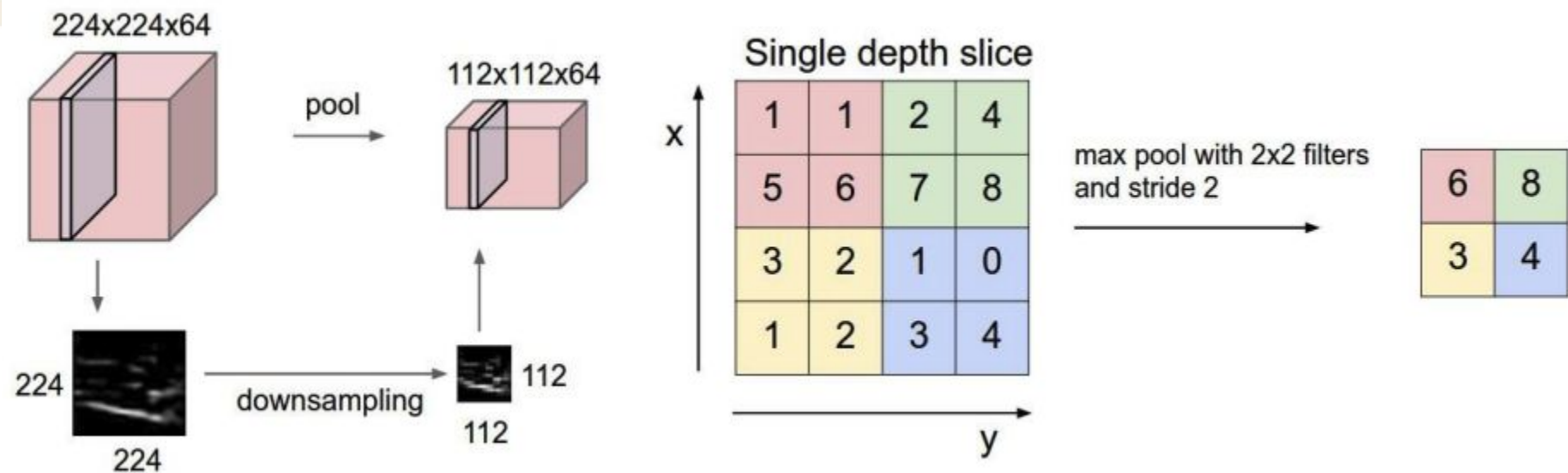
- Multiple convolutional layers
 - Allows the network to learn combinations of sub-parts, to increase complexity
- But how to encourage abstraction and summarization?
 - Pooling layer!



39

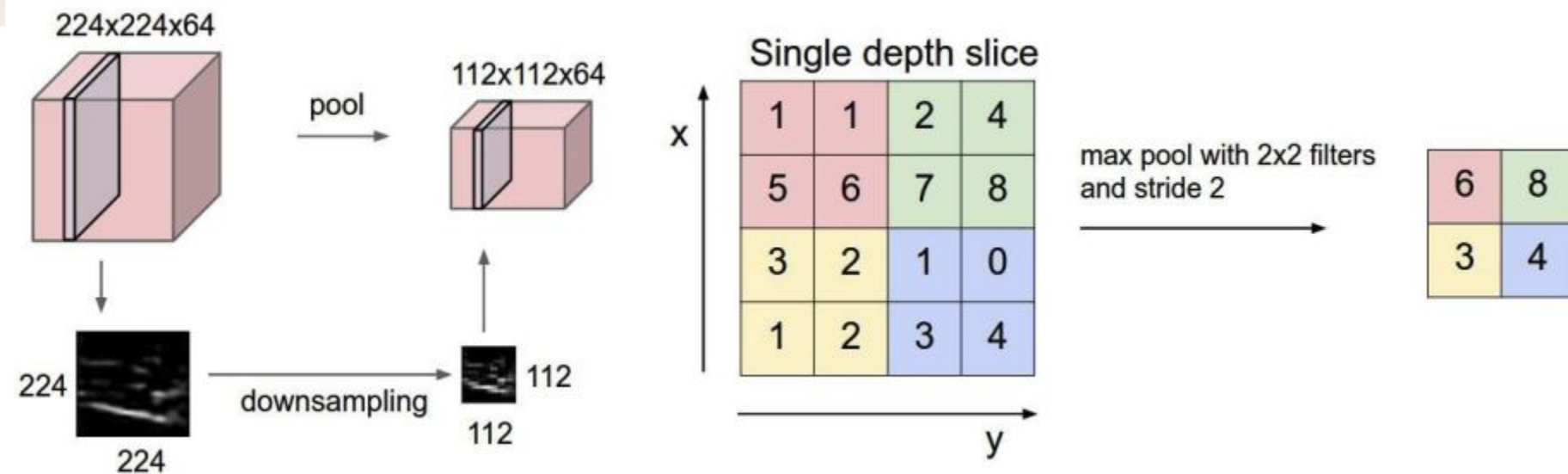
池化层

- 对响应图做下采样：
 - 用于汇总响应

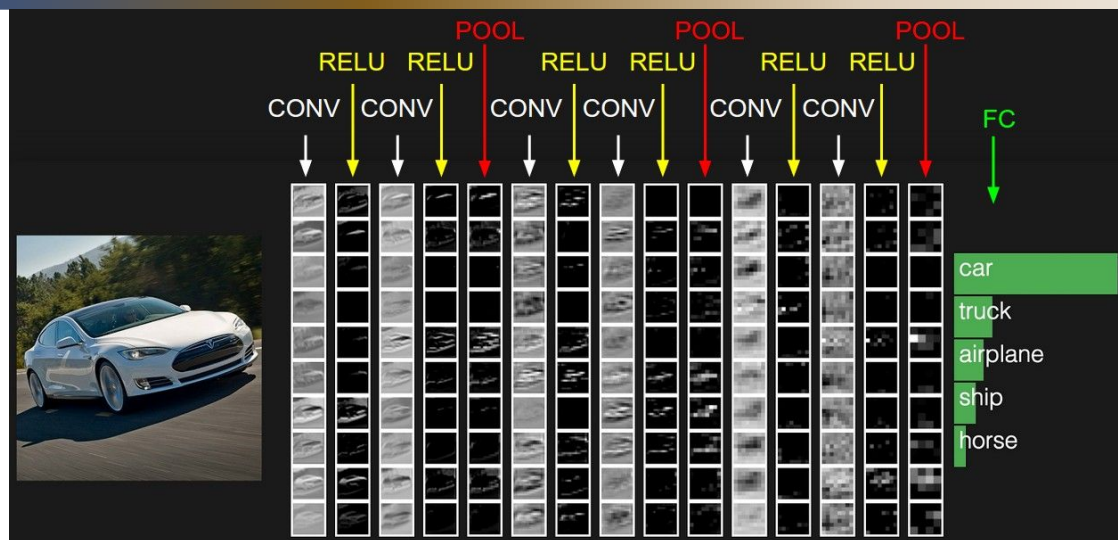


Pooling Layer

- Response map subsampling:
 - Allows summarization of the responses

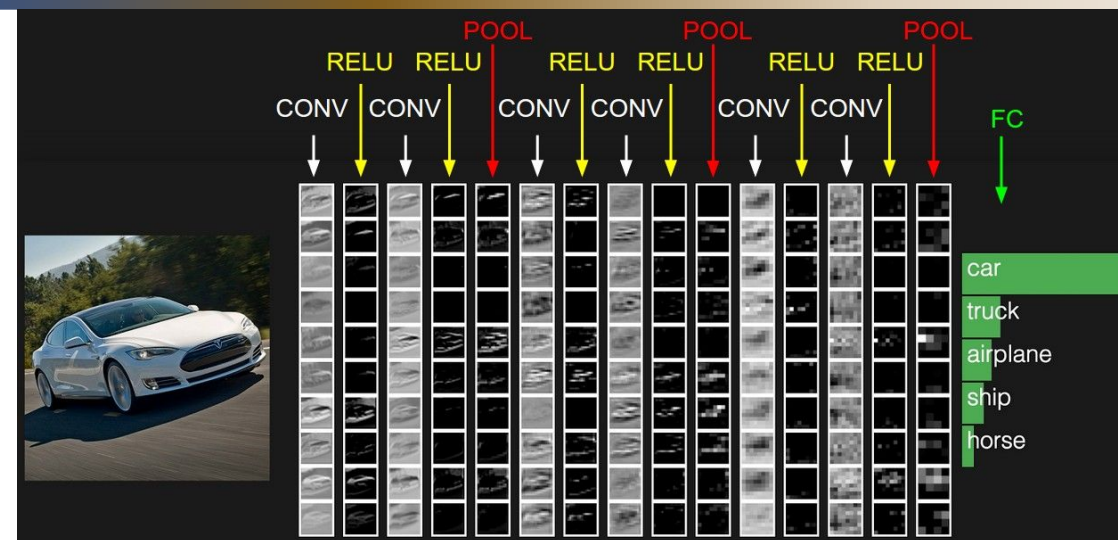


常见 CNN 结构



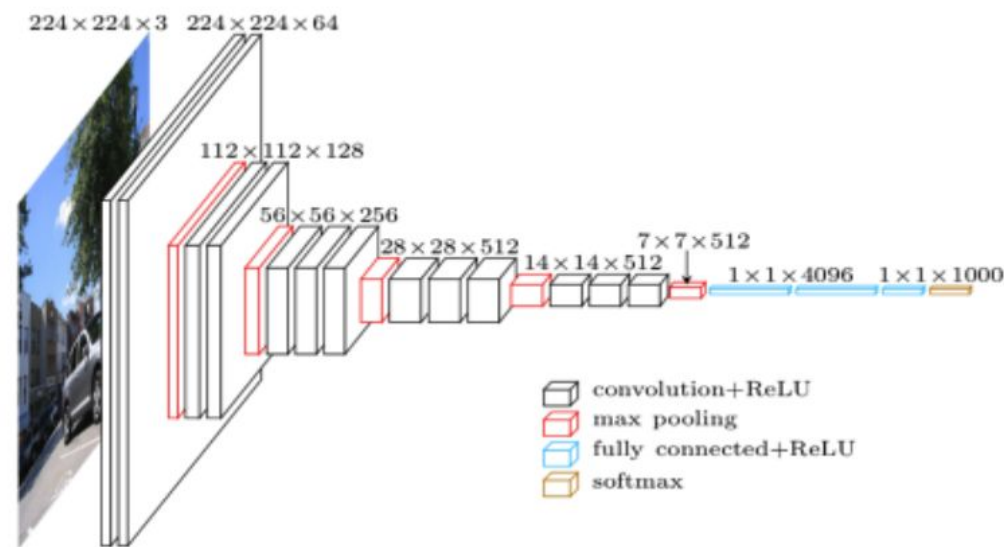
- 先用卷积层，再接非线性激活和池化
- 重复若干次
- 最后接全连接（MLP）层

Common CNN Architectures



- Start with a convolutional layer follow by non-linear activation and pooling
- Repeat this several times
- Ends with a fully connected (MLP) layer

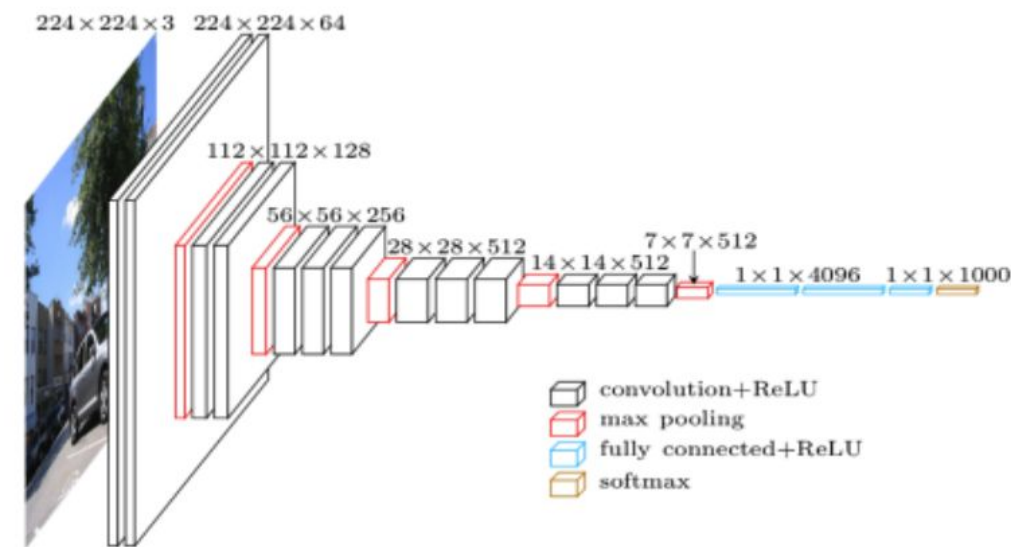
常见 CNN 结构



• VGGNet 模型

- 用于 1000 类图像分类任务
- 1.38 亿参数 / 16 - 19 层
- *因其连接简单、推理快，今天仍然有用

Common CNN Architectures

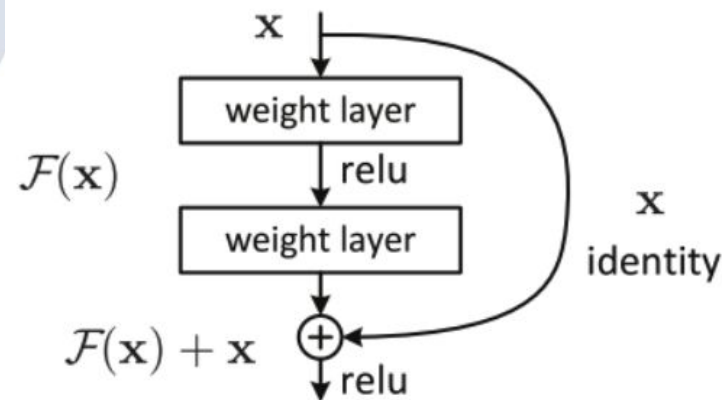


• VGGNet model

- Used for 1000-way image classification task
- 138 million parameters / 16-19 layers
- *Still relevant today due to its simple connection and fast inference

残差网络

- 加入残差连接

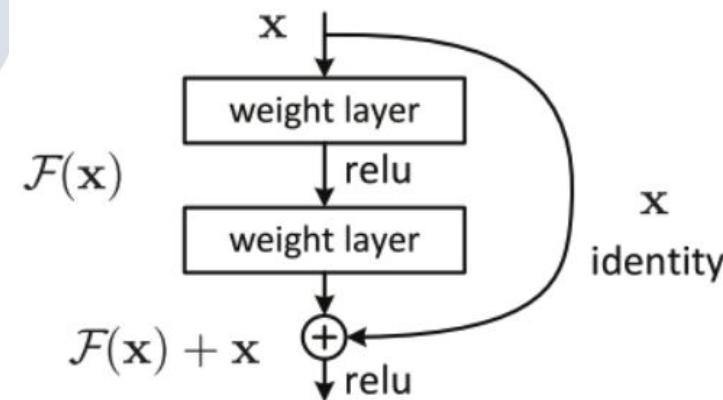


- ResNet (He et al., 2015)

- 最深可达 152 层
- 但只有 6000 万参数 (VGG 为 1.38 亿, 精度还高约 3%)

Residual Networks

- Adding residual connections



- ResNet (He et al., 2015)

- Up to 152 layers
- But only 60 million parameters (138M for VGG, 3% better)



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 关键模型
 - CNN
 - 稠密预测
 - 注意力模型
 - 大规模视觉-语言学习
 - 几何计算机视觉
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

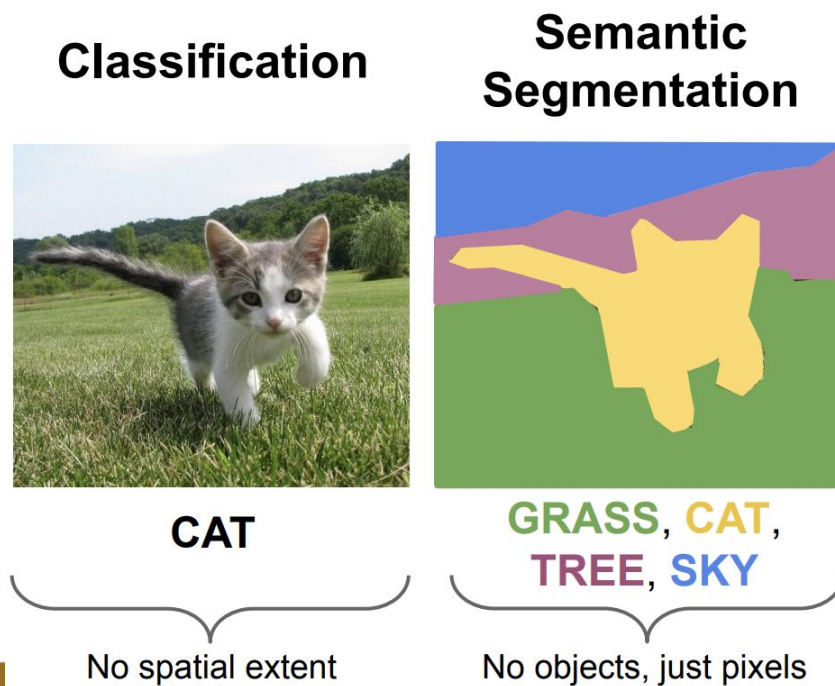
44

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

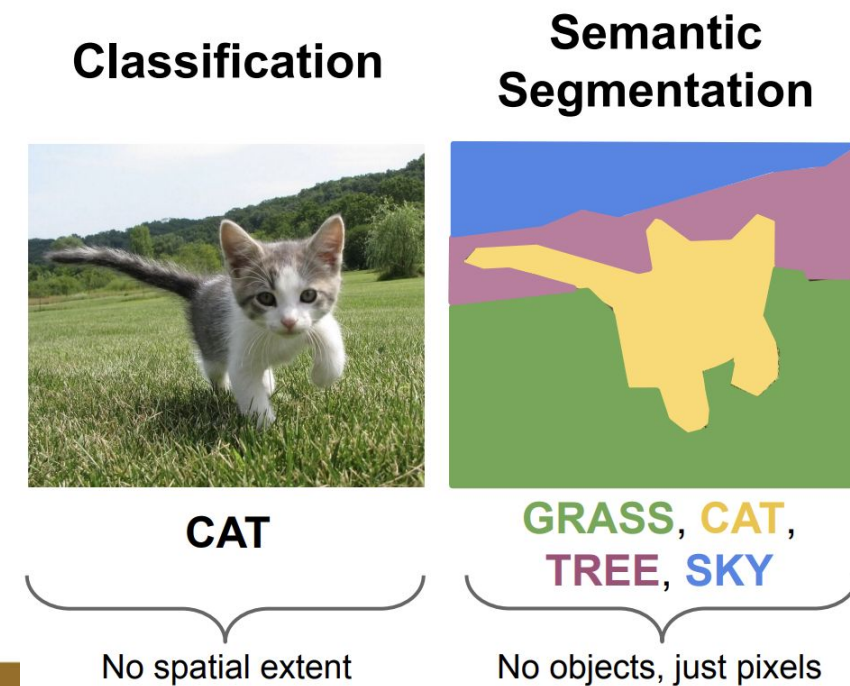
稠密预测

- 图像分类 vs 语义分割



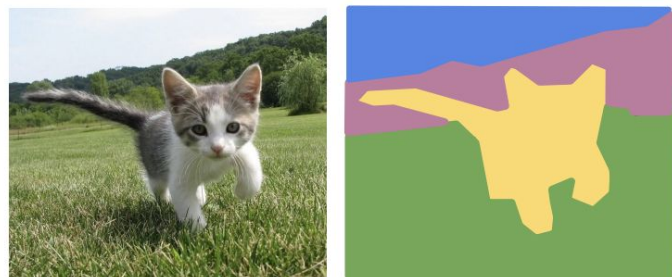
Dense Prediction

- Image classification vs semantic segmentation



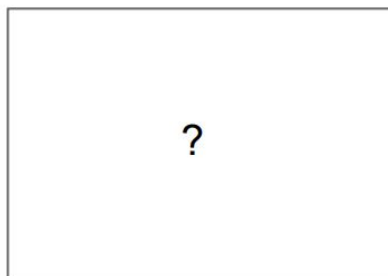
稠密预测 —— 语义分割

- 问题描述



GRASS, CAT,
TREE, SKY, ...

成对训练数据：每张
训练图像的每个像素都标有
一个语义类别。



测试时，对一张新图像的
每个像素分类

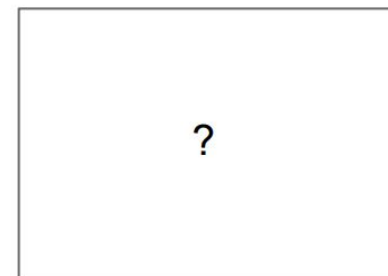
Dense Prediction - Semantic Segmentation

- Problem description



GRASS, CAT,
TREE, SKY, ...

Paired training data: for each
training image, each pixel is labeled
with a semantic category.

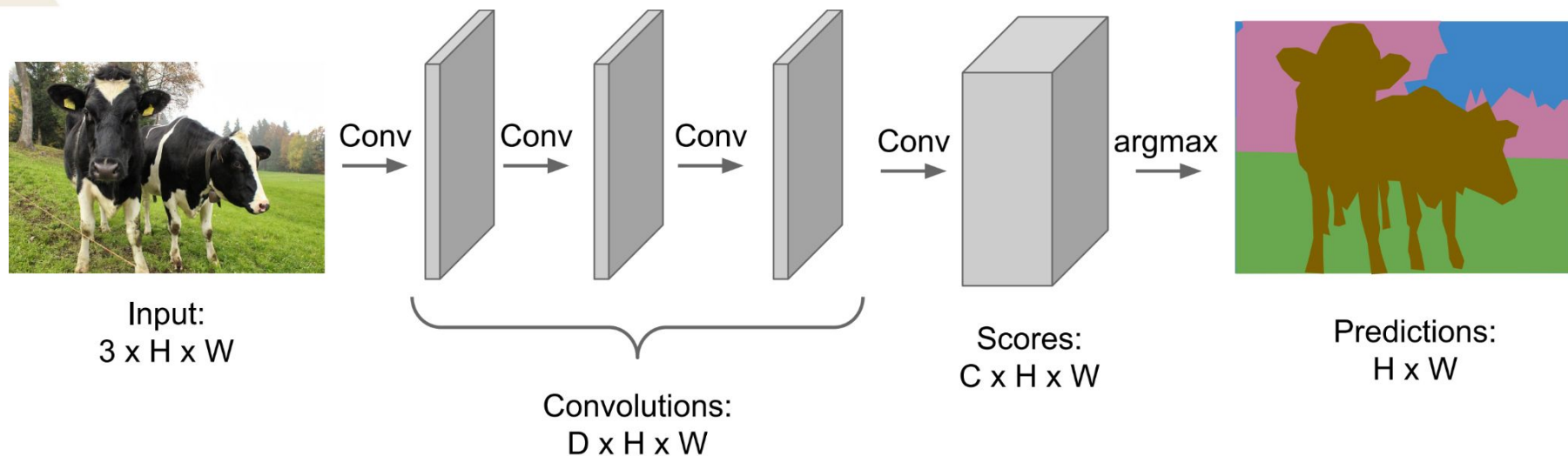


At test time, classify each pixel of a new
image

稠密预测 —— 语义分割

- 思路：卷积

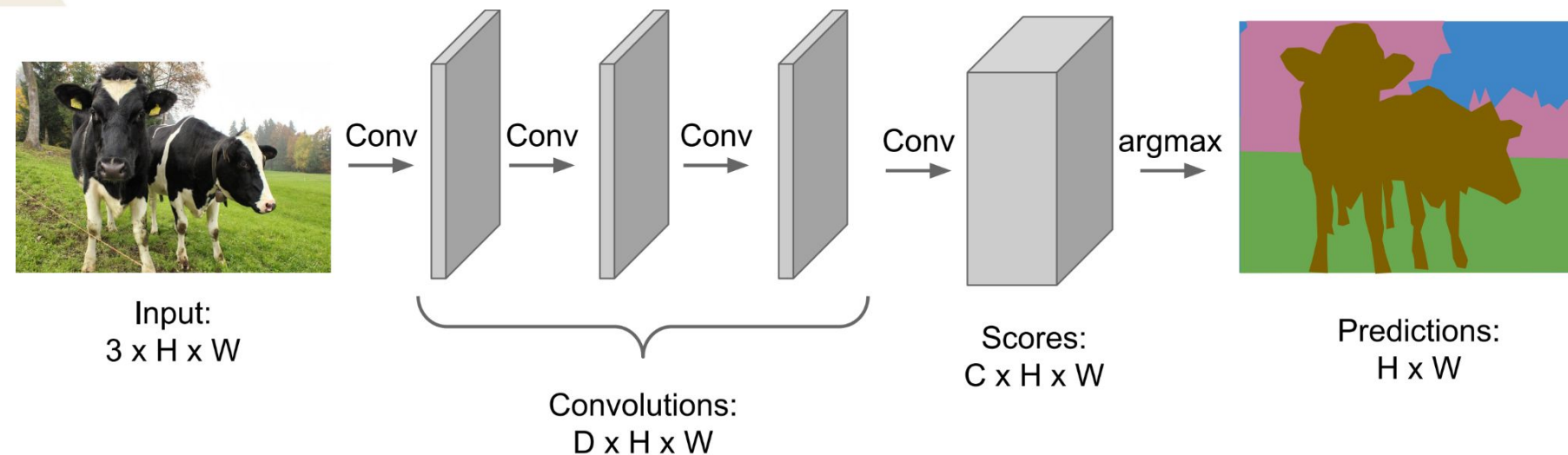
- 我们看到卷积可以得到相同尺寸的输出
- 因此可以设计一个只有卷积层、没有下采样池化的网络，一次性对所有像素做预测！



Dense Prediction - Semantic Segmentation

- Idea: Convolution

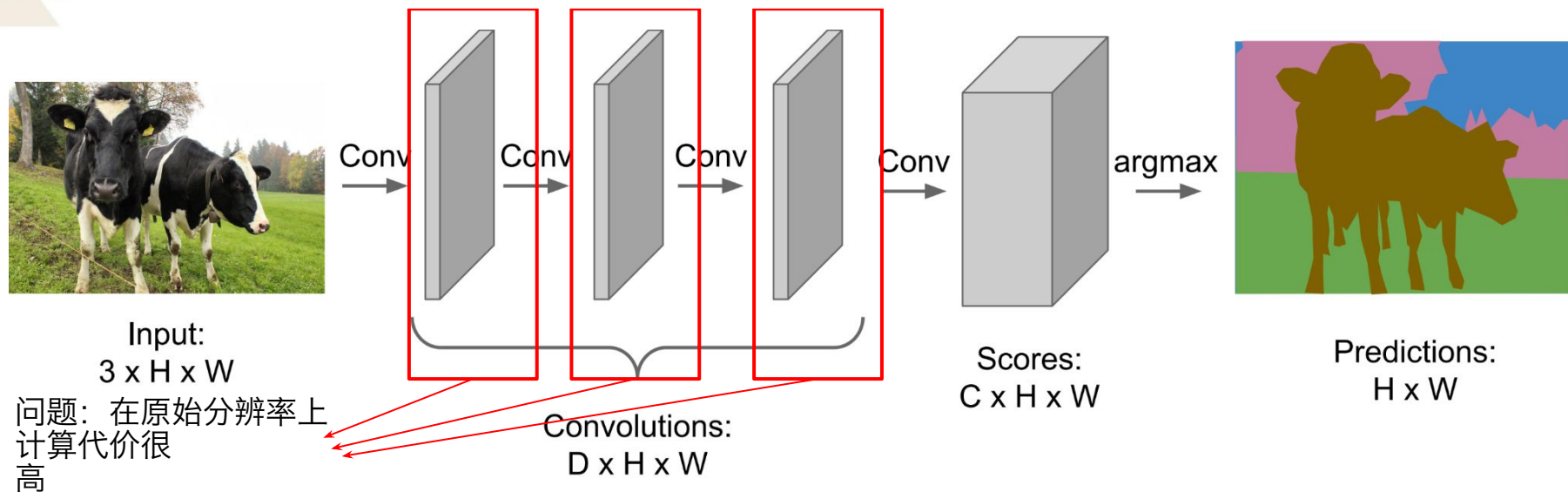
- We see that convolution can get same-sized outputs
- So design a network with only convolutional layers without downsampling poolings to make predictions for pixels all at once!



稠密预测 —— 语义分割

- 思路：卷积

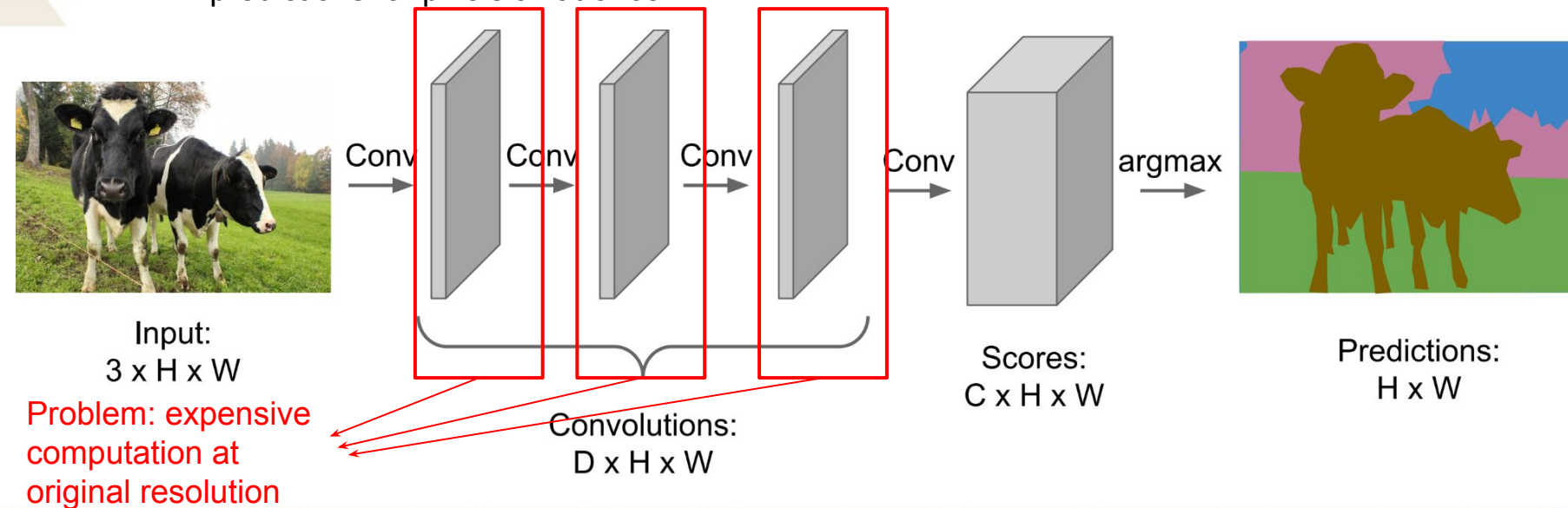
- 我们看到卷积可以得到相同尺寸的输出
- 因此可以设计一个只有卷积层、没有下采样池化的网络，一次性对所有像素做预测！



Dense Prediction - Semantic Segmentation

- Idea: Convolution

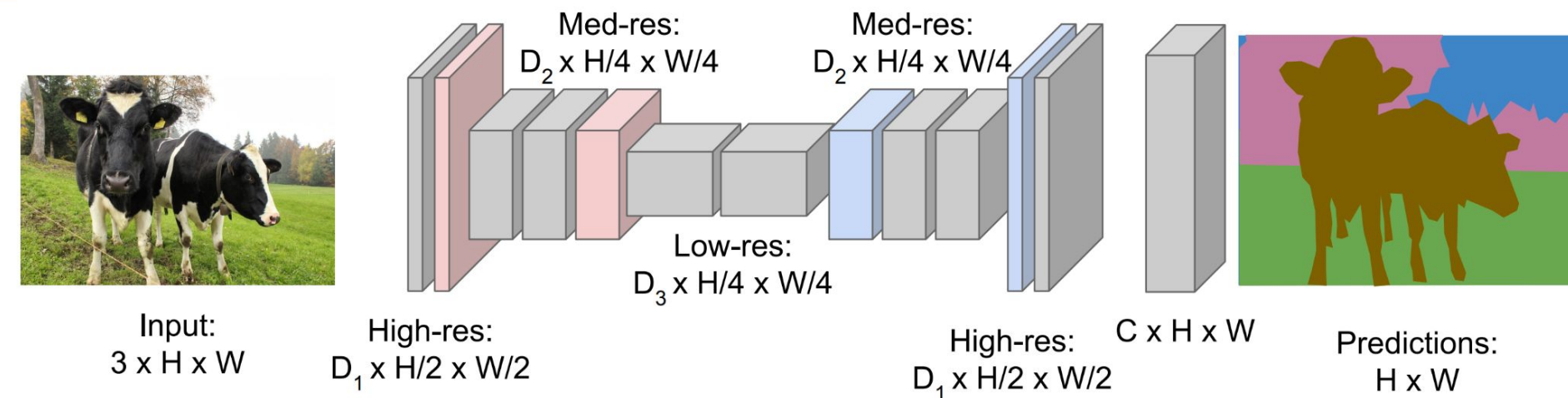
- We see that convolution can get same-sized outputs
- So design a network with only convolutional layers without downsampling poolings to make predictions for pixels all at once!



稠密预测 —— 语义分割

- 思路：卷积网络

- 我们看到卷积可以得到相同尺寸的输出
- 把网络设计成一堆卷积层，并在网络内部做下采样和上采样！

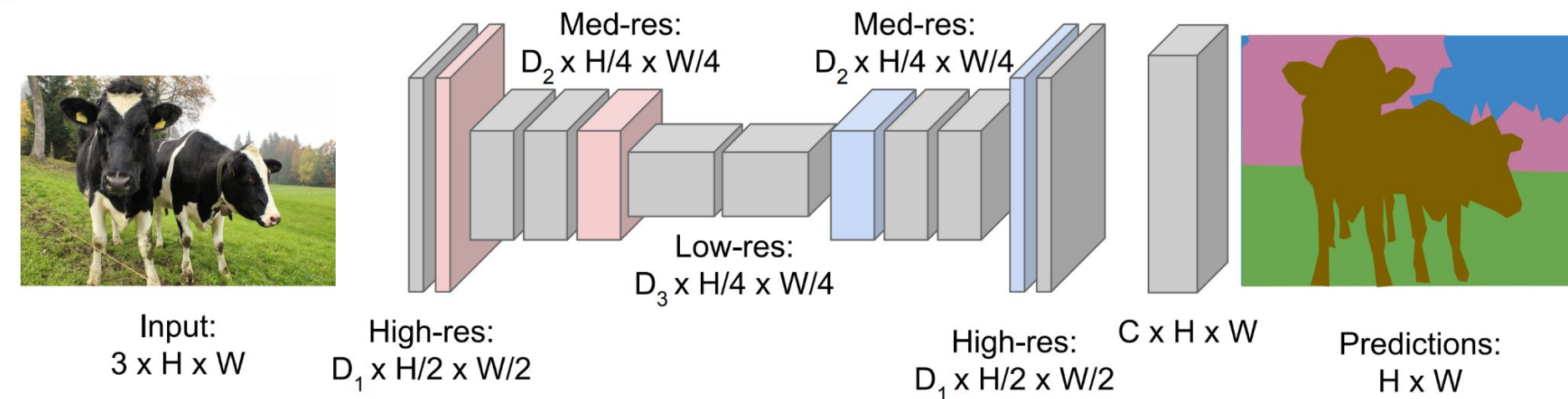


* Fully Convolutional Networks for Semantic Segmentation. CVPR 2015

Dense Prediction - Semantic Segmentation

- Idea: convolutional network

- We see that convolution can get same-sized outputs
- Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!

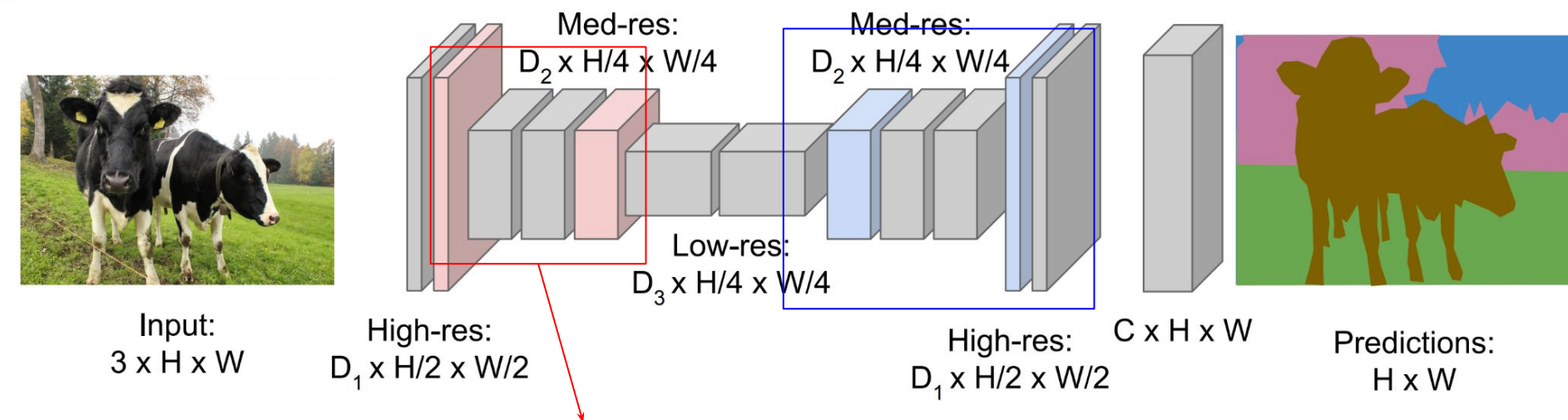


* Fully Convolutional Networks for Semantic Segmentation. CVPR 2015

稠密预测 —— 语义分割

- 思路：卷积网络

- 我们看到卷积可以得到相同尺寸的输出
- 把网络设计成一堆卷积层，并在网络内部做下采样和上采样！

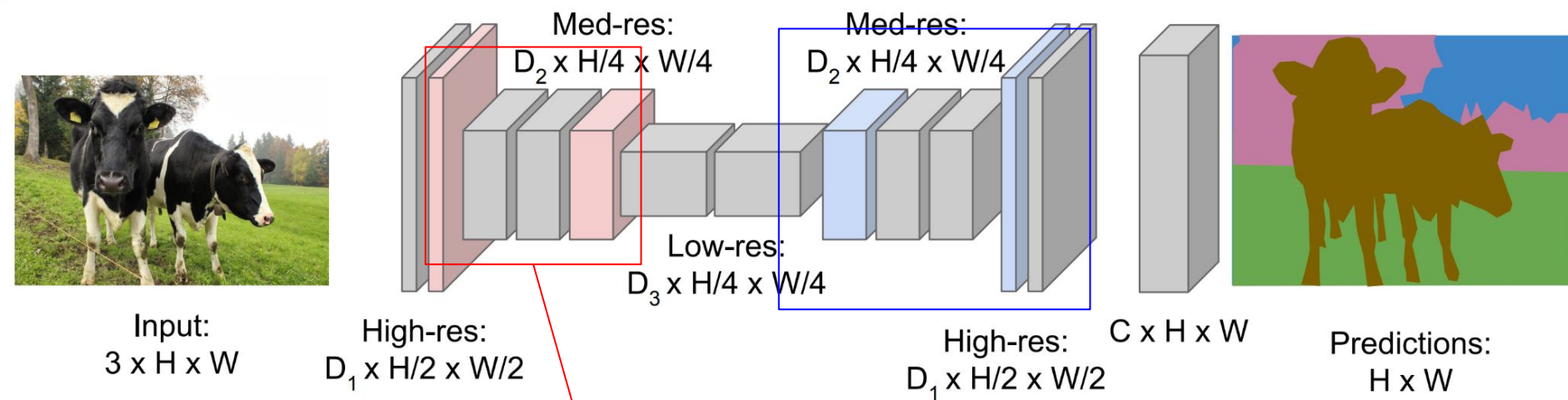


50

Dense Prediction - Semantic Segmentation

- Idea: convolutional network

- We see that convolution can get same-sized outputs
- Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!



Downsampling: We could use pooling, stride > 1 convolution

Upsampling: ?

50

稠密预测 —— 语义分割

- 上采样: 反池化 (Unpooling)

Nearest Neighbor

1	2
3	4

Input: 2 x 2

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Output: 4 x 4

“Bed of Nails”

1	2
3	4

Input: 2 x 2

1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Output: 4 x 4

把值放到固定位置，其余
填零

Dense Prediction - Semantic Segmentation

- Upsampling: Unpooling operation

Nearest Neighbor

1	2
3	4

Input: 2 x 2

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Output: 4 x 4

“Bed of Nails”

1	2
3	4

Input: 2 x 2

1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

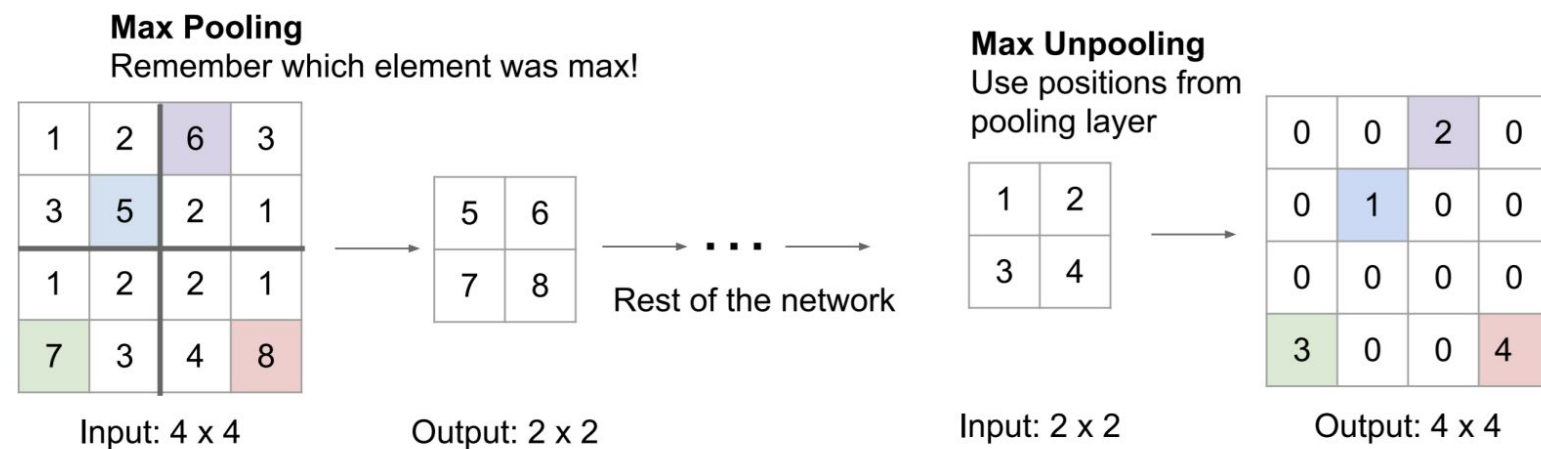
Output: 4 x 4

Put value at a fixed position and fill
the rest with zeros

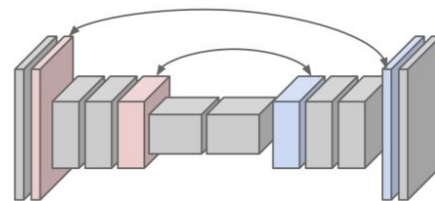
稠密预测 —— 语义分割

- 上采样：最大反池化 (Max Unpooling)

因为分割模型常常是对称的



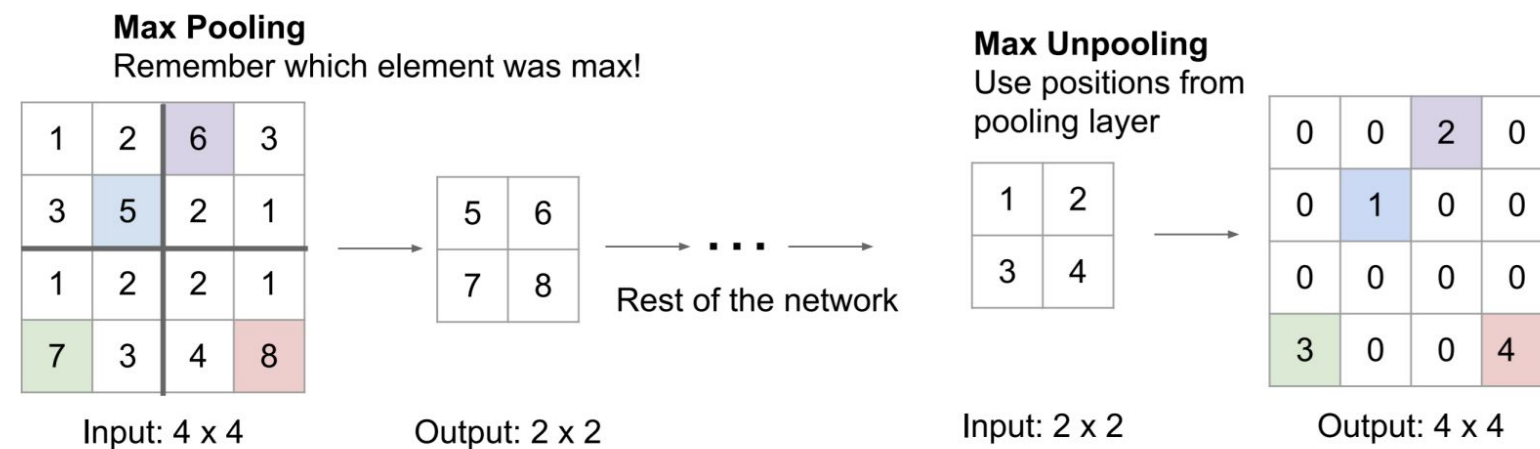
Corresponding pairs of
downsampling and
upsampling layers



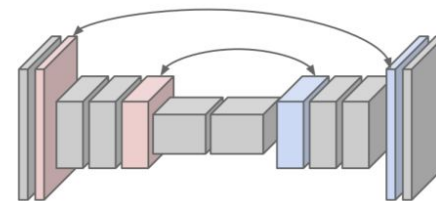
Dense Prediction - Semantic Segmentation

- Upsampling: Max Unpooling

Because segmentation models are often symmetric

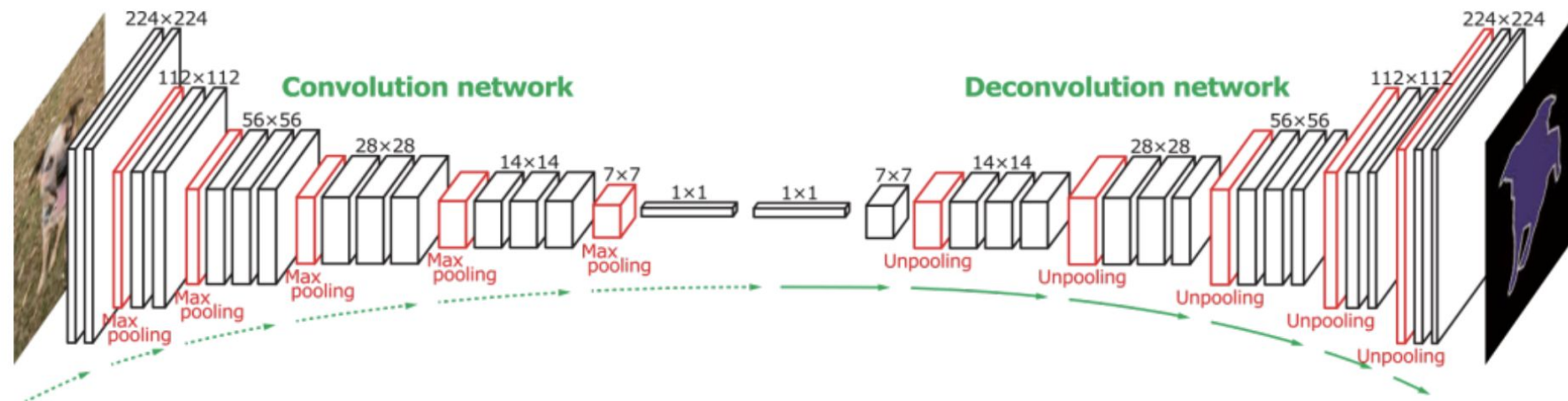


Corresponding pairs of
downsampling and
upsampling layers



稠密预测 —— 语义分割

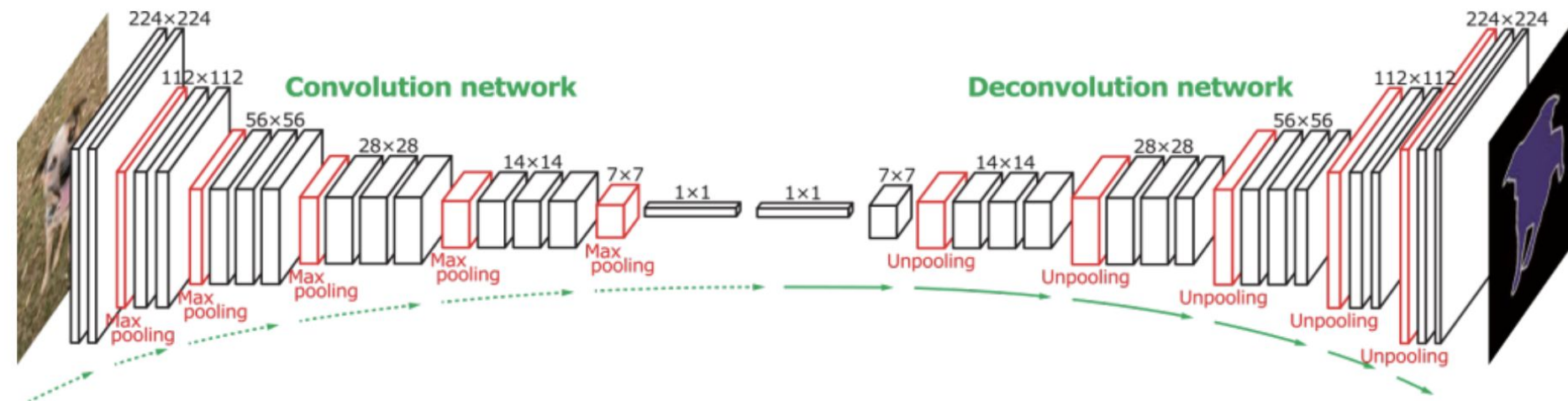
- 上采样：最大反池化 (Max Unpooling)



*Learning deconvolution network for semantic segmentation. ICV 2015

Dense Prediction - Semantic Segmentation

- Upsampling: Max Unpooling



*Learning deconvolution network for semantic segmentation. ICV 2015

稠密预测 —— 语义分割

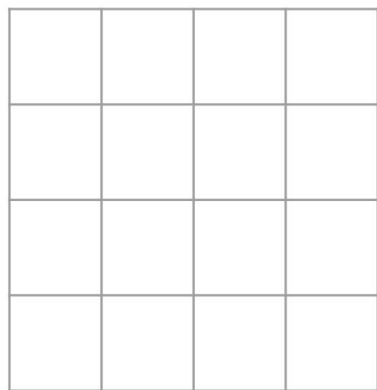
- 上采样: 转置卷积

- 回顾 —— 普通卷积

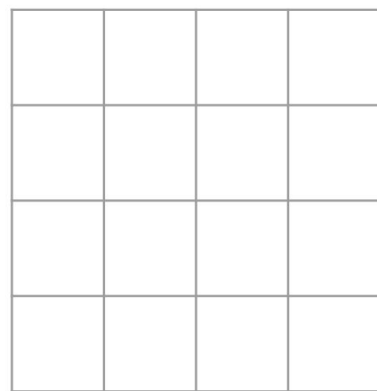
- 卷积核 (3 x 3), stride=1, pad=1 (输入 4x4 变成 6x6, 所以输出大小 $6-3+1=4 \times 4$)

0	1	1
0	0	0
-1	1	1

卷积核



Input: 4 x 4



Output: 4 x 4

Dense Prediction - Semantic Segmentation

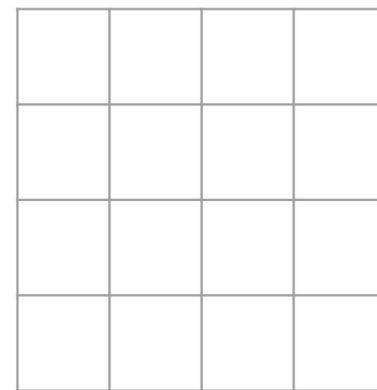
- Upsampling: Transposed Convolution

- Recall - normal convolution

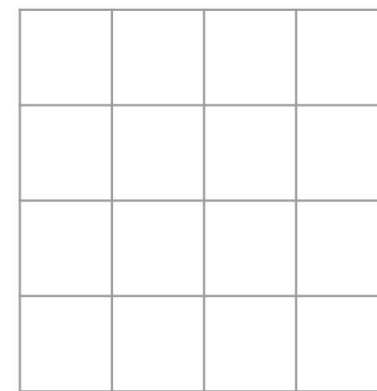
- Kernel (3 x 3), stride=1, pad=1 (input 4x4 becomes 6x6, so output size $6-3+1=4 \times 4$)

0	1	1
0	0	0
-1	1	1

Kernel



Input: 4 x 4

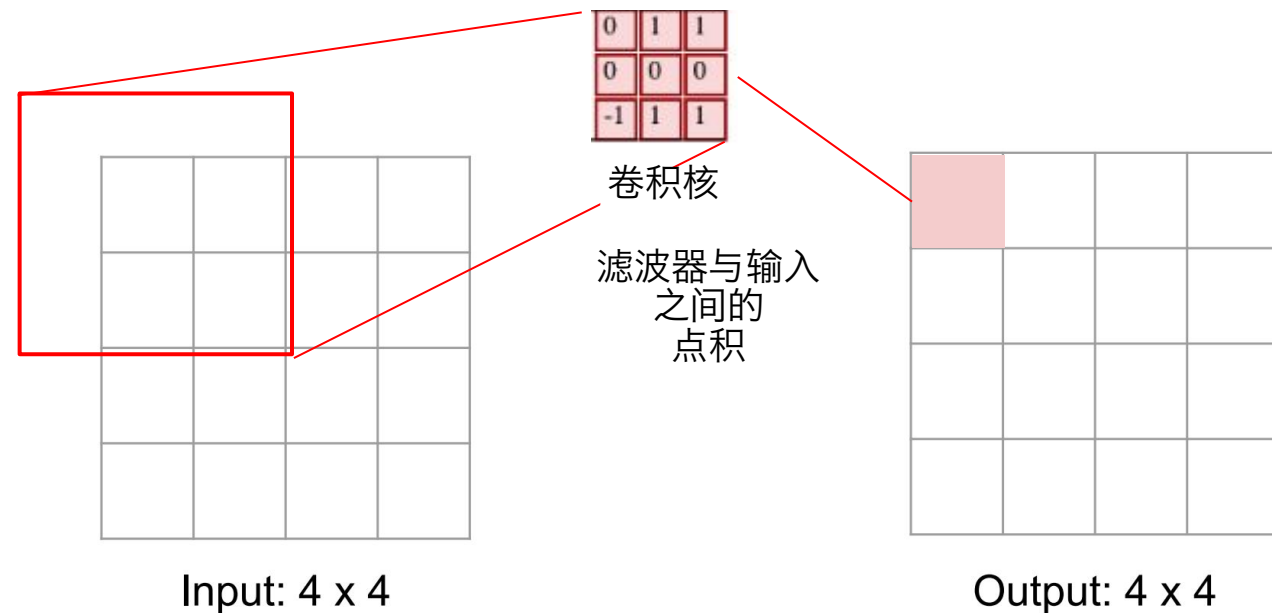


Output: 4 x 4

稠密预测 —— 语义分割

- 上采样: 转置卷积

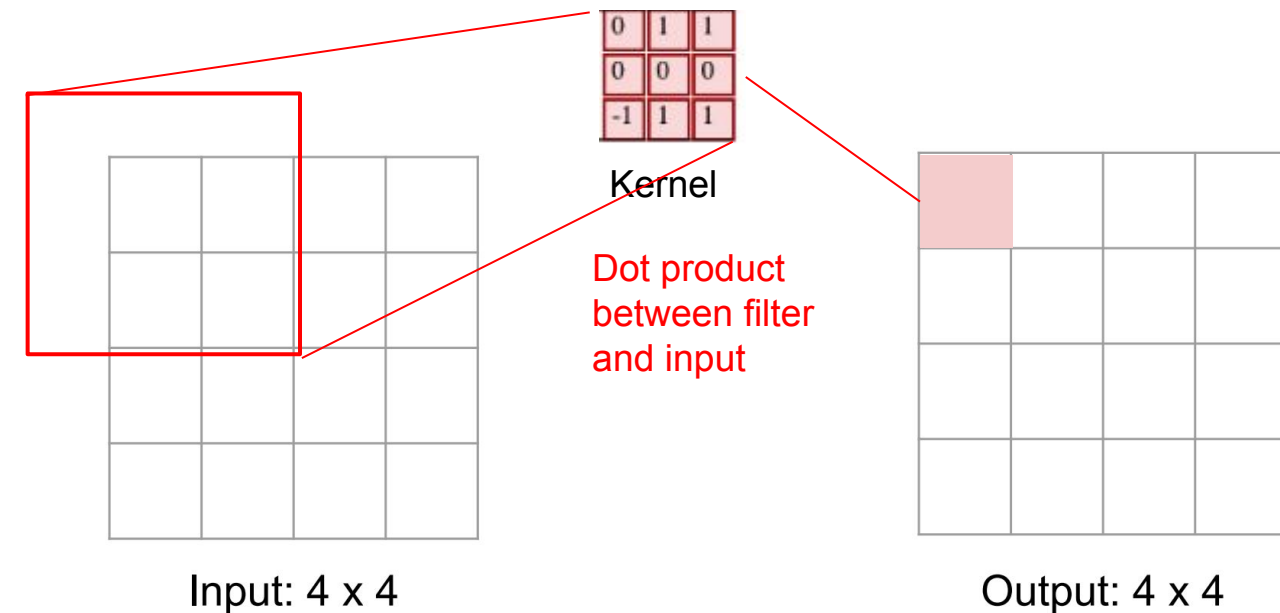
- 回顾 —— 普通卷积
- 卷积核 (3 x 3), stride=1, pad=1



Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

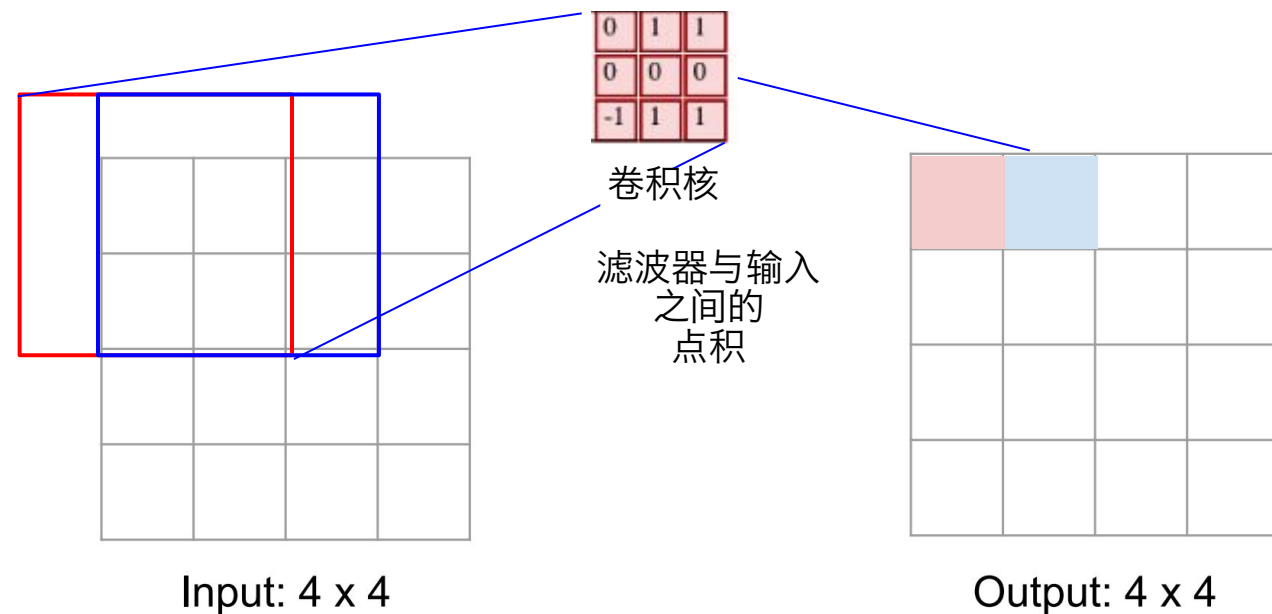
- Recall - normal convolution
- Kernel (3 x 3), stride=1, pad=1



稠密预测 —— 语义分割

- 上采样：转置卷积

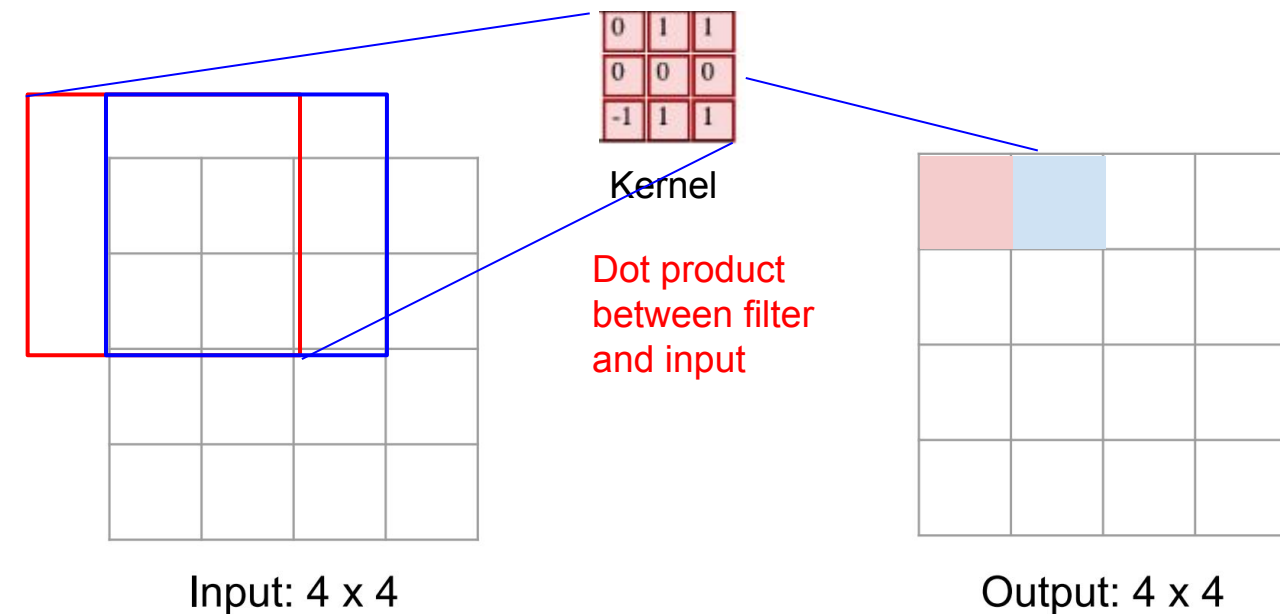
- 回顾 —— 普通卷积
- 卷积核 (3 x 3), stride=1, pad=1



Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

- Recall - normal convolution
- Kernel (3 x 3), stride=1, pad=1

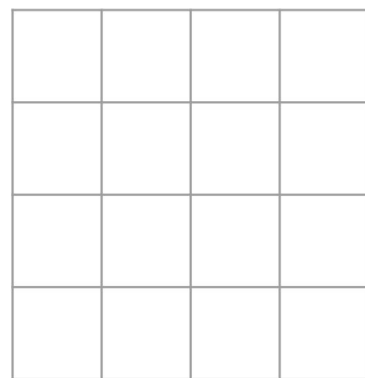


稠密预测 —— 语义分割

- 上采样: 转置卷积

- 回顾 —— 普通卷积
- 卷积核 (3 x 3), stride=2, pad=1

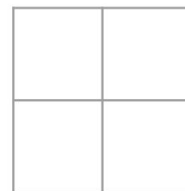
- 滤波器在输入上每移动 2 个像素, 输出才移动 1 个像素
- 输出通过可学习权重被 “下采样”



Input: 4 x 4



卷积核

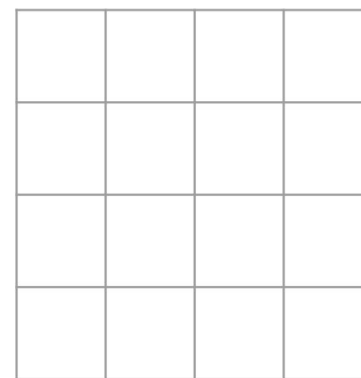


Output: 2 x 2

Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

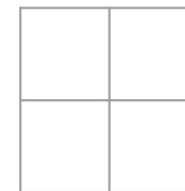
- Recall - normal convolution
- Kernel (3 x 3), **stride=2**, pad=1
 - Filter moves 2 pixels in the input for every one pixel in the output
 - Output is “downsampled” using learnable weights



Input: 4 x 4



Kernel

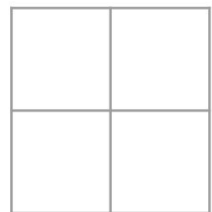


Output: 2 x 2

稠密预测 —— 语义分割

- 上采样：转置卷积

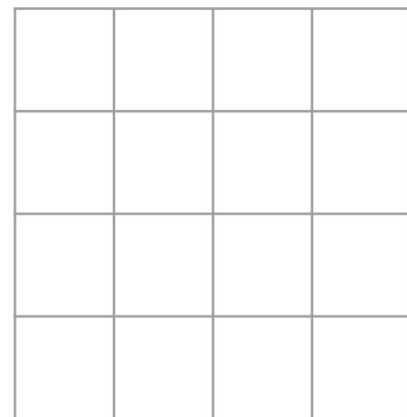
- (3 x 3) 转置卷积, stride=2, pad=1: 输出被上采样!



Input: 2 x 2

0	1	1
0	0	0
-1	1	1

卷积核

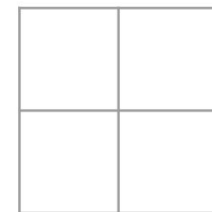


Output: 4 x 4

Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

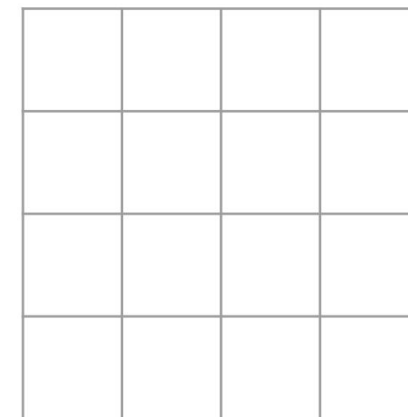
- (3 x 3) **transposed convolution**, stride=2, pad=1: Output is upsampled!



Input: 2 x 2

0	1	1
0	0	0
-1	1	1

Kernel

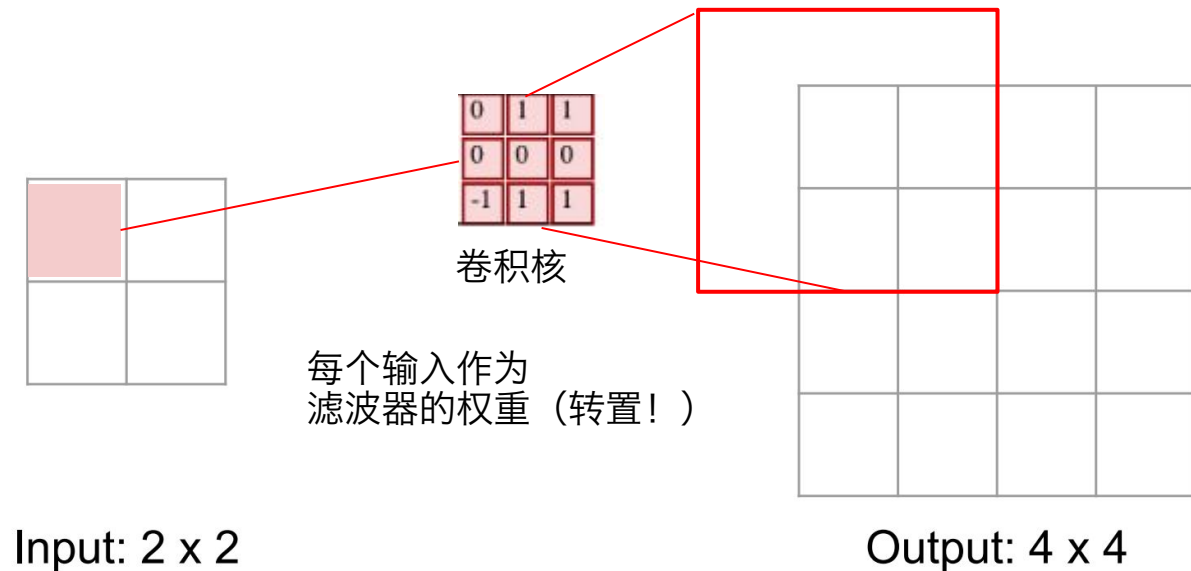


Output: 4 x 4

稠密预测 —— 语义分割

- 上采样: 转置卷积

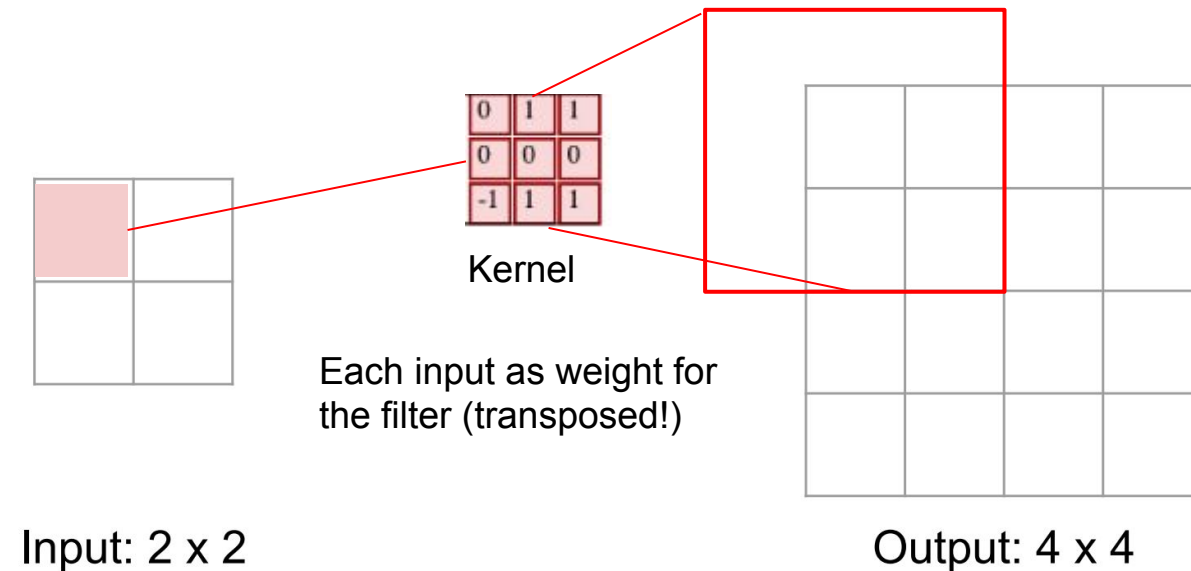
- (3 x 3) 转置卷积, stride=2, pad=1



Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

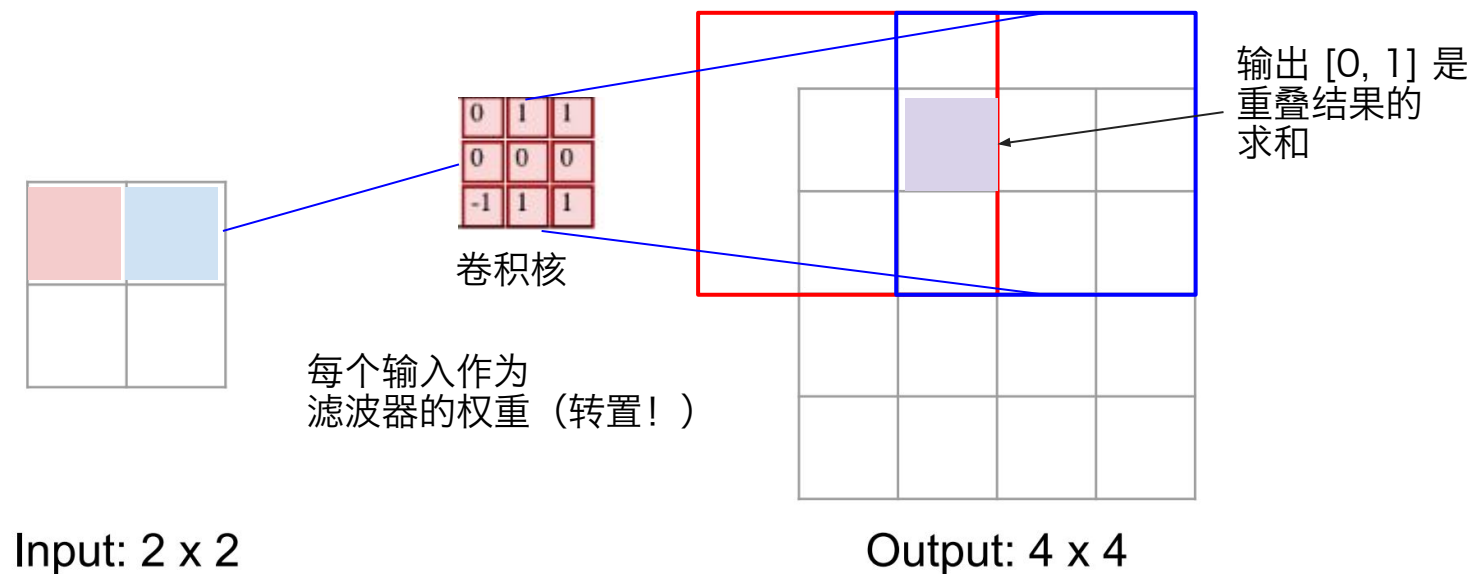
- (3 x 3) **transposed convolution**, stride=2, pad=1



稠密预测 —— 语义分割

- 上采样: 转置卷积

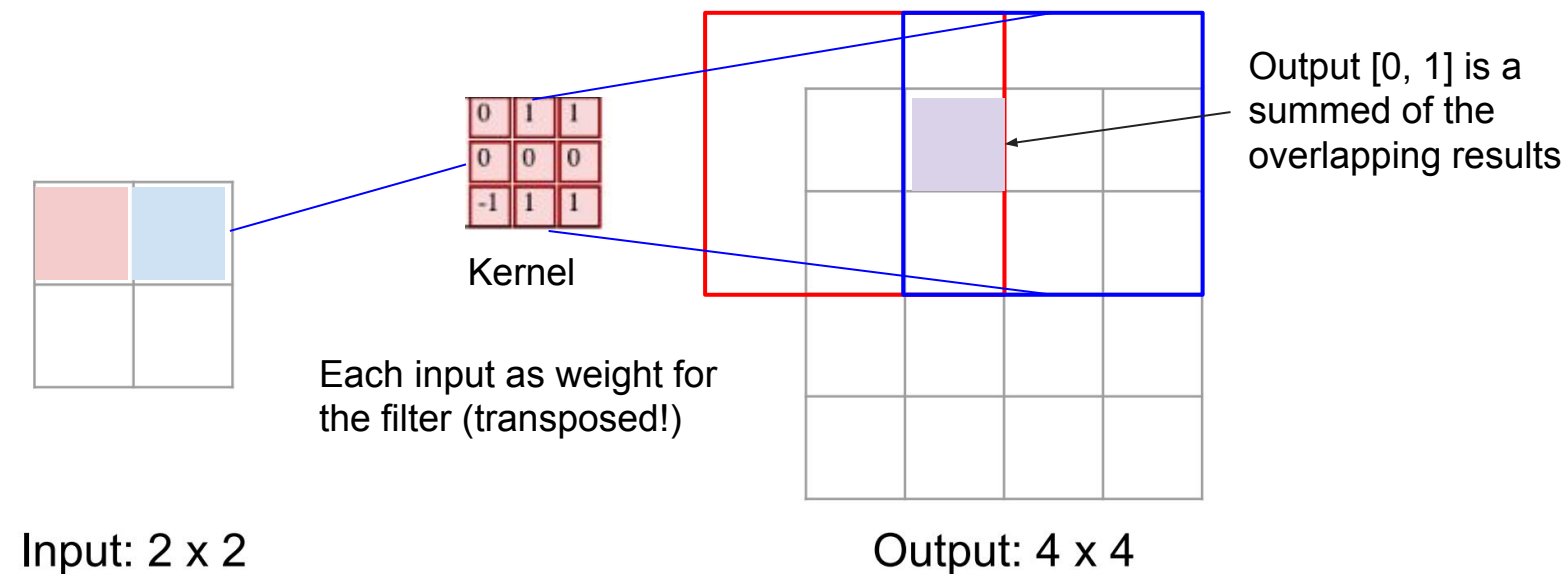
- (3 x 3) 转置卷积, stride=2, pad=1



Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

- (3 x 3) **transposed convolution**, stride=2, pad=1

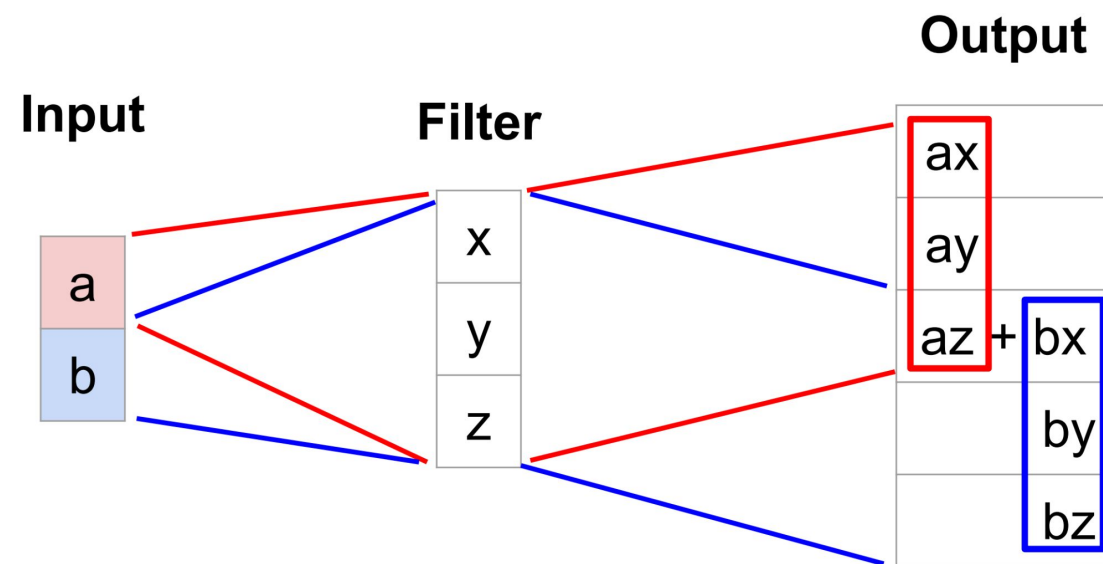


稠密预测 —— 语义分割

- 上采样：转置卷积

- 一维例子

- 输出包含按输入加权的滤波器副本，在输出中重叠的位置再相加

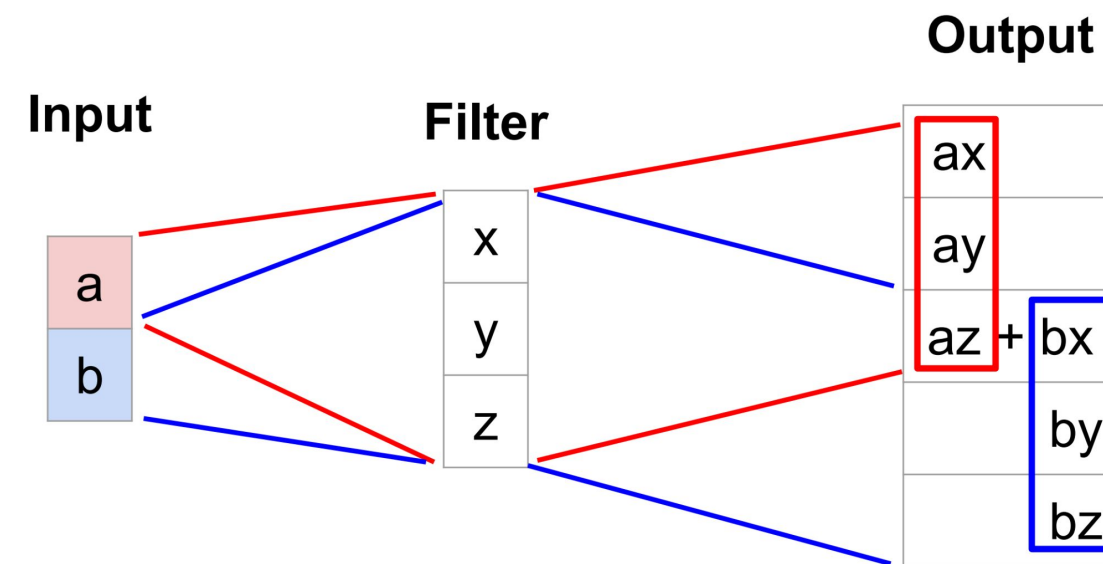


Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

- 1D example

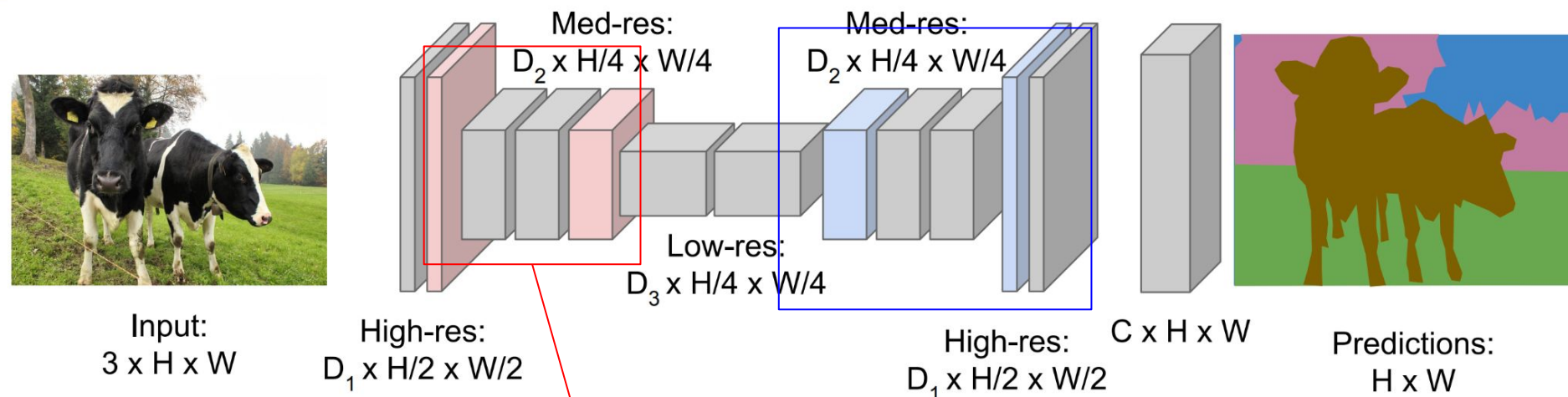
- Output contains copies of the filter weighted by the input, summing at where at overlaps in the output



稠密预测 —— 语义分割

- 思路：卷积网络

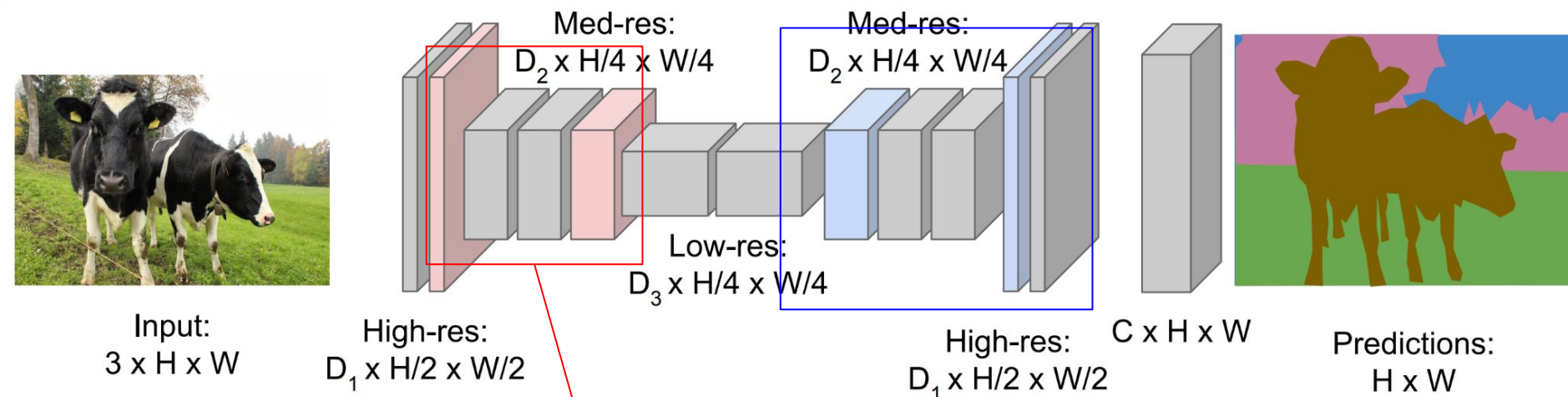
- 我们看到卷积可以得到相同尺寸的输出
- 把网络设计成一堆卷积层，并在网络内部做下采样和上采样！



Dense Prediction - Semantic Segmentation

- Idea: convolutional network

- We see that convolution can get same-sized outputs
- Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!



Downsampling: We could use pooling, stride > 1 convolution

Upsampling: Unpooling or strided transposed convolution

稠密预测 —— 语义分割

- 评价

- 数据集: Cityscapes、ADE20K
- 指标: m-IOU (平均交并比)

■ mIoU 是测试集所有图像上, 各分割目标 IoU 的平均值。



$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Dense Prediction - Semantic Segmentation

- Evaluation

- Datasets: Cityscapes, ADE20K
- Metric: m-IOU (mean Intersection over Union)
 - The mIoU is the average between the IoU of the segmented objects over all the images of the test dataset.



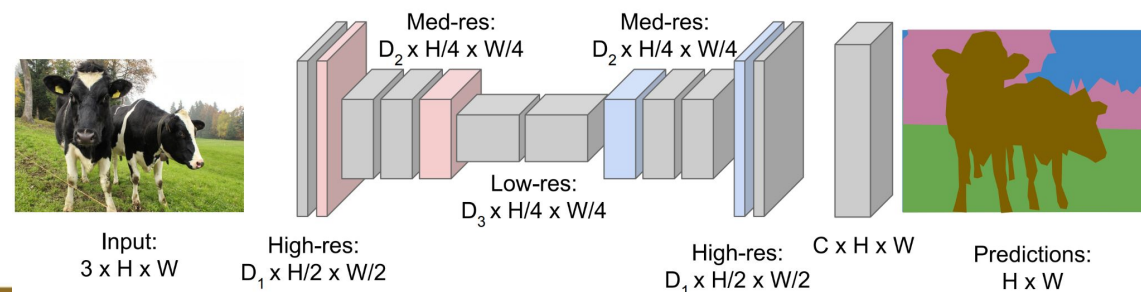
$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

稠密预测 —— 语义分割

• 小结

- 任务：给图像中每个像素打上类别标签
- 设计：卷积编码器 + 反卷积解码器
- 反卷积：做机器学习的人不太喜欢这个名字
- 更准确的说法是转置卷积
- 也可以用反池化
- 评价：m-IOU
- 工具

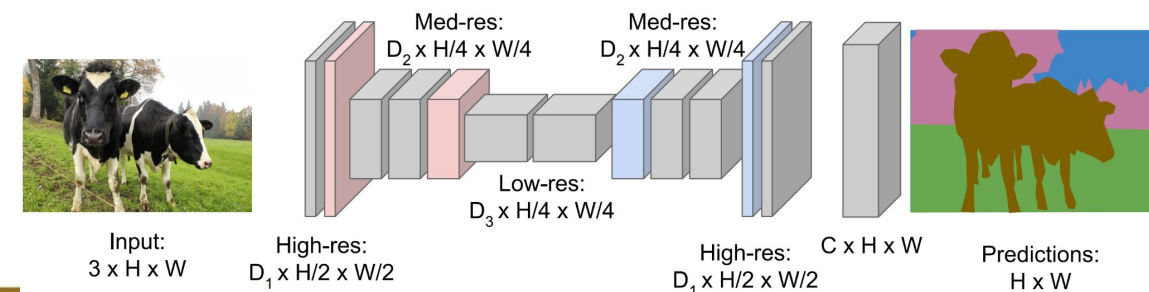
■ Deeplab



Dense Prediction - Semantic Segmentation

• Summary

- Task: Label each pixel in the image with a category label
- Design: convolution **encoder** and deconvolution **decoder**
 - Deconvolution: ML people don't like this name
 - More precisely - transposed convolution
 - And we could also use unpooling
- Evaluation: m-IOU
- Tools
 - Deeplab



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 关键模型
 - CNN
 - 稠密预测
 - 注意力模型
 - 大规模视觉-语言学习
 - 几何计算机视觉
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

65

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

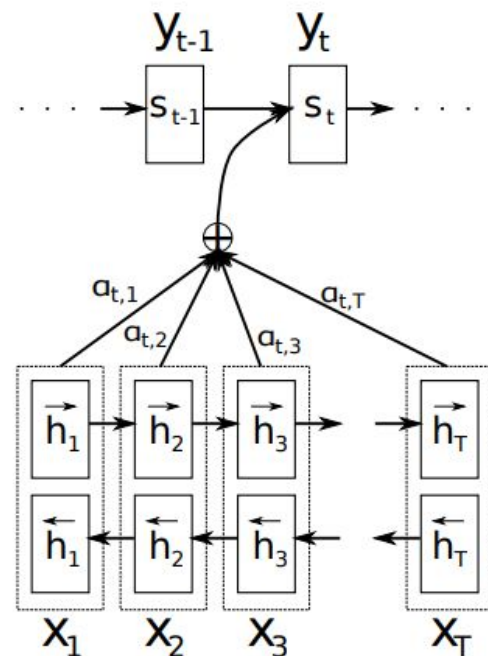
注意力模型 —— 经典方法

• 翻译 —— 学习词与词之间的对齐

- 给定源句子
- 以及已经生成的词
- 解码器：生成下一个词
- 生成第 t 个目标词时，对每个源词学习注意力（也叫对齐）

■ 直观上，解码器会学习该关注哪些源词
■ 因为生成对应输出时，并非所有输入都会被用到

输出词序列



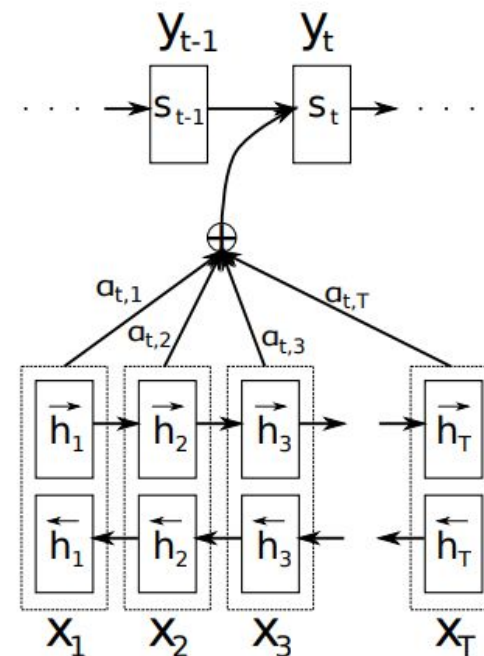
输入词序列



Attention Model - Classic

- Translation - learned alignment between words
 - Given source sentence $(x_1, x_2, \dots, x_T)$
 - and previous generated words
 - Decoder: Generate the next word y_t
 - Learned attention (a.k.a alignment) over each source word when generating the t -th target word
 - Intuitively, decoder learns **which source words to pay attention to**
 - because not all the inputs would be used in generating the corresponding output

Output word sequence



Input word sequence

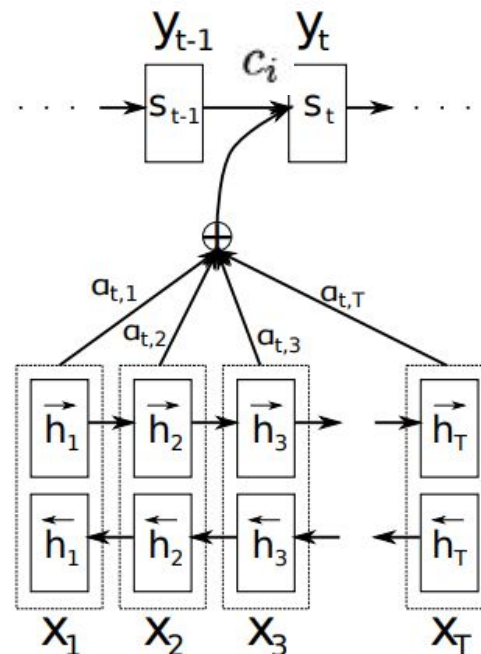


注意力模型 —— 经典方法

• 翻译 —— 学习词与词之间的对齐

- 注意力权重由前馈神经网络输出分数归一化得到；该网络由函数 a 描述，用来刻画输入位置 j 与输出位置 i 之间的对齐。
- 输出词 y_i 的上下文向量 c_i 由注意力与隐状态的加权和得到

$$e_{ij} = a(s_{i-1}, h_j) \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} \quad c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$



Attention Model - Classic

• Translation - learned alignment between words

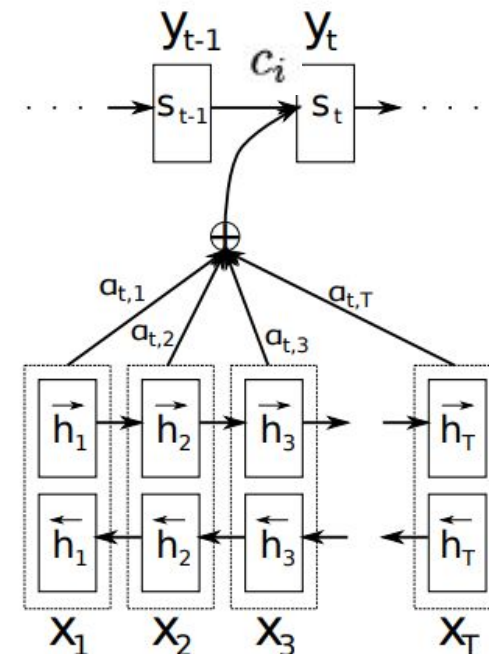
- The attention weights are calculated by normalizing the output score of a feed-forward neural network described by the function a that captures the alignment between input at j and output at i .
- The context vector c_i for the output word y_i is generated using the weighted sum of the attention and hidden states

$$e_{ij} = a(s_{i-1}, h_j) \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} \quad c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

Last output state

Hidden state from
LSTM/RNN at position j

*Neural Machine Translation by Jointly Learning to Align and Translate



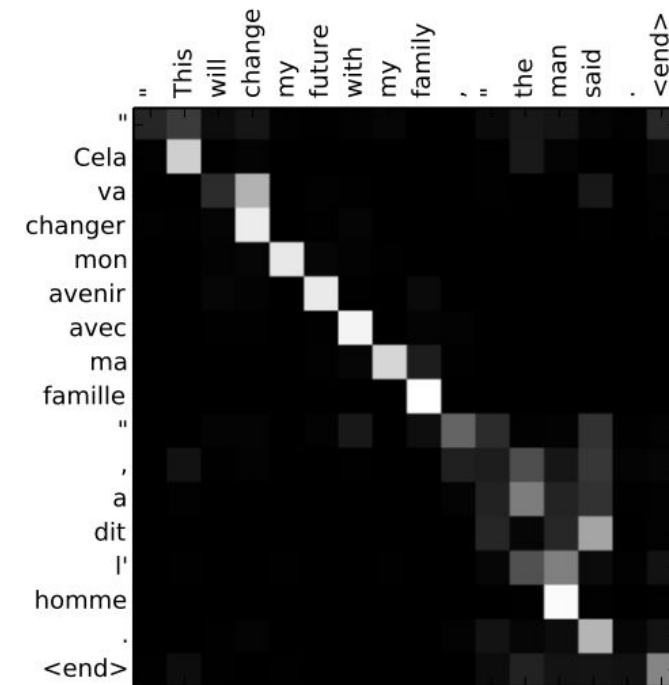
注意力模型 —— 经典方法

- 翻译 —— 学习词与词之间的对齐

○ 可视化

○ 把每个源词对齐到目标词

■ 几乎是一一对齐，对于



(d)

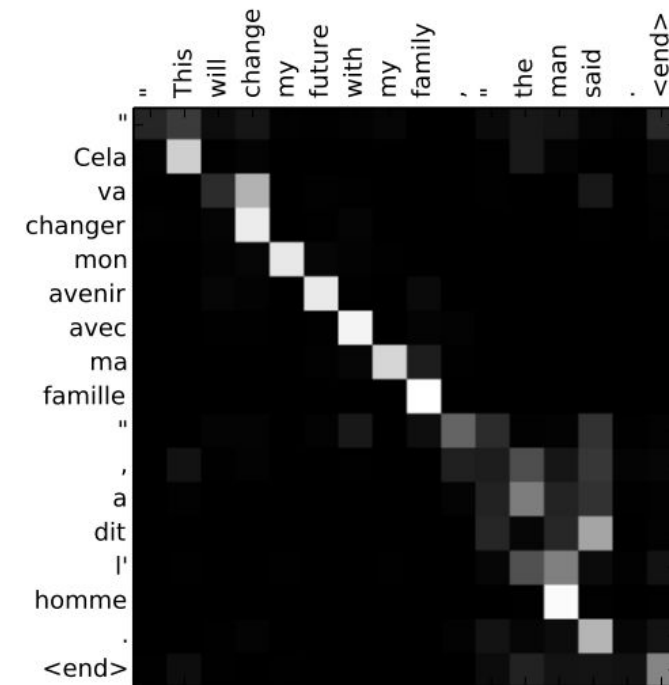
Attention Model - Classic

- Translation - learned alignment between words

○ Visualization

○ Aligning each source word to target word

■ Almost a 1-to-1 alignment for



(d)

注意力模型 —— 经典方法

- 视觉问答 —— 对齐图像局部与问题

$$\tilde{v}_I = \sum_i p_i v_i$$

图像局部

注意力：图像局部的权重，根据问题和图像各部分计算

$$v_i \in \mathbf{R}^d$$

$$v_I \in \mathbf{R}^{d \times m}$$

$$u = \tilde{v}_I + v_Q$$

编码器向量：问题
+ 被关注的图像信息

$$p_{ans} = \text{softmax}(W_a u)$$

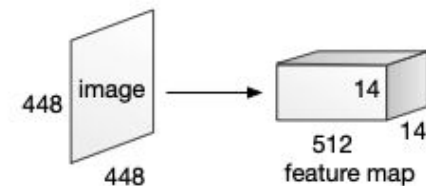


Figure 2: CNN based image model

m=14 x 14



What is the mustache made of?

Attention Model - Classic

- Visual Question Answering - aligning image part and questions

$$\tilde{v}_I = \sum_i p_i v_i$$

Part-of-image

Attention: weights of Part-of-image, computed based on question and image parts

$$v_i \in \mathbf{R}^d$$

$$v_I \in \mathbf{R}^{d \times m}$$

$$u = \tilde{v}_I + v_Q$$

The encoder vector: question
+ attended image info

$$p_{ans} = \text{softmax}(W_a u)$$

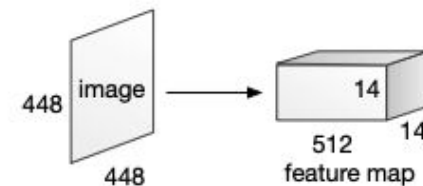


Figure 2: CNN based image model

m=14 x 14



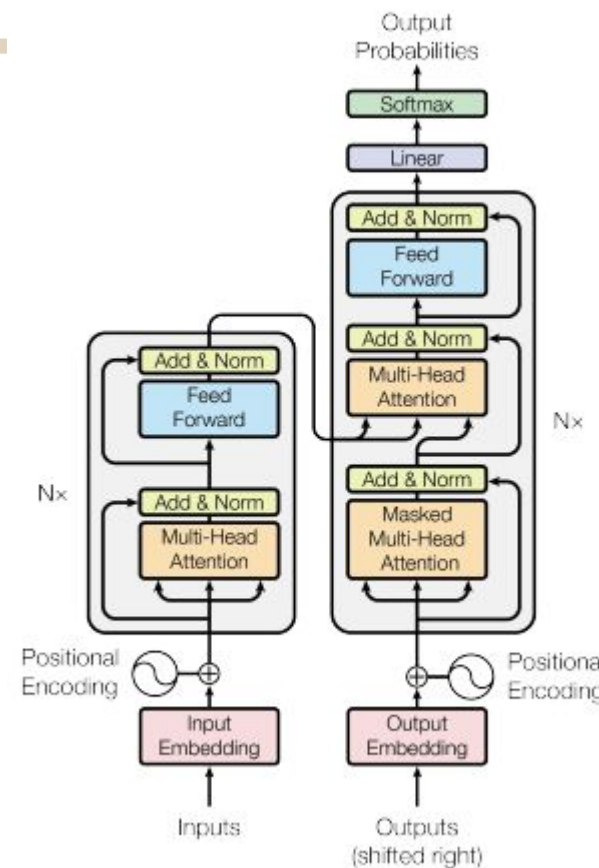
What is the mustache made of?

*Bias vector is not shown for simplicity

[1] Stacked Attention Networks for Image Question Answering. CVPR 2016

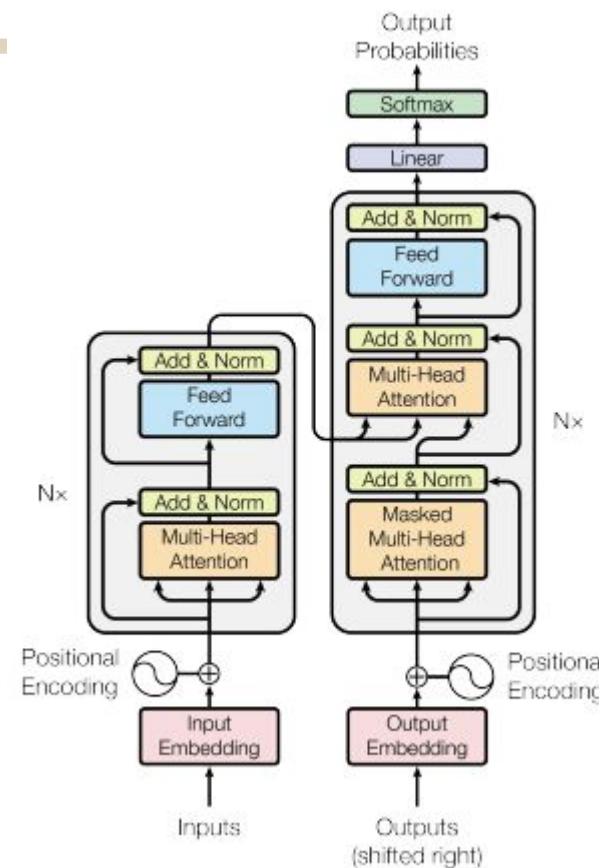
注意力模型 —— 自注意力

- 翻译 —— 学习词与词之间的对齐
 - 用注意力层替换 LSTM / RNN
 - 普通 RNN 很慢，长期记忆也很差
 - LSTM 和 GRU 解决了记忆问题，但仍然慢，而且是串行的



Attention Model - Self-Attention

- Translation - learned alignment between words
 - **Replacing the LSTMs / RNNs with attention layers**
 - Vanilla RNNs are Slow and have terrible long-term memory
 - LSTMs and GRUs fix the memory problem, but are still slow and sequential



注意力模型 —— 自注意力

- 翻译 —— 学习词与词之间的对齐

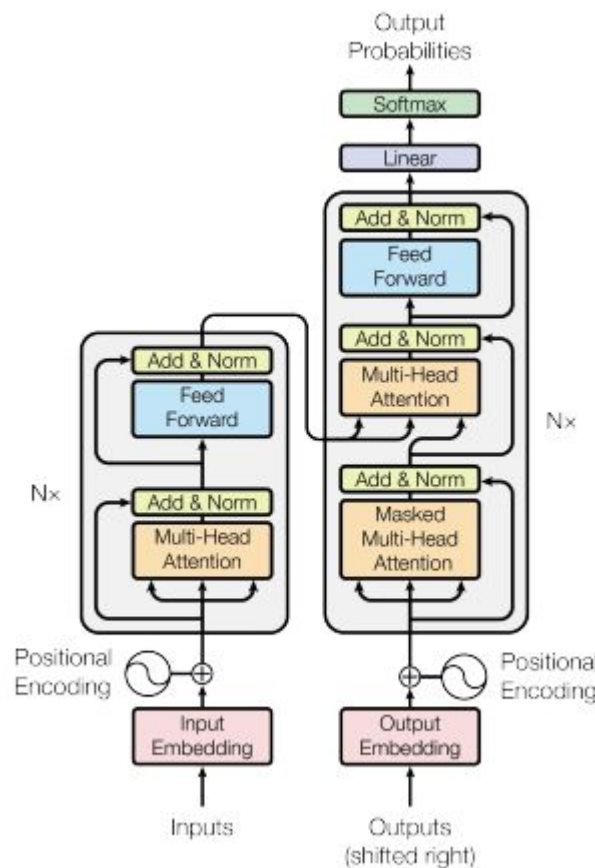
- 用注意力层替换 LSTM / RNN
- 自注意力与交叉注意力

- 注意力中的 Query、Key、Value

- Query: 发出注意力的一方
- Keys: 帮助 Query 判断应对每个 Value 分配多少注意力

- 注意力权重是 query-key 相似度
 - Values: 被关注的内容

- 因此是 $\text{computed_attention} \times \text{values}$



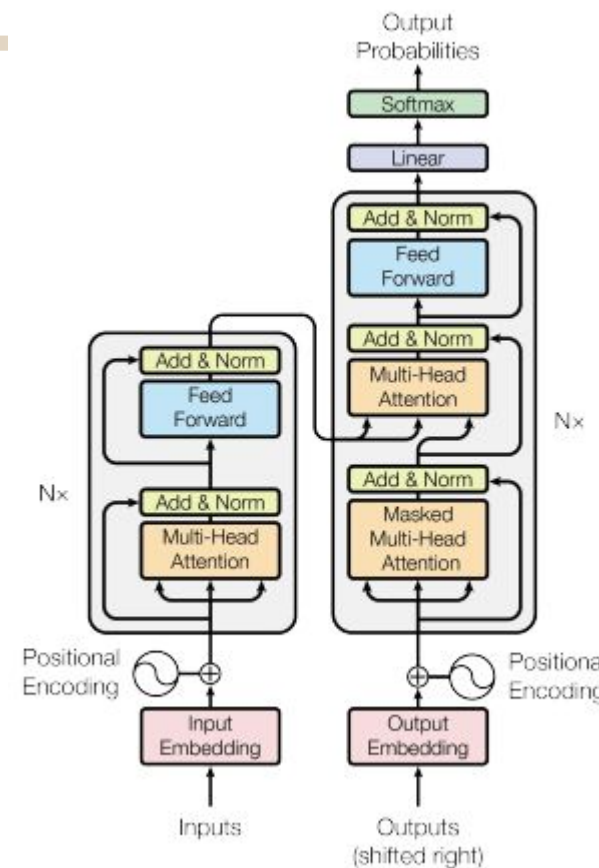
Attention Model - Self-Attention

- Translation - learned alignment between words

- Replacing the LSTMs / RNNs with attention layers
- Self-attention and cross-attention

- **Query, Key, Value in attention**

- Query: This is what pays the attention
- Keys: These help queries figure out how much attention to pay to each of the values
 - Attention weights are query-key similarities
- Values: These are paid attention to
 - So $\text{computed_attention} \times \text{values}$



注意力模型 —— 自注意力

假设有 N 个 token 输入，就要计算 N 个注意力权重

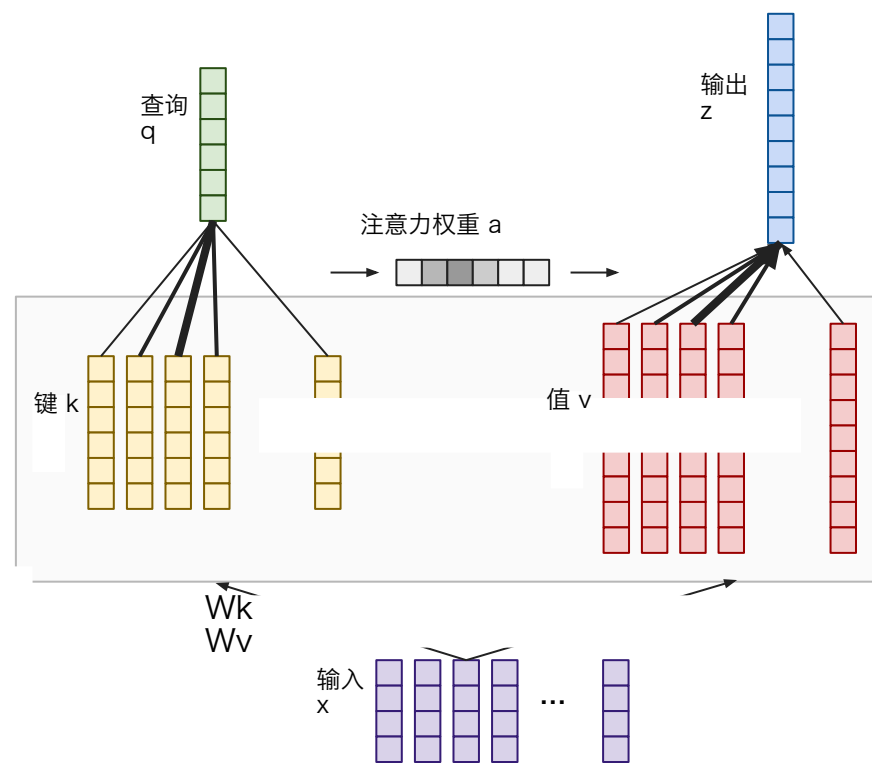
$$\hat{a}_i = q \cdot k_i$$

经 softmax 归一化

因此给定一个 query，对每个 key 得到注意力。

$$z = \sum_i \hat{a}_i v_i = \hat{a} \cdot v$$

$q = W_q \cdot y$ “交叉注意力”
或者来自同一输入 x ！这时称为“自注意力”：



Attention Model - Self-Attention

Suppose you have N token inputs, computing N attention weights

1. Attention weights $a_{1:N}$ are **query-key** similarities:

$$\hat{a}_i = q \cdot k_i$$

Normalized via softmax

So given one query, get attention for each key.

2. Output z is attention-weighted average of values $v_{1:N}$:

$$z = \sum_i \hat{a}_i v_i = \hat{a} \cdot v$$

3. Usually, k and v are derived from the same input x :

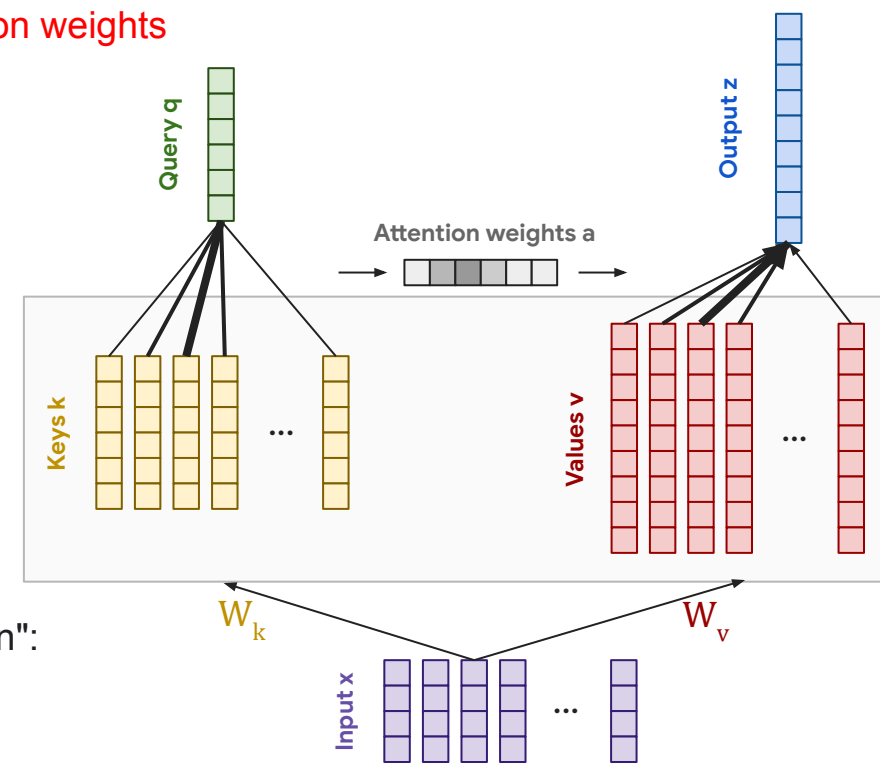
$$k = W_k \cdot x \quad v = W_v \cdot x$$

The query q can come from a separate input y :

$$q = W_q \cdot y \quad \text{“Cross attention”}$$

Or from the same input x ! Then we call it "self attention":

$$q = W_q \cdot x$$



注意力模型 —— 自注意力

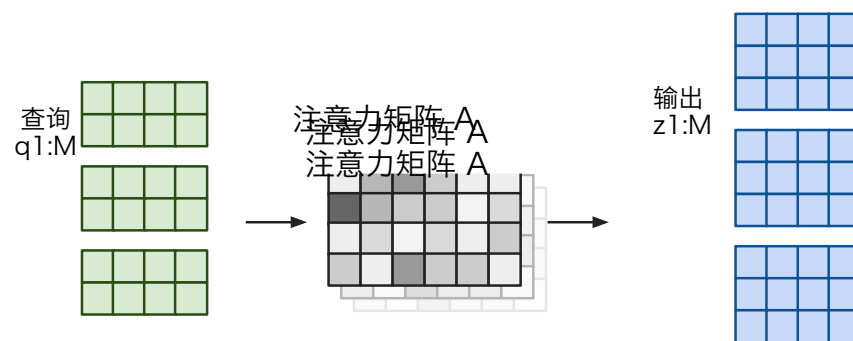
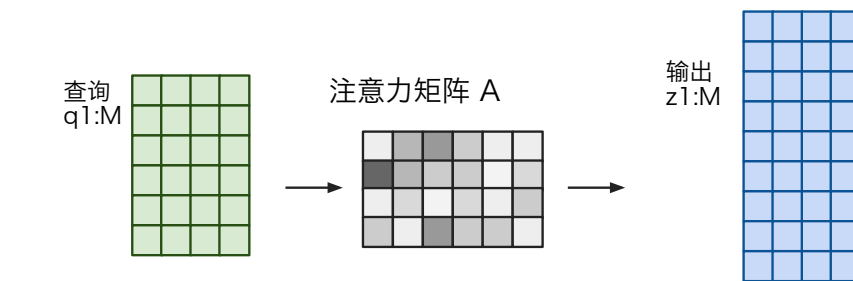
但还没完！还有几个细节：

1. 我们通常用很多查询 $q_{1:M}$ ，而不只一个。把它们堆叠起来就得到注意力矩阵 $A_{1:N,1:M}$ 以及多个输出：

2. 我们通常使用“多头”注意力。也就是把该运算重复 K 次，再沿特征维拼接。各头的 W 不同。

$$Z = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

注意复杂度是 $O(N^2)$ ， N 是 token 数量



Attention Model - Self-Attention

But that's not actually it! There are a few more details:

1. We usually use **many queries** $q_{1:M}$, not just one. Stacking them leads to the Attention matrix $A_{1:N,1:M}$ and subsequently to many outputs:

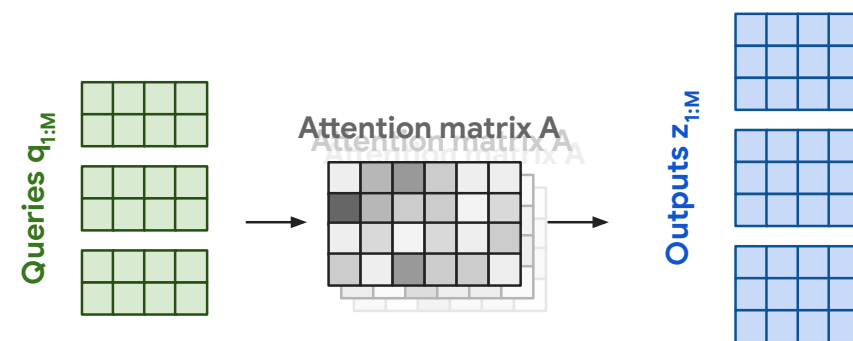
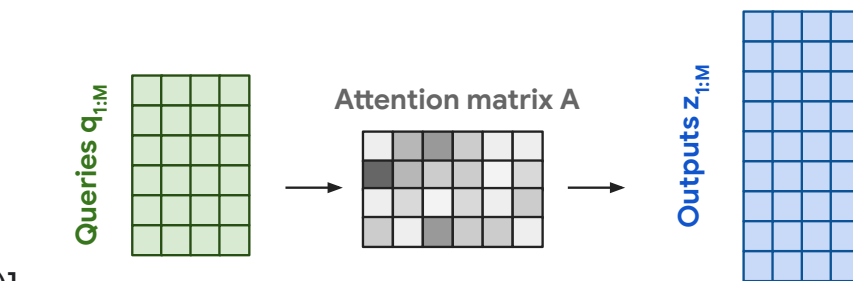
$$z_{1:M} = \text{Attn}(q_{1:M}, x) = [\text{Attn}(q_1, x) \mid \text{Attn}(q_2, x) \mid \dots \mid \text{Attn}(q_M, x)]$$

2. We usually use "**multi-head**" attention. This means the operation is repeated K times and the results are concatenated along the feature dimension. W s differ.

3. The most commonly seen formulation:

$$Z = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

Note that the complexity is $O(N^2)$, N is number of tokens



注意力模型 —— Vision Transformer

- 把 Transformer 模型用到视觉上
 - 把图像像素表示为序列
 - 但记住复杂度是 $O(N^2)$

Attention Model - Vision Transformer

- Applying the transformer model for vision
 - Representing image pixels as **sequences**
 - But remember the complexity is $O(N^2)$

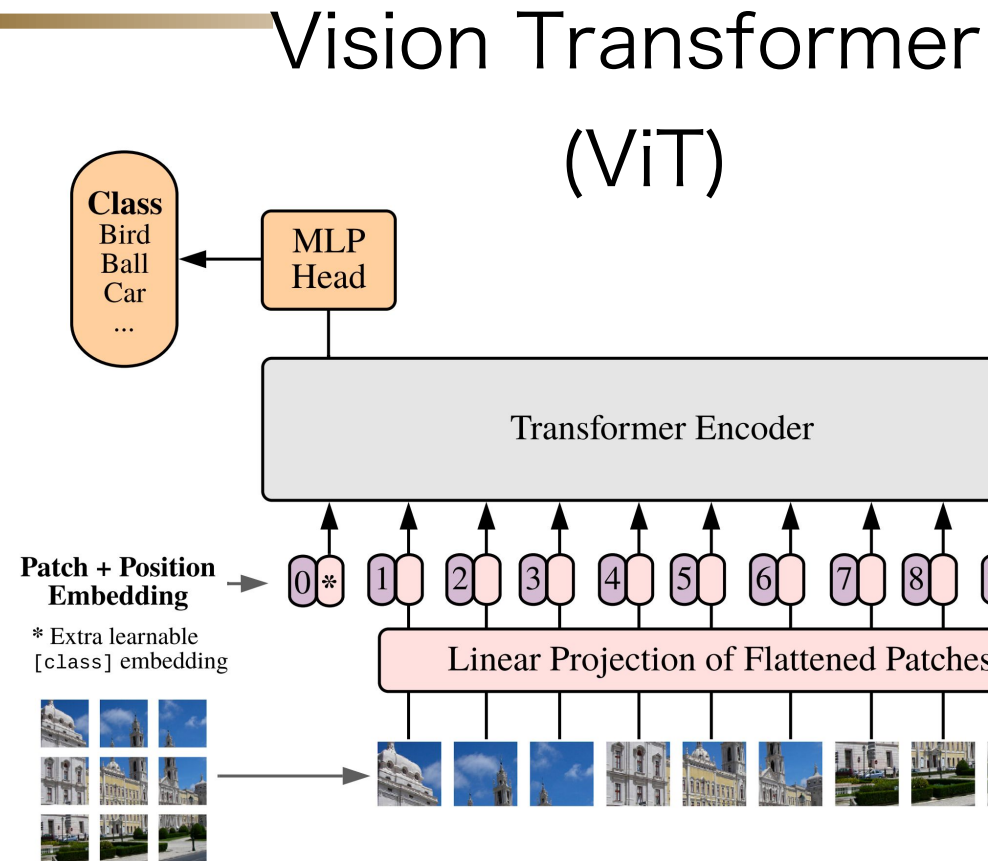
注意力模型 —— Vision Transformer

许多先前工作试图在像素级引入自注意力。

对 $224\text{px} \times 224\text{px}$ 来说，序列长度约 5 万，太长了！

因此多数工作把注意力限制在局部像素邻域，或作为检测结果之上的高层机制。

使用完整、独立 Transformer 架构的关键突破，是把图像切成 $16\text{px} \times 16\text{px}$ 的图像块来“分词”，把每个图像块当作一个 token，例如嵌入到输入空间。



Attention Model - Vision Transformer

Many prior works attempted to introduce self-attention at the pixel level.

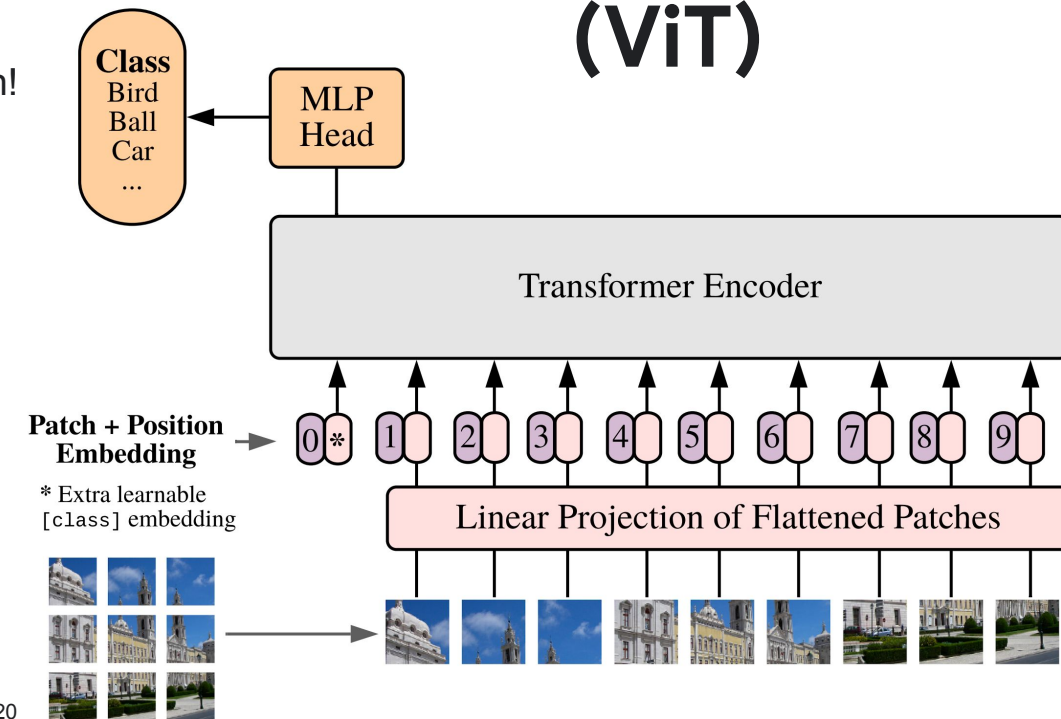
For 224px^2 , that's 50k sequence length, too much!

Thus, most works restrict attention to local pixel neighborhoods, or as high-level mechanism on top of detections.

The **key breakthrough** in using the full Transformer architecture, standalone, was to **"tokenize" the image by cutting it into patches** of 16px^2 , and treating each patch as a token, e.g. embedding it into input space.

*An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. 2020

Vision Transformer (ViT)



注意力模型 —— Vision Transformer

- 小结
可视化



Attention Model - Vision Transformer

- Summary
Visualization



小结

• CNN vs Vision Transformer

○ 关键设计差异

■ 卷积具有平移不变性* (这就是所谓“归纳偏置”，一种写进结构设计里的性质)

• 物体平移后，特征图上应有相同的物体响应

■ 而注意力具有排列不变性

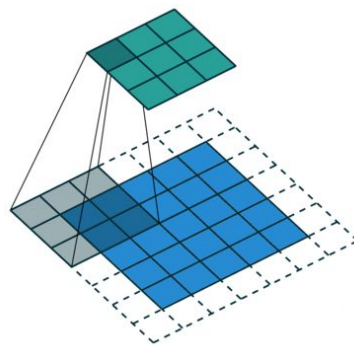
• 改变元素顺序，注意力结果不变

• 归纳偏置更宽松

○ 因此

■ 数据更多时，Transformer 往往优于 CNN

○ 另一个好处 —— Transformer 可用于多模态数据



Summary

• CNNs vs Vision Transformers

○ Key design different

■ Convolution is translation invariant* (this is what we call “inductive bias”, a property that is built into the design)

• An object translated should has the same object response on the feature maps

■ While attention is permutation invariant

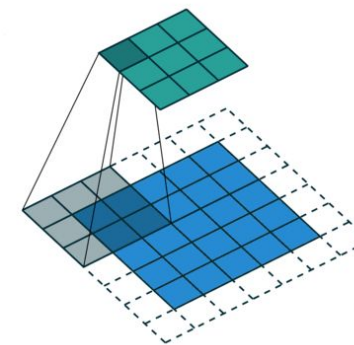
• Change the order of the element the attention stays the same

• The inductive bias is more relaxed

○ Therefore

■ Given more data, transformers are better than CNNs

○ Another perk - transformers can be used for multimodal data



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 关键模型
 - CNN
 - 稠密预测
 - 注意力模型
 - 大规模视觉-语言学习
 - 几何计算机视觉
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

78

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

78

大规模视觉-语言学习

- CLIP (Contrastive Language - Image Pre-training) , OpenAI, 2021

- 在从互联网抓取的 4 亿图文对上训练
- 证明大规模 (带噪声的) 视觉-语言预训练可以带来很强性能 (配合合适的提示工程)

- 任意零样本下游分类任务
- AI 生成内容 (文生图)

- DALL-E 等

Large-scale Vision-Language Learning

- CLIP (Contrastive Language–Image Pre-training) by OpenAI in 2021

- Train on 400 million image+text crawled from the internet
- Proved that large-scale (noisy) vision-language pre-training can lead to great performance (with proper prompt engineering)
 - Any zero-shot downstream classification tasks
 - AI generated content (text-to-image generation)
 - DALL-E, etc.

大规模视觉-语言学习

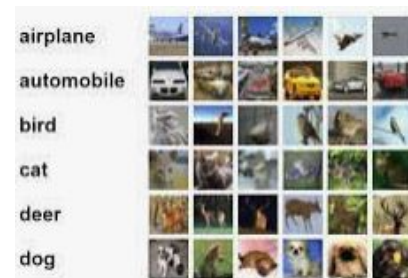
- CLIP (Contrastive Language - Image Pre-training) , OpenAI, 2021

- 在从互联网抓取的 4 亿图文对上训练
- 证明大规模 (带噪声的) 视觉-语言预训练可以带来很强性能 (配合合适的提示工程)

- 任意零样本下游分类任务
- AI 生成内容 (文生图)

- DALL-E 等
- 这对具身智能有什么用?

- 我们需要让感知模型泛化, 而不只是识别离散物体类别



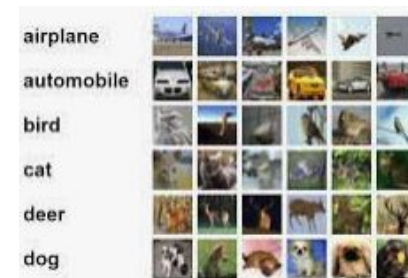
Large-scale Vision-Language Learning

- CLIP (Contrastive Language-Image Pre-training) by OpenAI in 2021

- Train on 400 million image+text crawled from the internet
- Proved that large-scale (noisy) vision-language pre-training can lead to great performance (with proper prompt engineering)
 - Any zero-shot downstream classification tasks
 - AI generated content (text-to-image generation)
 - DALL-E, etc.

- Why is this useful for Embodied AI?

- We need to generalize our perception model to just recognizing discrete object classes
 - CIFAR-10/100, ImageNet-1K, ImageNet-21K
 - Finite set of vocabularies



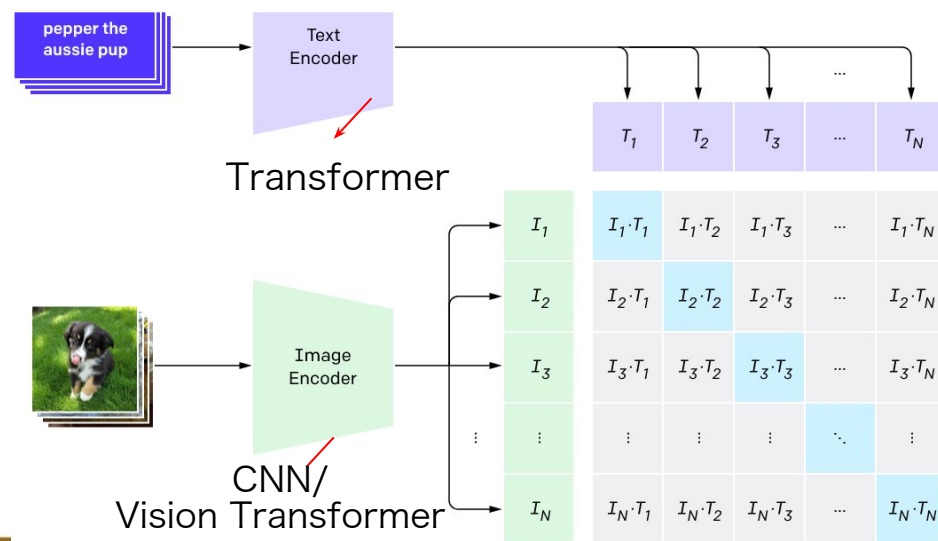
大规模视觉-语言学习

- CLIP (Contrastive Language - Image Pre-training) , OpenAI, 2021

- 两个关键设计

- 对比预训练

- 正例图文对 + 随机负例, 用 CE 损失训练 (需要很大 batch size 才能生效 —— 32k)



* <https://openai.com/blog/clip/>

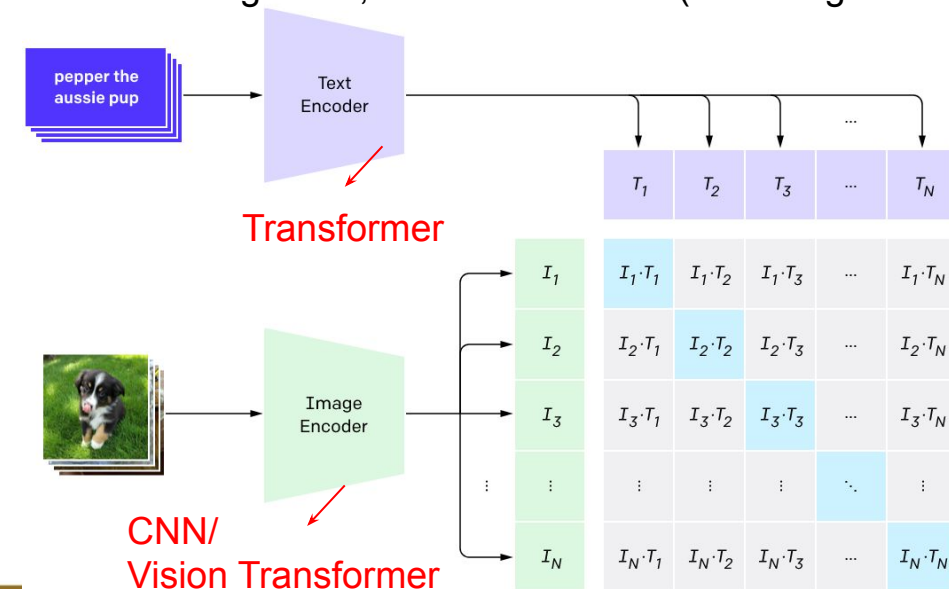
Large-scale Vision-Language Learning

- CLIP (Contrastive Language–Image Pre-training) by OpenAI in 2021

- Two Key Designs

- Contrastive pre-training

- Positive image-text + random negatives, train with CE loss (need large batch size to work - 32k)



* <https://openai.com/blog/clip/>

大规模视觉-语言学习

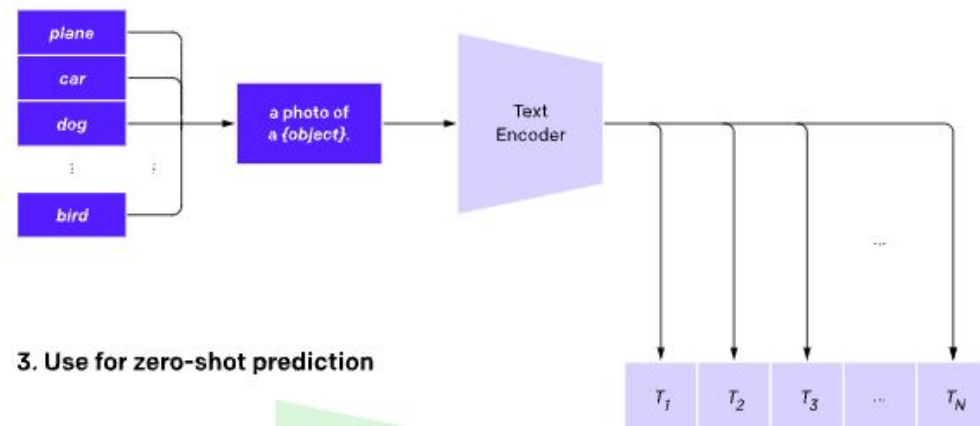
- CLIP (Contrastive Language - Image Pre-training) , OpenAI, 2021

- 两个关键设计

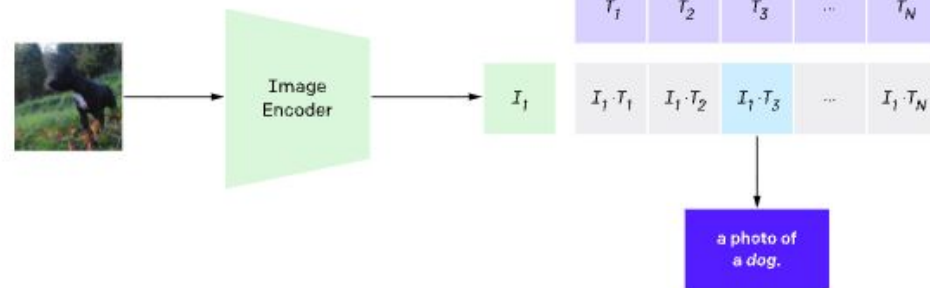
- 对比预训练
- 提示工程

- “A photo of [class]”
- 只需加上前缀文本
- 提升零样本效果

2. Create dataset classifier from label text



3. Use for zero-shot prediction



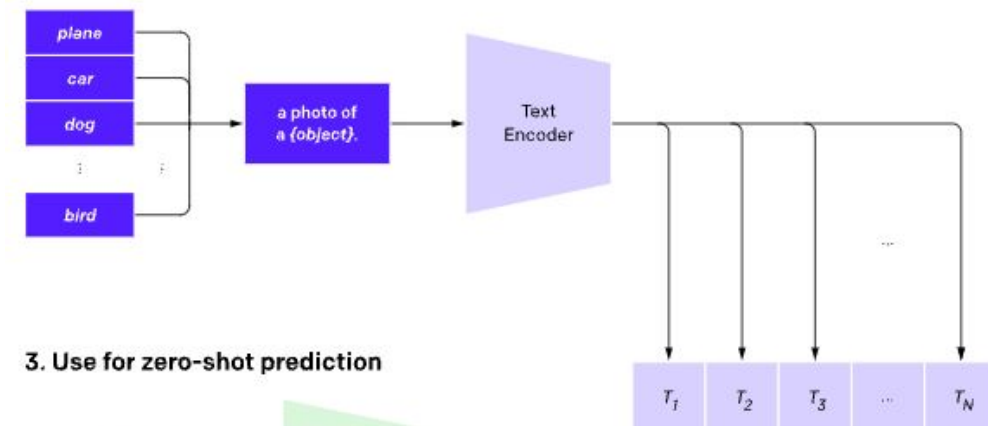
Large-scale Vision-Language Learning

- CLIP (Contrastive Language–Image Pre-training) by OpenAI in 2021

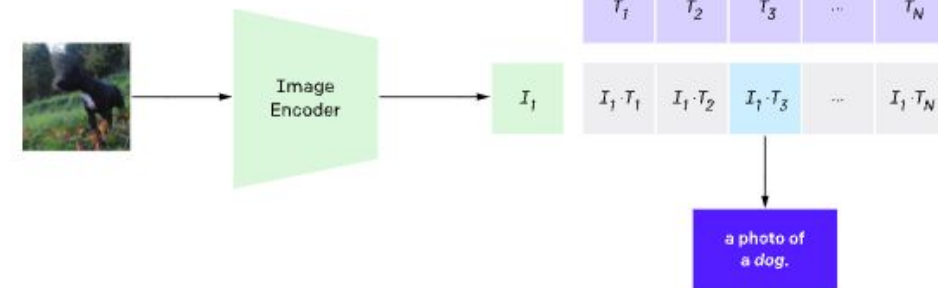
- Two Key Designs

- Contrastive pre-training
- Prompt Engineering
 - “A photo of [class]”
 - Simply add prefix text
 - Improves zero-shot

2. Create dataset classifier from label text



3. Use for zero-shot prediction



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 几何计算机视觉
 - 相机模型
 - 透视投影
 - 相机标定
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

Agenda

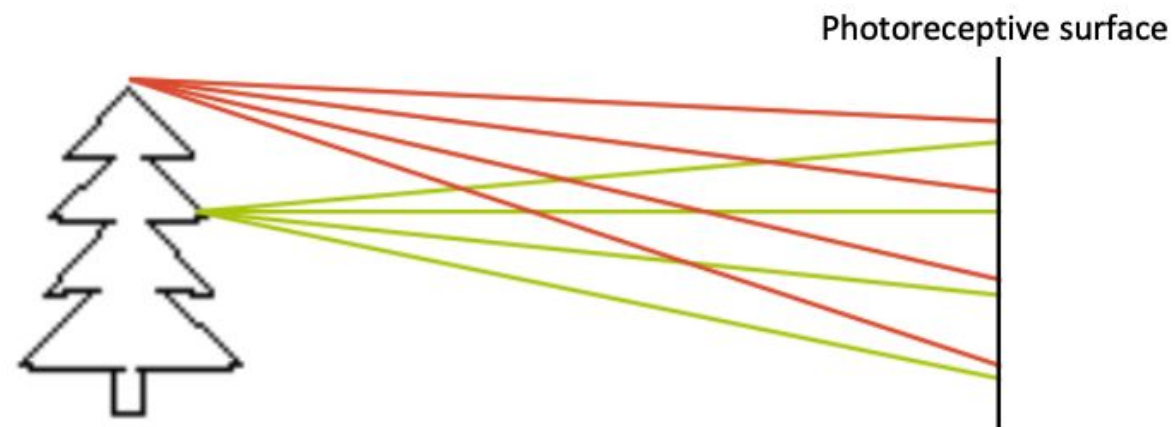
- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

相机

- 如何拍摄世界的一张图像

- (数字) 相机 -> 捕获光线 -> 转换成数字图像
- 光线被物体反射后向各方向散射
- 如果只放一个感光面，拍到的图像会极度模糊

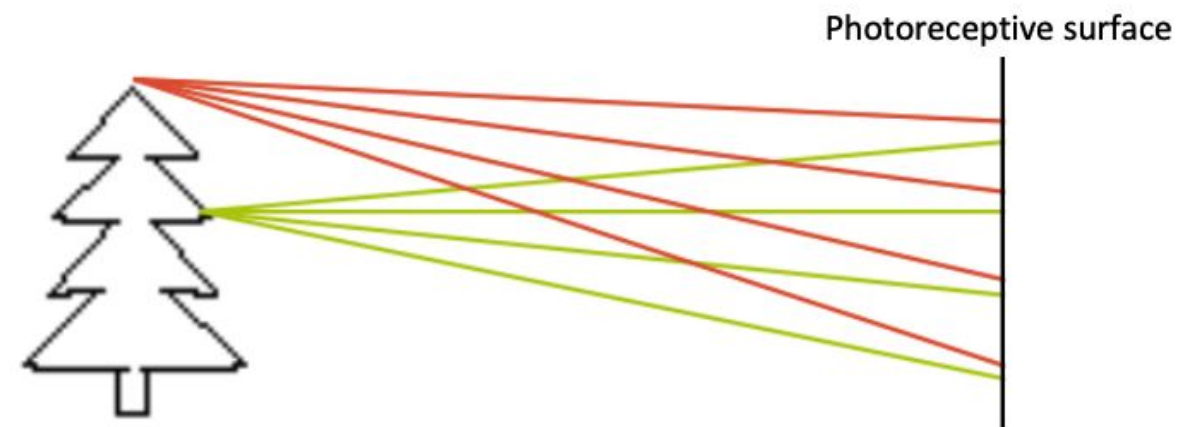
■ 为什么？



Cameras

- How to capture an image of the world

- (Digital) Cameras -> capture light -> converted to digital image
- Light is reflected by the object and scattered in all directions
- if we simply add a photoreceptive surface, the captured image will be extremely blurred
 - why?

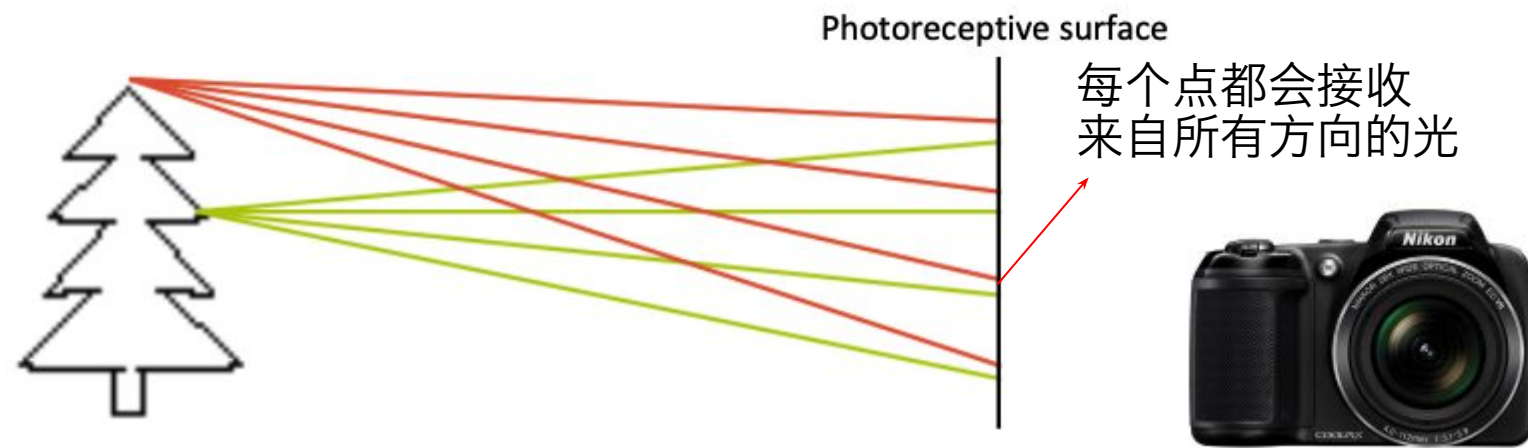


相机

- 如何拍摄世界的一张图像

- (数字) 相机 -> 捕获光线 -> 转换成数字图像
- 光线被物体反射后向各方向散射
- 如果只放一个感光面，拍到的图像会极度模糊

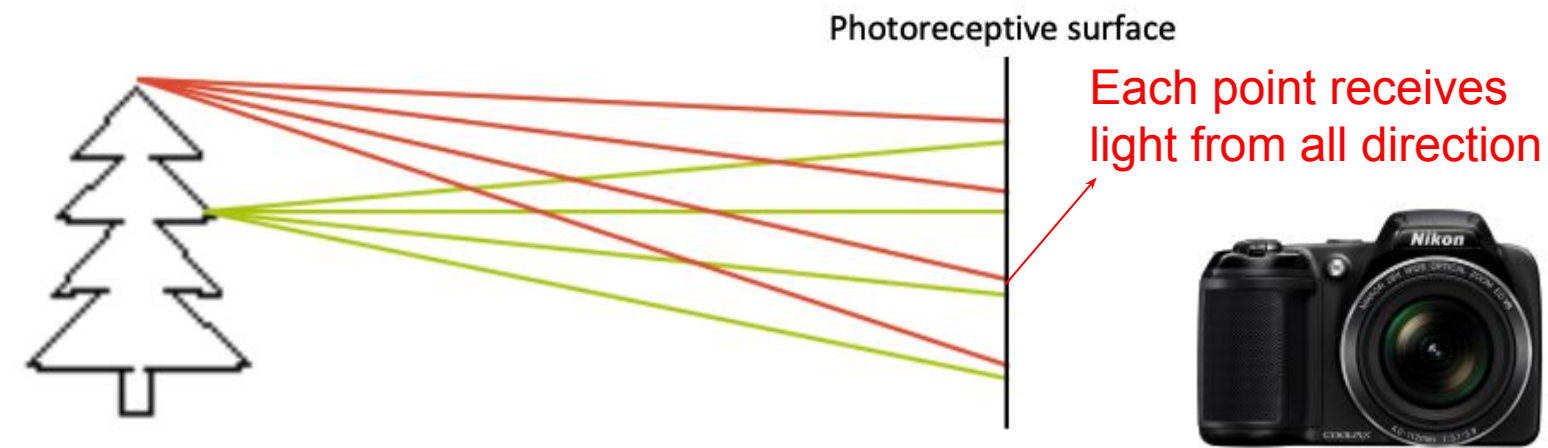
■ 为什么？



Cameras

- How to capture an image of the world

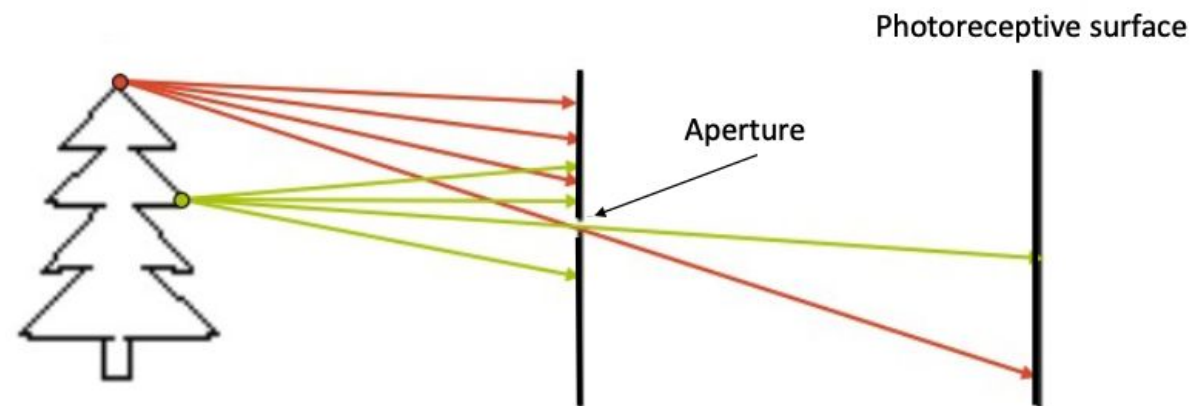
- (Digital) Cameras -> capture light -> converted to digital image
- Light is reflected by the object and scattered in all directions
- if we simply add a photoreceptive surface, the captured image will be extremely blurred
 - why?



针孔相机

- 如何拍摄世界的一张图像

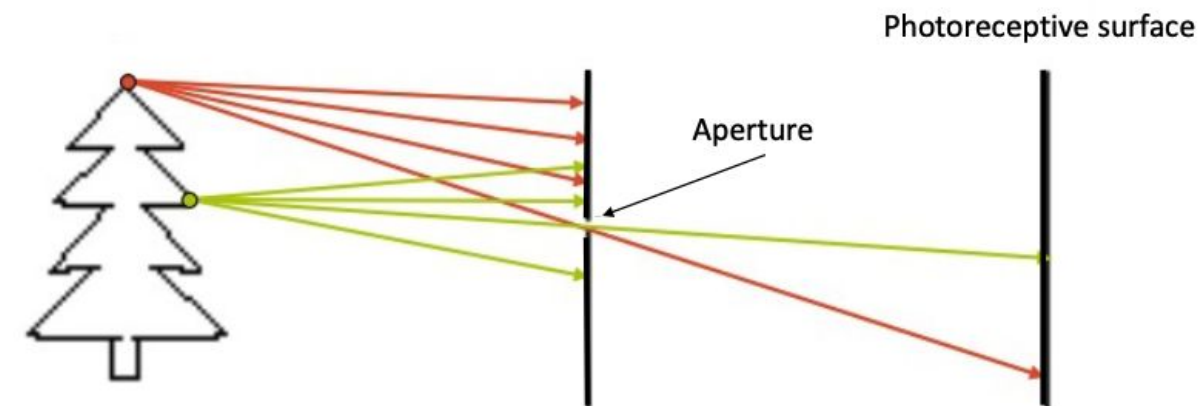
- 加一块挡板挡住大部分光线
- 光线只能从中心的小孔进入
- 针孔相机: 没有透镜、只有一个很小孔径（针孔）的相机



Pinhole Camera

- How to capture an image of the world

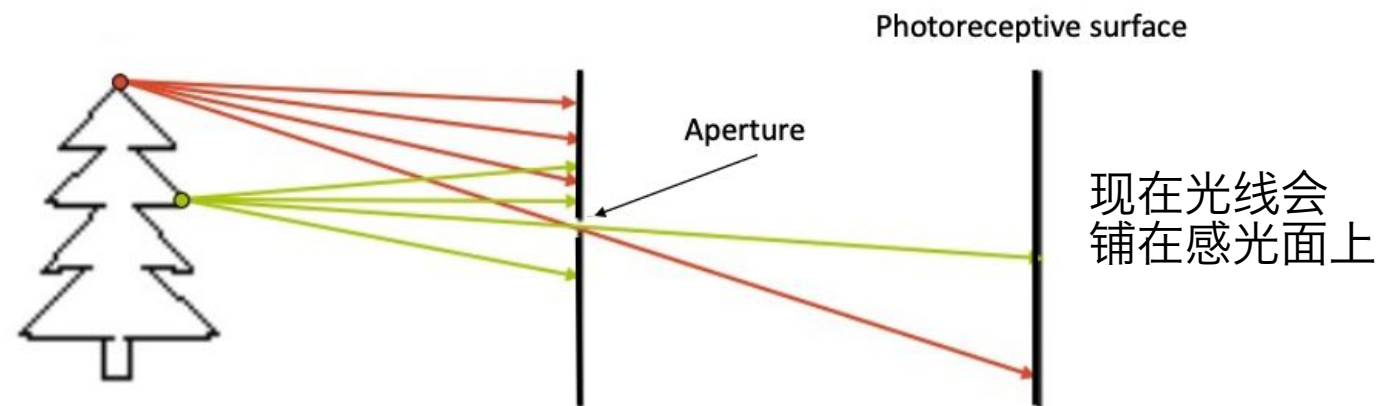
- Add a barrier to block off most of the light/rays
- The light can only come through the aperture in the center
- Pinhole camera: a camera without a lens but with a tiny aperture, a pinhole



针孔相机

- 如何拍摄世界的一张图像

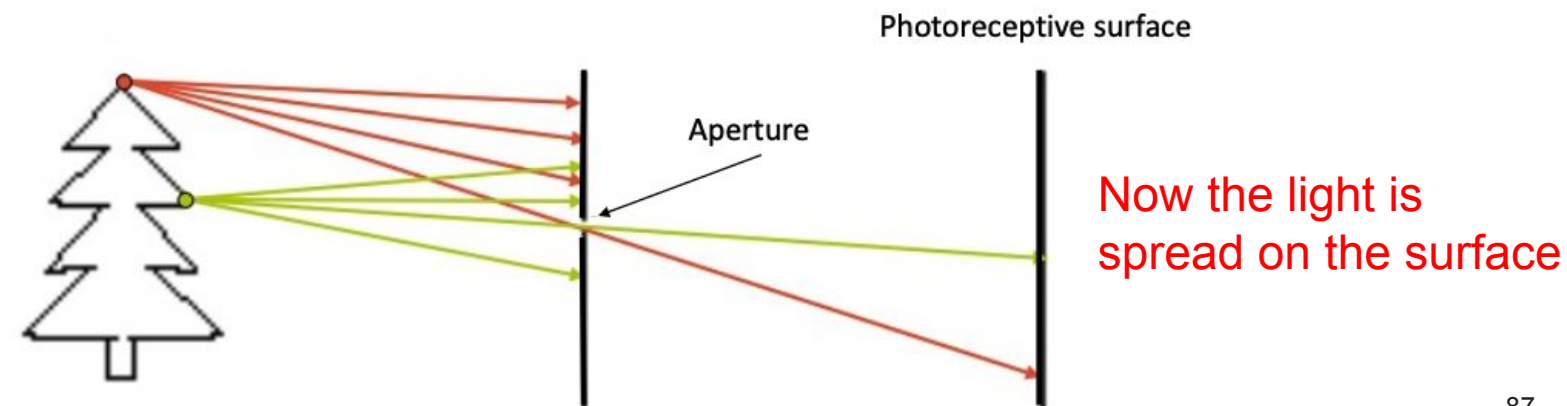
- 加一块挡板挡住大部分光线
- 光线只能从中心的小孔进入
- 针孔相机: 没有透镜、只有一个很小孔径 (针孔) 的相机



Pinhole Camera

- How to capture an image of the world

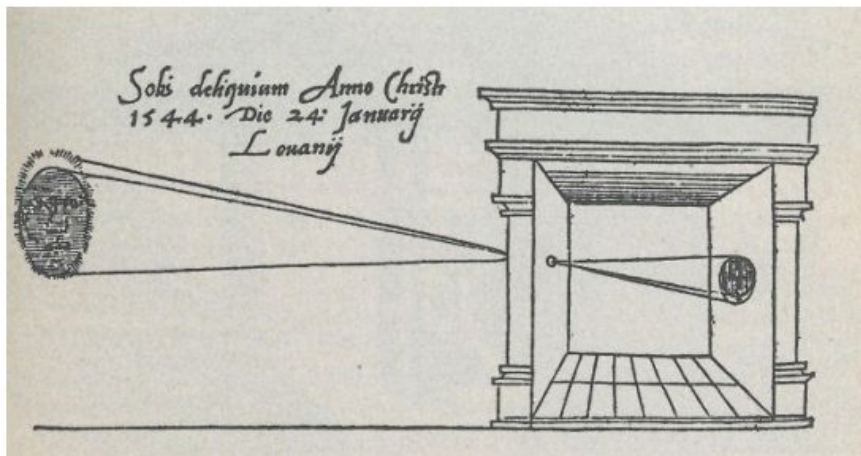
- Add a barrier to block off most of the light/rays
- The light can only come through the aperture in the center
- Pinhole camera: a camera without a lens but with a tiny aperture, a pinhole



针孔相机

- 历史悠久

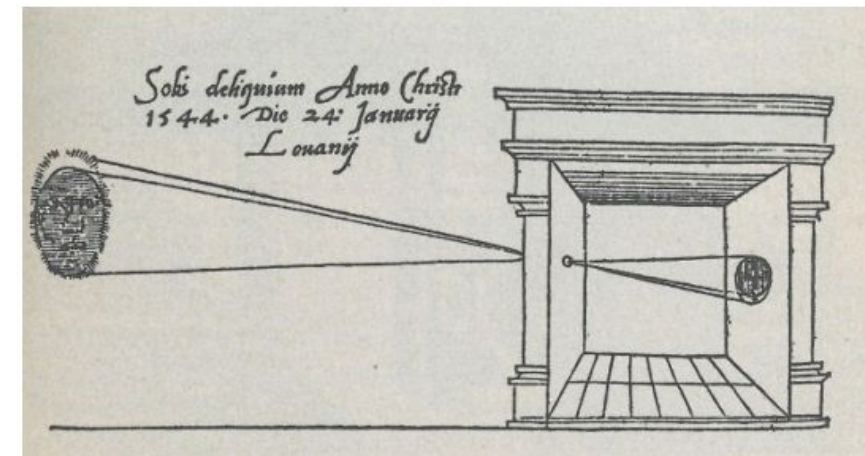
- Leonardo Da Vinci (1502) 给出了最早的清晰描述
- 已知最早发表的 camera obscura 图纸出自 Gemma Frisius (1544)



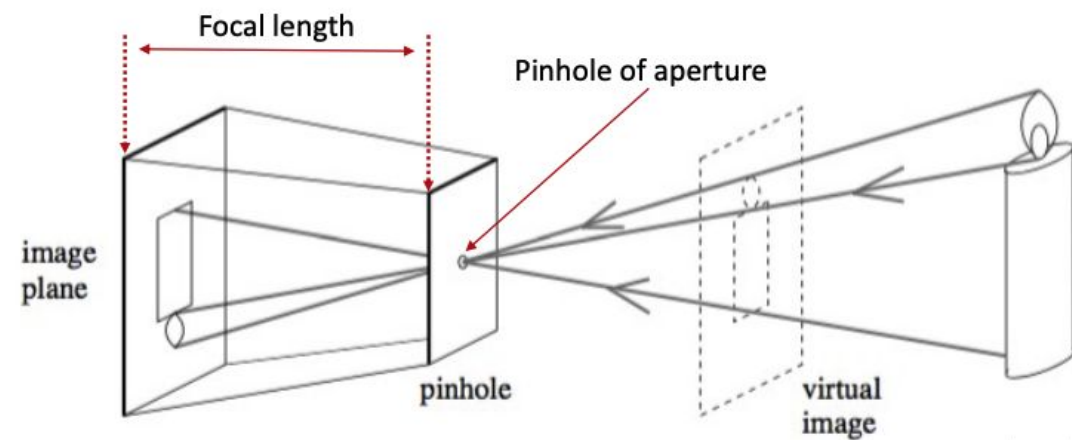
Pinhole Camera

- A long history

- First clear description from Leonardo Da Vinci (1502)
- Oldest known published drawing of a camera obscura by Gemma Frisius (1544)

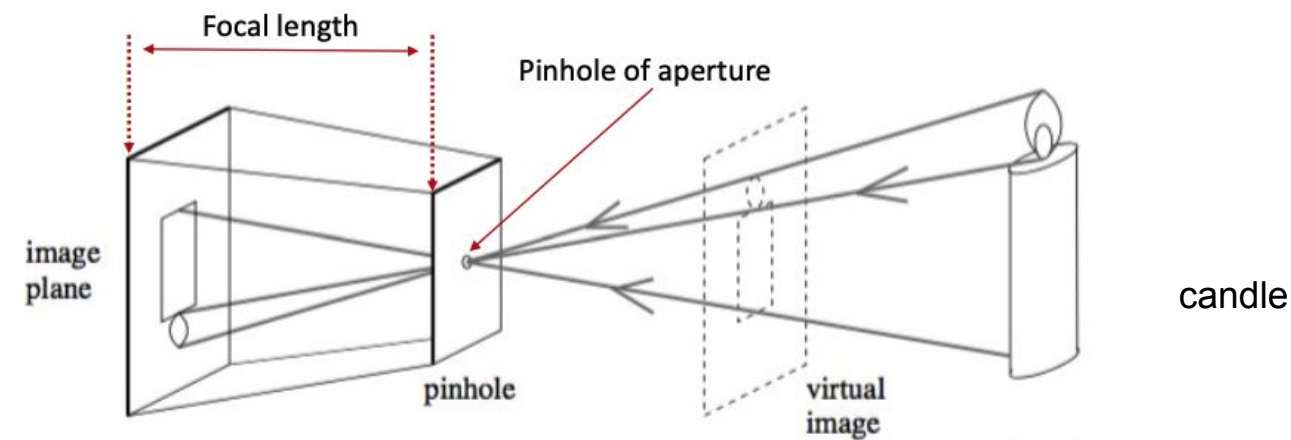


针孔相机



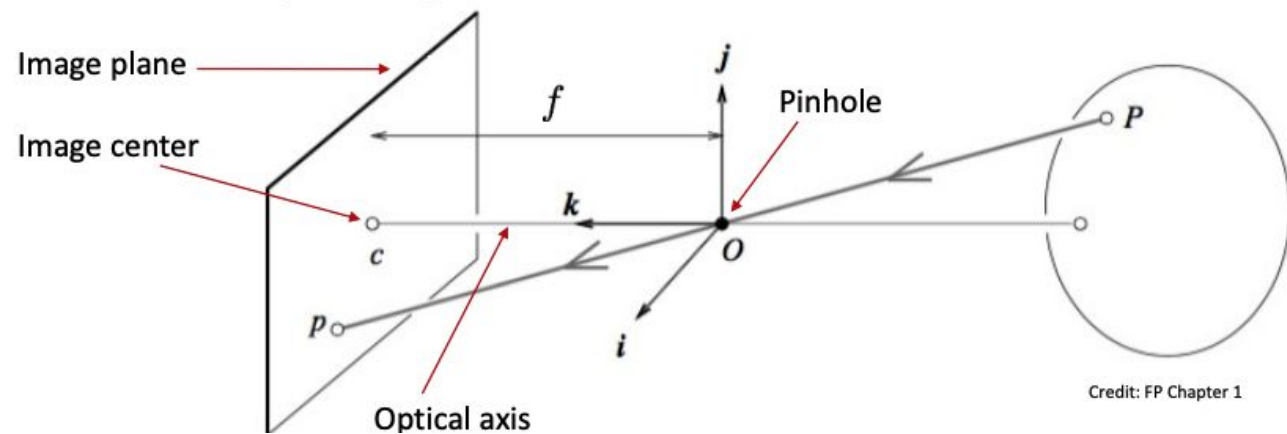
- 透视投影会产生倒立的像
- 有时为方便起见，可以考虑针孔前方平面上的虚像
- 虚像与实像等价，只是尺度不同

Pinhole Camera



- Perspective projection creates *inverted images*
- Sometimes it is convenient to consider a virtual image associated with a plane lying in front of the pinhole
- Virtual image is equivalent to the actual one except for on scale

针孔相机



- 如何把世界中的一点投影到像平面

假设我们
已知相机坐标系
中的 P

$$P = (X, Y, Z)$$

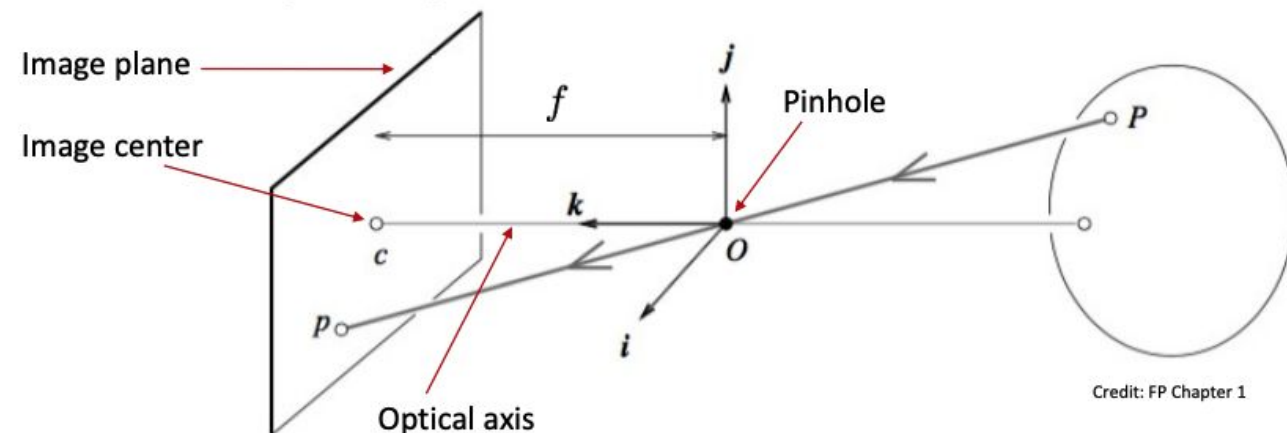
Perspective

$$p = (x, y, z)$$

如何得到
像平面上的 p
(三维, O 是
原点)



Pinhole Camera



- How to project a point in the world to the image plane

Suppose we
know P in the
camera frame

$$P_c = (X_c, Y_c, Z_c)$$

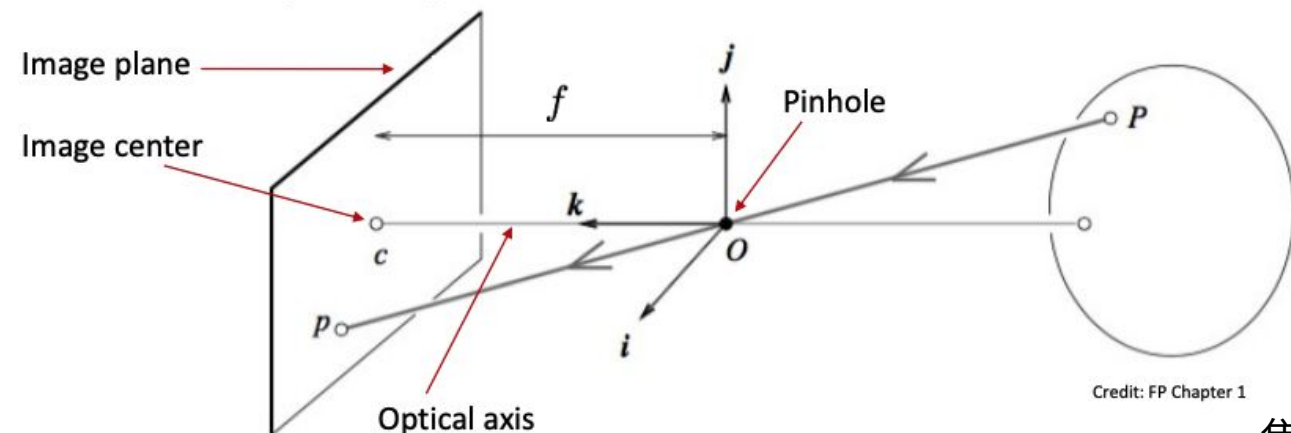
Perspective

$$p = (x, y, z)$$

How to get p on
the image plane
(3D, O is the
origin)



针孔相机



- 如何把世界中的一点投影到像平面

○ 因为 P、O 和 p 共线（在同一条直线上）：

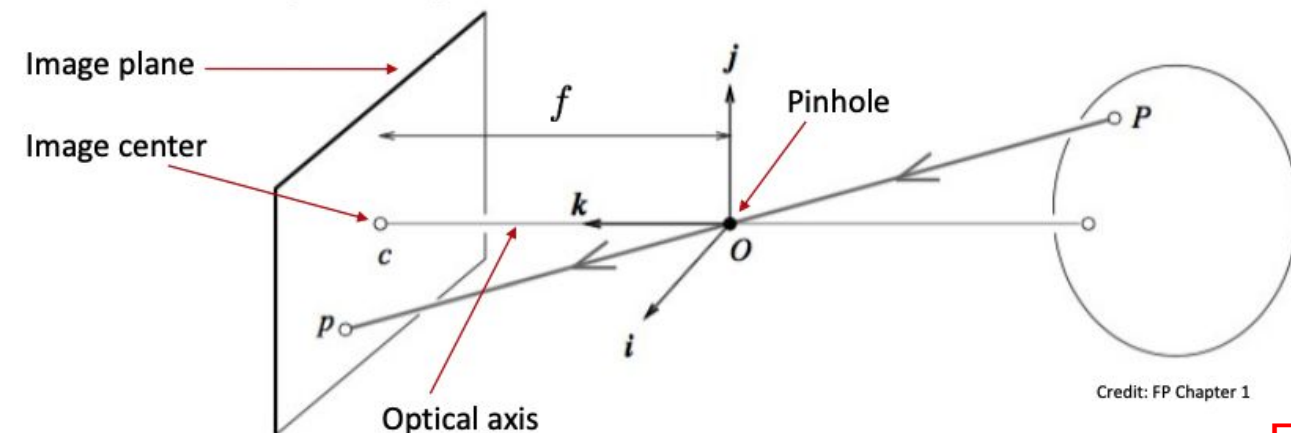
$$\begin{cases} x = \lambda X \\ y = \lambda Y \\ z = \lambda Z \end{cases} \Leftrightarrow \lambda = \frac{x}{X} = \frac{y}{Y} = \frac{z}{Z} \Rightarrow \begin{cases} x = f \frac{X}{Z} \\ y = f \frac{Y}{Z} \end{cases}$$

焦距

$$z = f$$

所以如果知道焦距，我们就知道 p 的三维坐标。后面会用到这个结论。

Pinhole Camera



- How to project a point in the world to the image plane

○ Since P, O, and p are collinear (on the same line): $\overline{Op} = \lambda \overline{OP}$

$$\begin{cases} x = \lambda X \\ y = \lambda Y \\ z = \lambda Z \end{cases} \Leftrightarrow \lambda = \frac{x}{X} = \frac{y}{Y} = \frac{z}{Z} \Rightarrow \begin{cases} x = f \frac{X}{Z} \\ y = f \frac{Y}{Z} \end{cases}$$

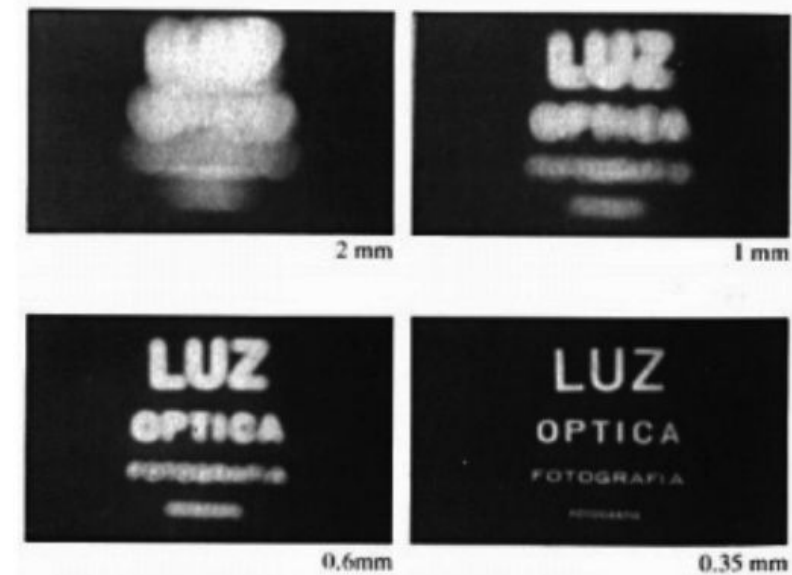
Focal Length

$$z = f$$

So if we know the focal length, we know the 3D coordinates of p. Will use this conclusion later.

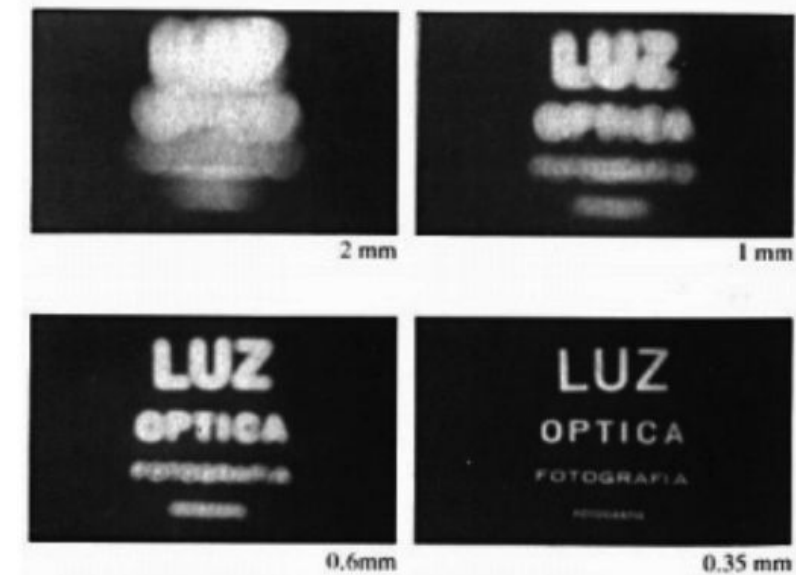
针孔相机的问题

- 孔更大: 穿过针孔的光更多
→ 更模糊
- 孔更小: 更暗
- 解决办法: 用透镜代替
针孔



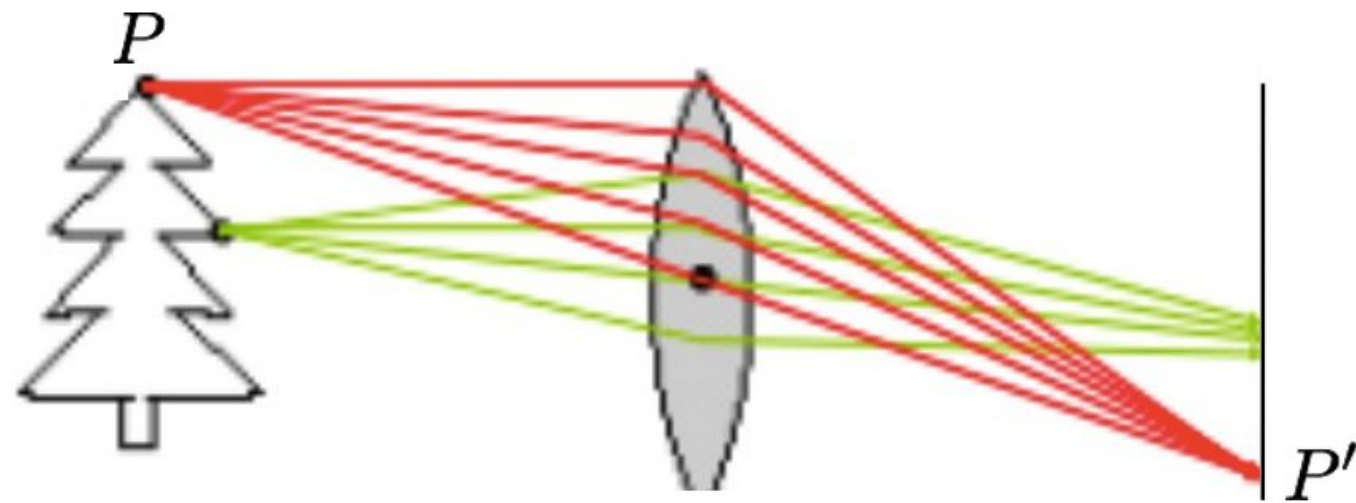
Issues with Pinhole Camera

- Larger hole: more lights passing through the pinhole → blurry
- Smaller: darker
- Solution: add a lens to replace the pinhole



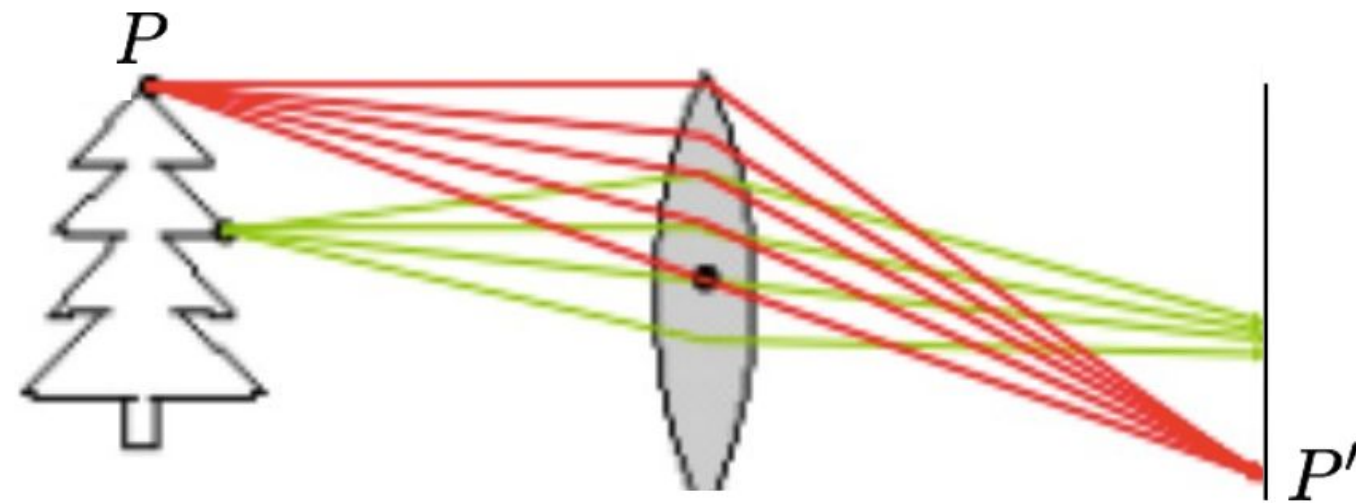
透镜

- 透镜：通过折射来会聚光线的光学元件（玻璃）

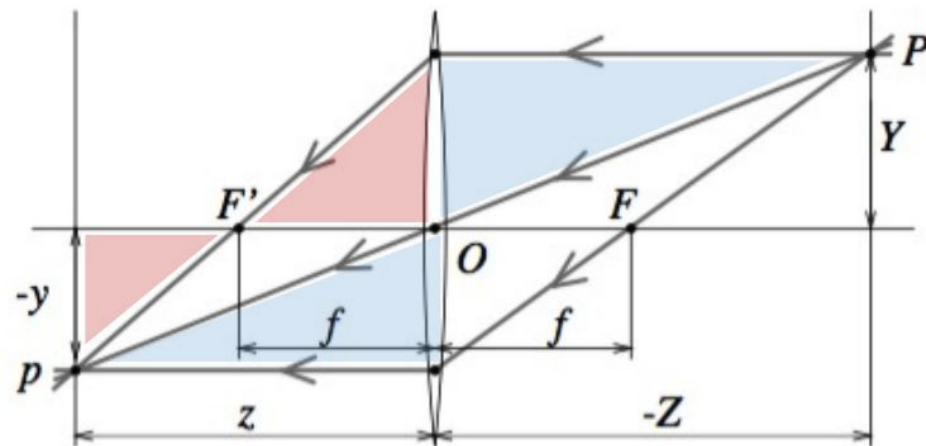


Lenses

- Lens: an optical element (glass) that focuses light by means of refraction



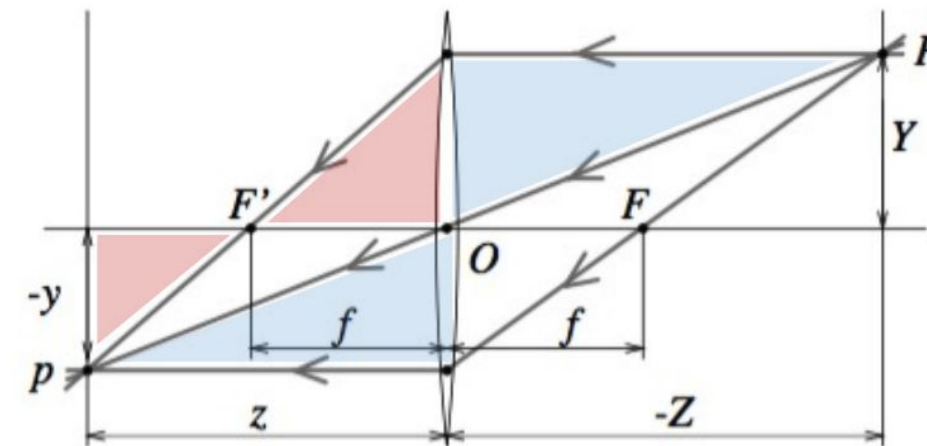
薄透镜模型



• 关键性质

- 穿过 O (透镜中心) 的光线不发生折射
- 平行于光轴的光线会聚到焦点 F'
- 所有穿过 P 的光线经透镜后会聚到点 p

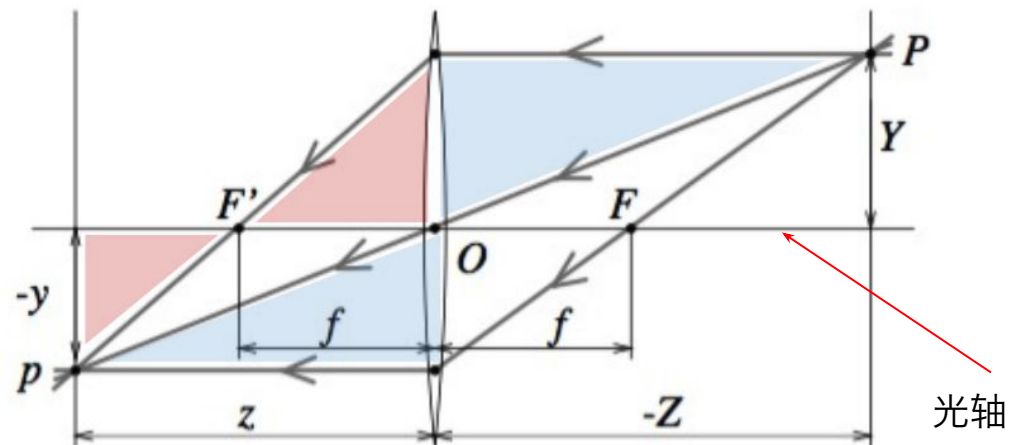
Thin Lens Model



• Key properties

- Rays passing through O (lens center) are not refracted
- Rays parallel to the optical axis are focused on the focal point F'
- All rays passing through P are focused by the lens on the point p

薄透镜模型

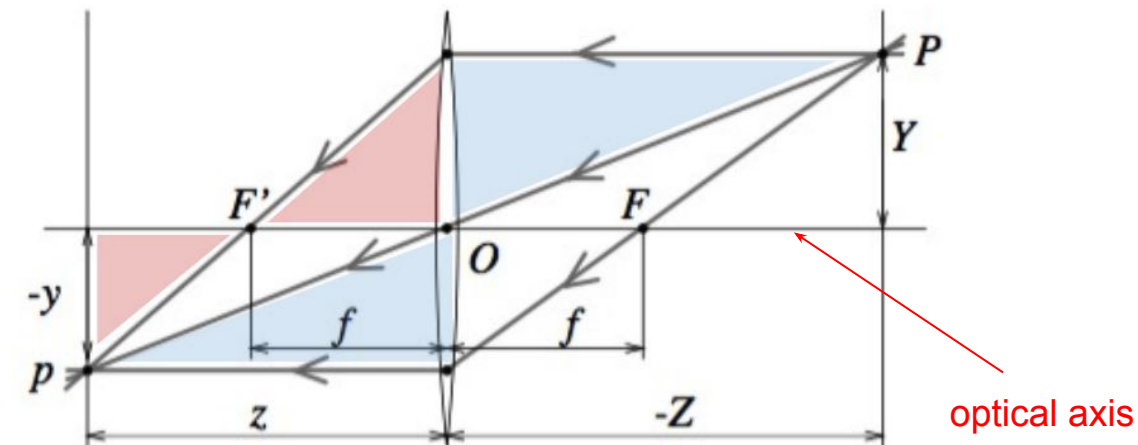


因此与针孔相比,
p 能接收到来自 P 的
更多光线

• 关键性质

- 穿过 O (透镜中心) 的光线不发生折射
- 平行于光轴的光线会聚到焦点 F'
- 所有穿过 P 的光线经透镜后会聚到点 p

Thin Lens Model



So p receives more
rays from P
compared to pinhole

• Key properties

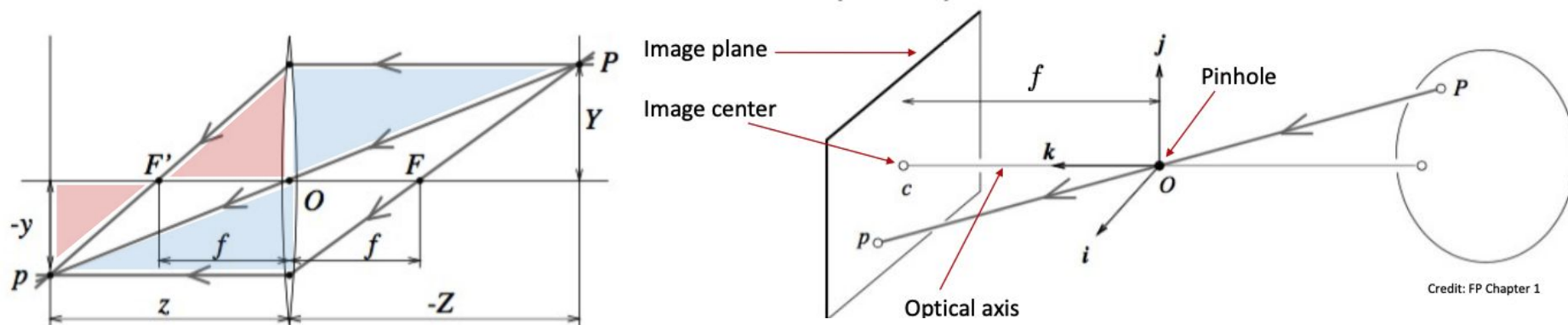
- Rays passing through O (lens center) are not refracted
- Rays parallel to the optical axis are focused on the focal point F'
- All rays passing through P are focused by the lens on the point p

薄透镜模型

• 说明

- 如果把 z 看作焦距，薄透镜模型与针孔模型的透视变换方程是相同的
- 为简化起见，薄透镜模型忽略景深效应

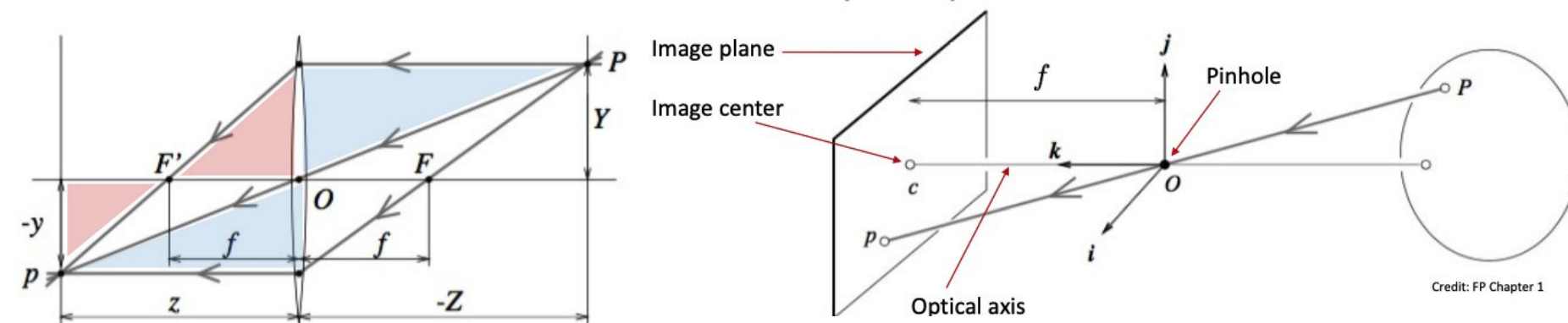
■ 假设相机对焦在无穷远



Thin Lens Model

• Notes

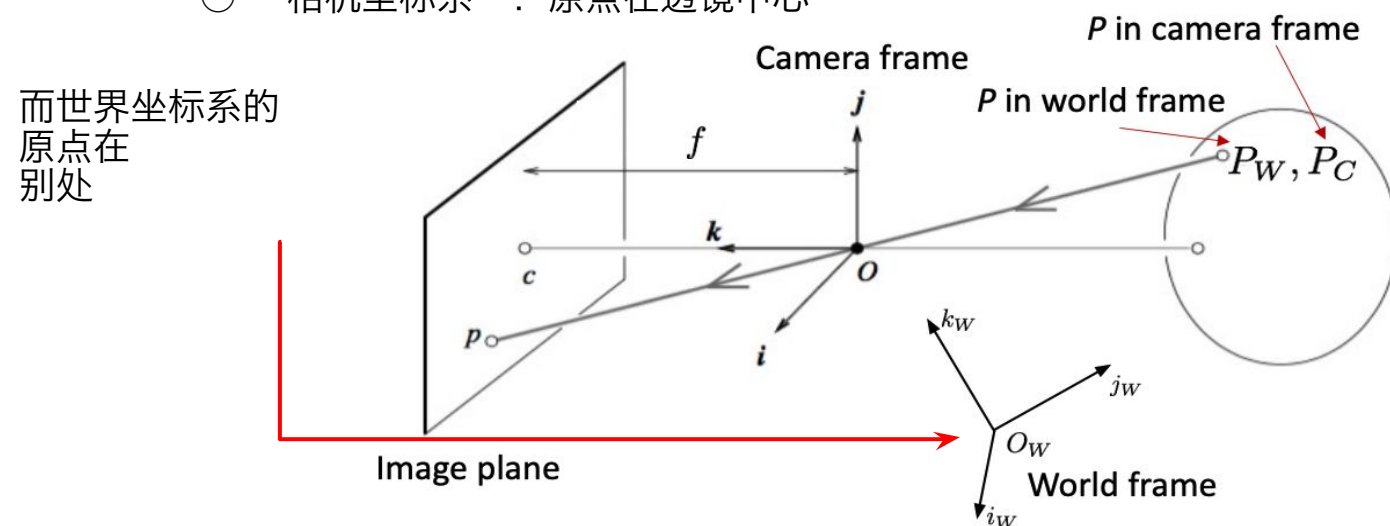
- If we consider z as the focal length, the perspective transformation equation is the same between thin lens model and pinhole model
- We ignore the depth of field effect for the thin lens model for simplicity
 - Assuming camera is focused at infinite far



透视投影

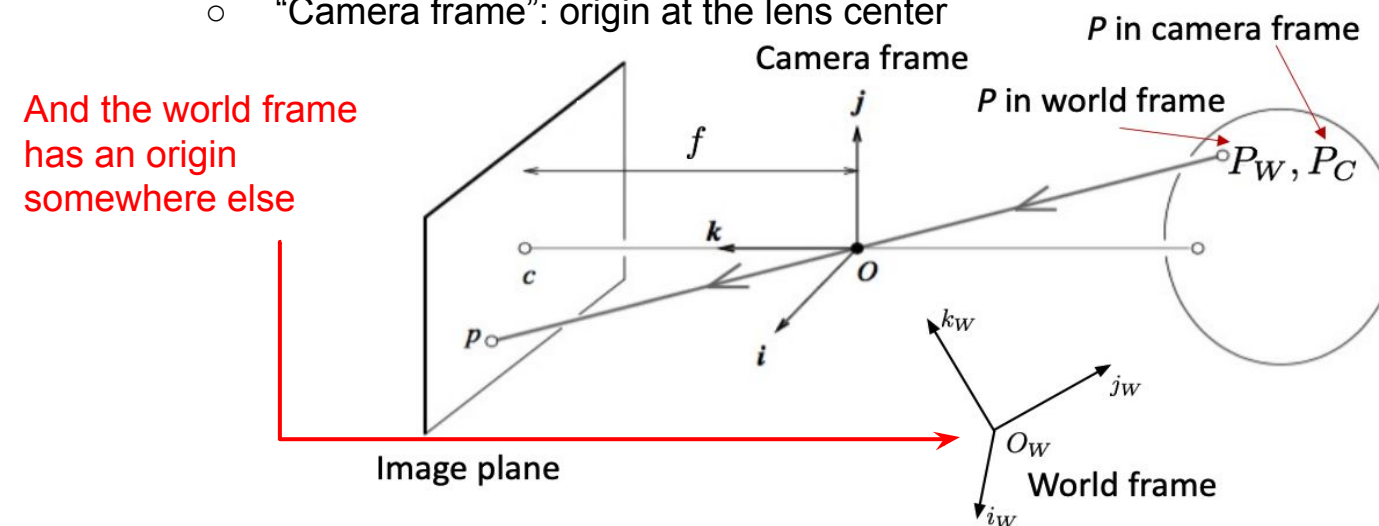
- 目标: 把世界点投影到相机图像上
- 假设: 采用针孔相机模型 (在薄透镜模型下同样成立, 假设相机对焦在 ∞)

- “坐标系 (frame)”：坐标系统
- “相机坐标系”：原点在透镜中心



Perspective Projection

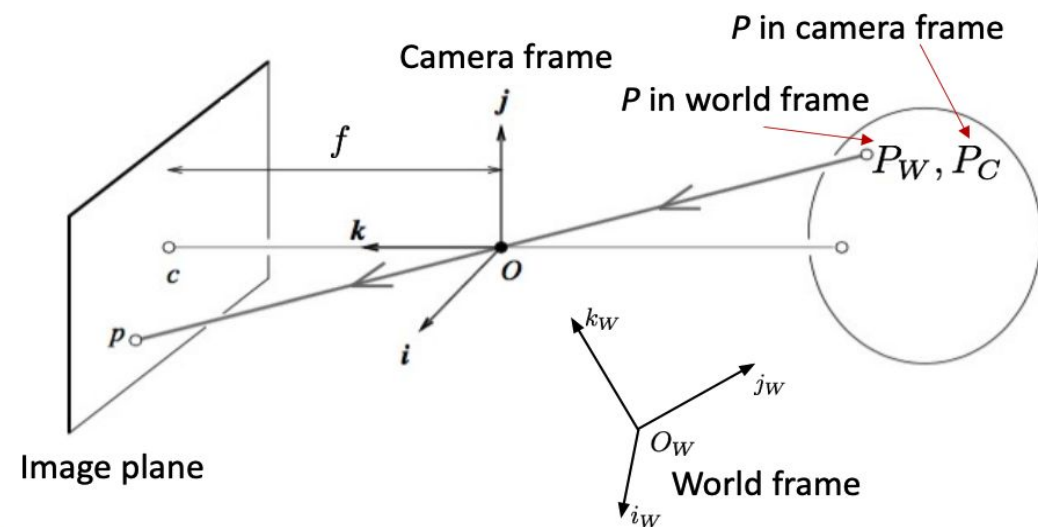
- Goal: project world points onto the camera image
- Assumption: we have a pinhole camera model (all results also hold under thin lens model, assuming camera is focused at ∞)
 - “frame”: coordinate system
 - “Camera frame”: origin at the lens center



透视投影

• 步骤

- 1. 把 P_c 映射到 p (像平面)
- 2. 把 p 映射到 (u, v) 像素坐标
- 3. 把 P_w 变换到 P_c



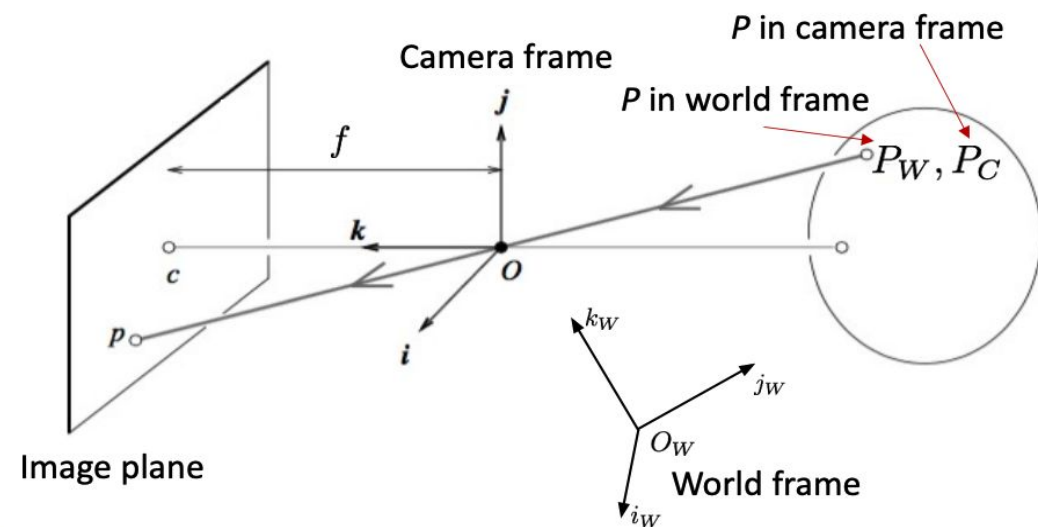
P_w 和 P_c 是同一个点, 只是处在不同坐标系 (坐标系) 中, 原点不同



Perspective Projection

• Steps

- 1. Map P_c into p (image plane)
- 2. Map p into (u, v) pixel coordinates
- 3. Transform P_w into P_c



P_w and P_c are the same point, but in different frame (coordinate system) with different origins



透视投影

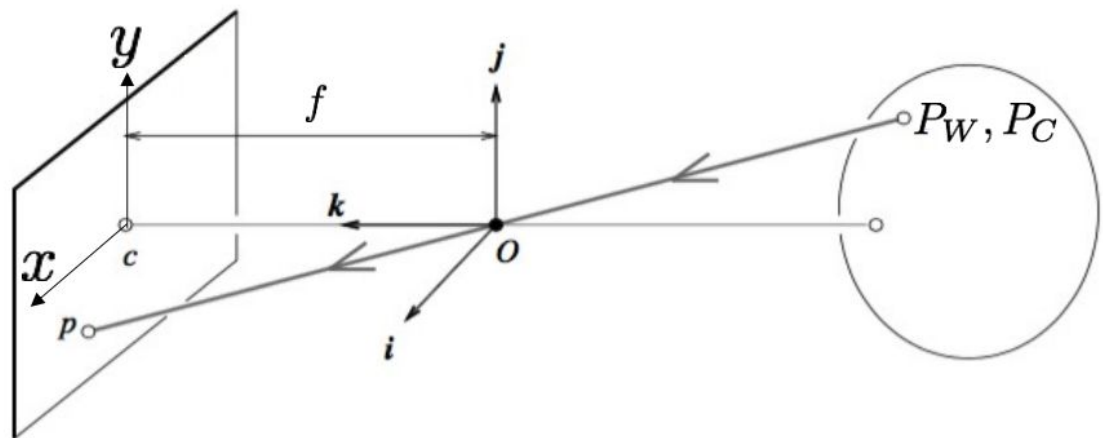
• 步骤

- 1. 把
- 我们已经有

映射到

(像平面)

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$



Perspective Projection

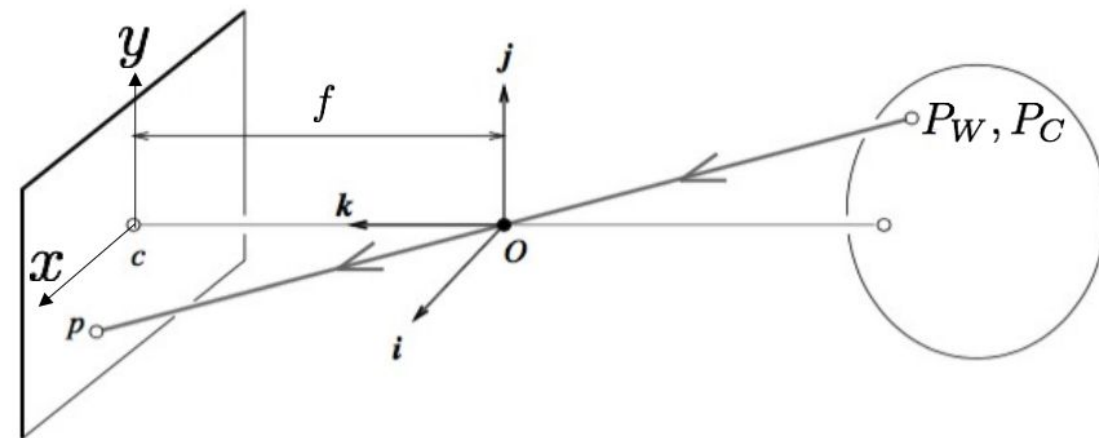
• Steps

- 1. Map $P_c = (X_C, Y_C, Z_C)$ into $p = (x, y)$ (image plane)

- We already have

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$

why?



透视投影

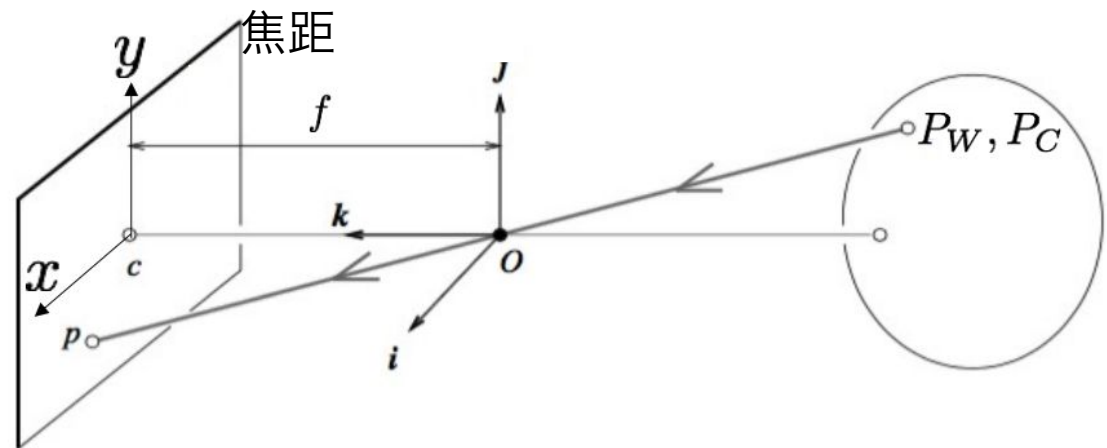
步骤

- 1. 把
- 我们已经有

映射到

(像平面)

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$



记住 P、O、p
共线 (在同一条直线上)

$$\overline{Op} = \lambda \overline{OP}$$

$$\begin{cases} x = \lambda X_C \\ y = \lambda Y_C \\ z = \lambda Z_C \end{cases}$$

$$\lambda = \frac{x}{X_C} = \frac{y}{Y_C} = \frac{z}{Z_C}$$

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases} \quad \boxtimes$$

100

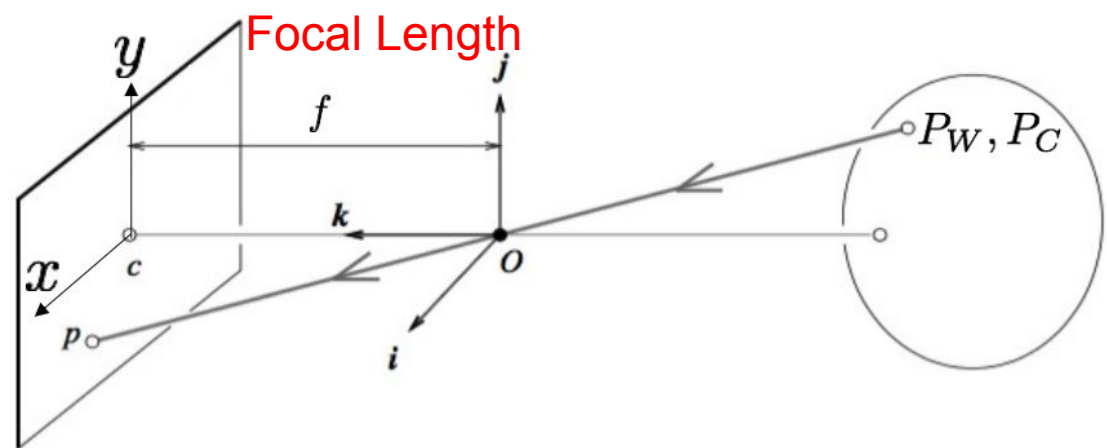
Perspective Projection

Steps

- 1. Map $P_c = (X_C, Y_C, Z_C)$ into $p = (x, y)$ (image plane)

- We already have

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$



Remember P, O, p are
collinear (on the same line)

$$\overline{Op} = \lambda \overline{OP}$$

$$\begin{cases} x = \lambda X_C \\ y = \lambda Y_C \\ z = \lambda Z_C \end{cases}$$

$$\lambda = \frac{x}{X_C} = \frac{y}{Y_C} = \frac{z}{Z_C}$$

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$

100

透视投影

步骤

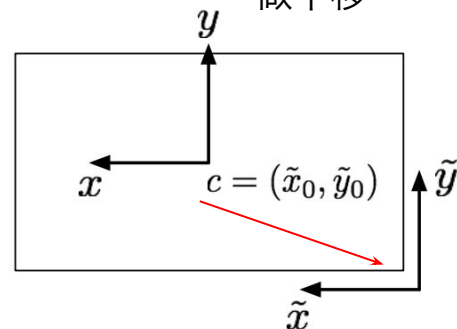
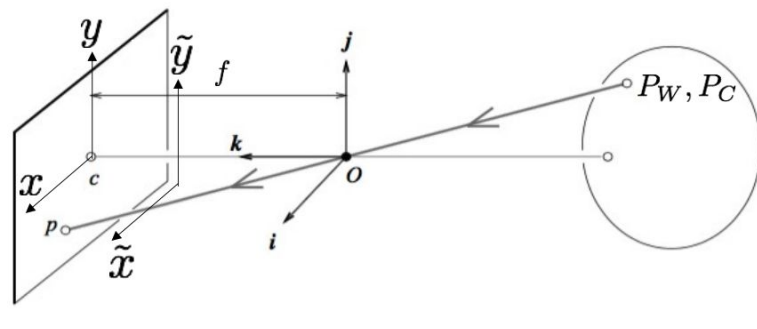
- 2. 把 $p = (x, y)$ 映射到 (u, v) 像素坐标
- 把坐标转换到像平面的角点处

这些 x, y 仍用
相机坐标系表示

我们要图像坐标系中的 x, y

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases} \quad \tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$$

假设 c 是已知的
像平面中心,
只需加上 c
做平移



Perspective Projection

Steps

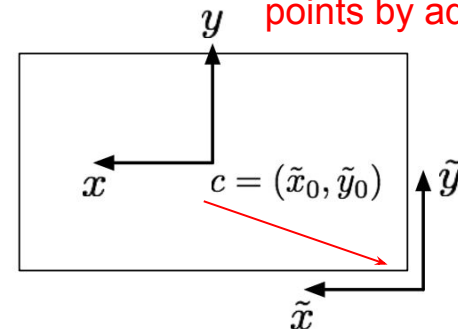
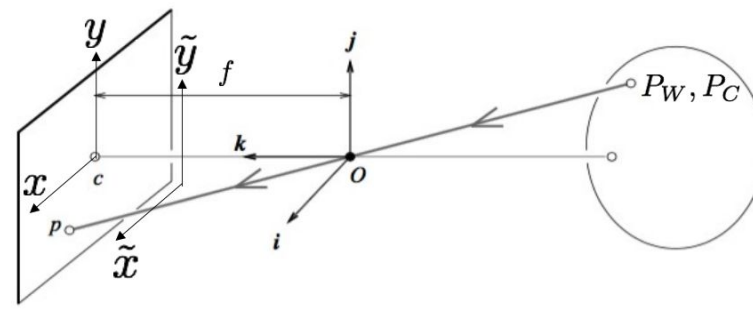
- 2. Map $p = (x, y)$ into (u, v) pixel coordinates
- **Convert the coordinates to the corners of the image plane**

These x, y are still represented using
coordinates in the camera frame

We want x, y in the image frame

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases} \quad \tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$$

Suppose c is the known
center of the image plane,
we can simply translate the
points by adding c



透视投影

步骤

- 2. 把 $\tilde{x}, \tilde{y}$ 映射到 (u,v) 像素坐标
- 把图像坐标转换成像素坐标 (图像可能以厘米等单位; 像素不同)

■ 只需改变坐标的尺度

我们已经有

$$\tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$$

图像坐标中
每单位距离的
像素
数量

$$\begin{aligned} u &= k_x \tilde{x} = \underbrace{k_x f}_{\alpha} \frac{X_C}{Z_C} + \underbrace{k_x \tilde{x}_0}_{u_0} \\ v &= k_y \tilde{y} = \underbrace{k_y f}_{\beta} \frac{Y_C}{Z_C} + \underbrace{k_y \tilde{y}_0}_{v_0} \end{aligned}$$

$\Rightarrow$

$$\begin{aligned} u &= \alpha \frac{X_C}{Z_C} + u_0 \\ v &= \beta \frac{Y_C}{Z_C} + v_0 \end{aligned}$$

Nonlinear transformation



Perspective Projection

Steps

- 2. Map $p = (x, y)$ into (u,v) pixel coordinates
- Convert the image coordinates to **pixel coordinates (image is in centimeters, etc; pixel is different)**
 - Simply change the scale of the coordinates

We already have $\tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$

Number of pixels
per unit distance
in image
coordinates

$$\begin{aligned} u &= k_x \tilde{x} = \underbrace{k_x f}_{\alpha} \frac{X_C}{Z_C} + \underbrace{k_x \tilde{x}_0}_{u_0} \\ v &= k_y \tilde{y} = \underbrace{k_y f}_{\beta} \frac{Y_C}{Z_C} + \underbrace{k_y \tilde{y}_0}_{v_0} \end{aligned}$$

$\Rightarrow$

$$\begin{aligned} u &= \alpha \frac{X_C}{Z_C} + u_0 \\ v &= \beta \frac{Y_C}{Z_C} + v_0 \end{aligned}$$

Nonlinear transformation



齐次坐标

- 我们希望把变换写成线性映射，因此需要齐次坐标

从欧氏坐标到齐次坐标。本质上就是多一维

$$\begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{pmatrix} x \\ y \\ z \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Homogeneous coordinates

- We want to represent the transformation as a linear mapping, so we need homogeneous coordinates

From euclidean coordinates to homogeneous coordinates. Basically you have one more dimension

$$\begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{pmatrix} x \\ y \\ z \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

透视投影

- 投影也可以等价地写成齐次坐标形式

相机坐标系中的 XYZ

焦距 +
主点
偏移 +
像素 - 真实世界
尺度

$$\overbrace{\begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}}^K \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha X_c + u_0 Z_c \\ \beta Y_c + v_0 Z_c \\ Z_c \end{pmatrix}$$

像素坐标中的 x,y

Camera matrix/
Matrix of intrinsic parameters

P_c in homogeneous
coordinates

Homogeneous pixel
coordinates

$$u = k_x \tilde{x} = \underbrace{\alpha}_{k_x f} \frac{X_c}{Z_c} + \underbrace{u_0}_{k_x \tilde{x}_0}$$

$$v = k_y \tilde{y} = \underbrace{\beta}_{k_y f} \frac{Y_c}{Z_c} + \underbrace{v_0}_{k_y \tilde{y}_0}$$

$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$



Perspective Projection

- Projection can be equivalently written in homogeneous coordinates

The XYZ in camera frame

Focal length +
Principal point
offset + the
pixel-real-world
scale

$$\overbrace{\begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}}^K \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha X_c + u_0 Z_c \\ \beta Y_c + v_0 Z_c \\ Z_c \end{pmatrix}$$

The x,y in pixels

Camera matrix/
Matrix of intrinsic parameters

P_c in homogeneous
coordinates

Homogeneous pixel
coordinates

$$u = k_x \tilde{x} = \underbrace{\alpha}_{k_x f} \frac{X_c}{Z_c} + \underbrace{u_0}_{k_x \tilde{x}_0}$$

$$v = k_y \tilde{y} = \underbrace{\beta}_{k_y f} \frac{Y_c}{Z_c} + \underbrace{v_0}_{k_y \tilde{y}_0}$$

$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$



透视投影

- 投影也可以等价地写成齐次坐标形式

Point p in homogeneous pixel coordinates $\nearrow p^h = [K \quad 0_{3 \times 1}] P_C^h \nwarrow$ Point P_c in homogeneous camera coordinates

Perspective Projection

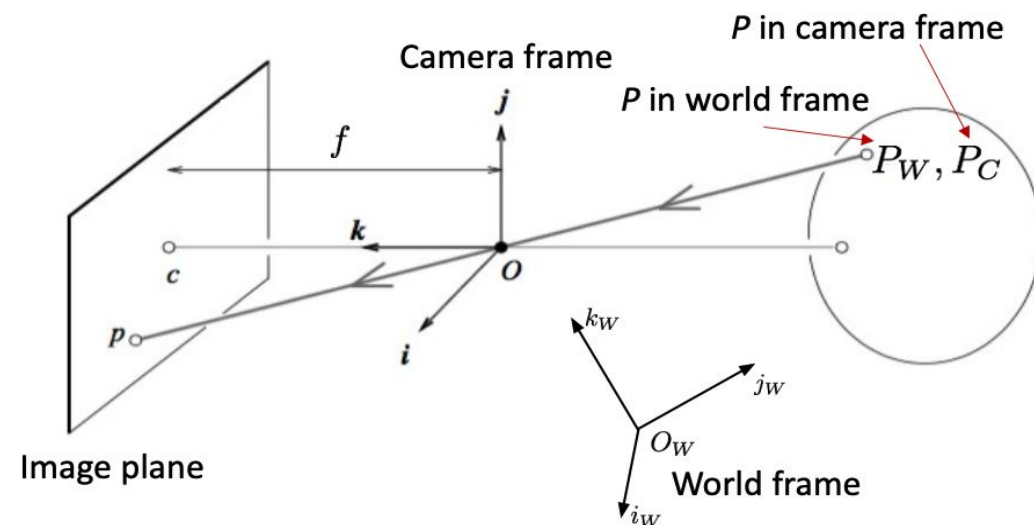
- Projection can be equivalently written in homogeneous coordinates

Point p in homogeneous pixel coordinates $\nearrow p^h = [K \quad 0_{3 \times 1}] P_C^h \nwarrow$ Point P_c in homogeneous camera coordinates

透视投影

• 步骤

- 1. 把 P_c 映射到 p (像平面) ☒
- 2. 把 p 映射到 (u, v) 像素坐标 ☒
- 3. 把 P_w 变换到 P_c

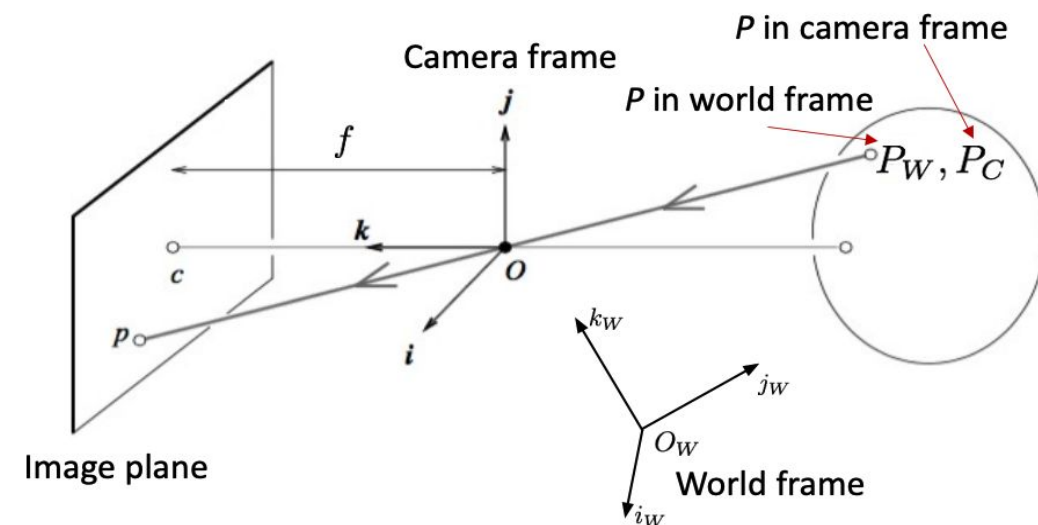


$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$

Perspective Projection

• Steps

- 1. Map P_c into p (image plane) ✓
- 2. Map p into (u, v) pixel coordinates ✓
- 3. Transform P_w into P_c

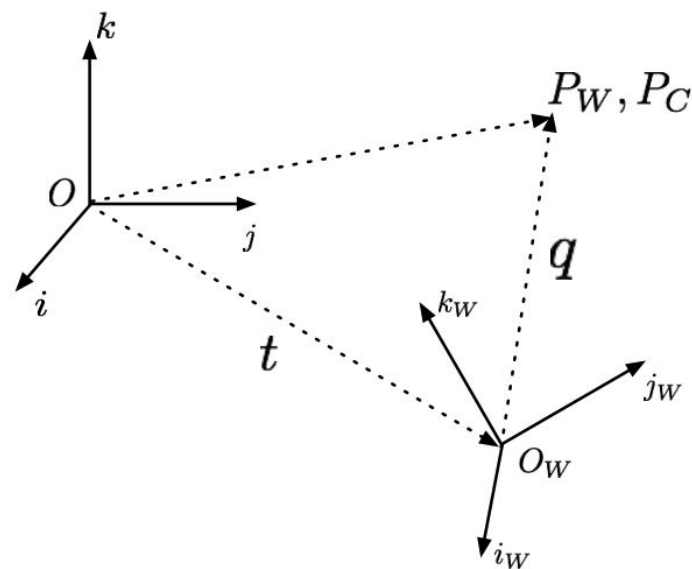


$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$

透视投影

- 步骤

- 3. 把 P_W 变换到 P_C



$$P_C = t + q$$

旋转 + 平移:
刚体变换

$$q = R P_W$$

where R is the rotation matrix
relating camera and world
frames

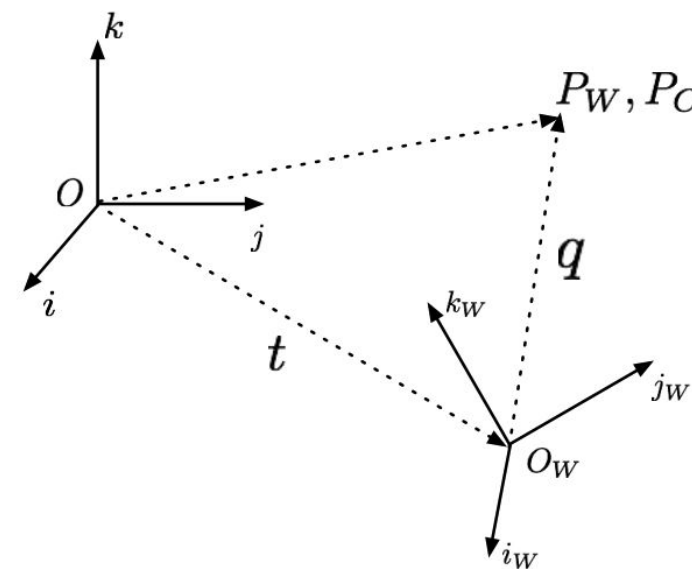
$$R = \begin{bmatrix} i_W \cdot i & j_W \cdot i & k_W \cdot i \\ i_W \cdot j & j_W \cdot j & k_W \cdot j \\ i_W \cdot k & j_W \cdot k & k_W \cdot k \end{bmatrix}$$

$$\Rightarrow P_C = t + R P_W$$

Perspective Projection

- Steps

- 3. Transform P_W into P_C



$$P_C = t + q$$

Rotation + Translation:
Rigid transformation

$$q = R P_W$$

where R is the rotation matrix
relating camera and world
frames

$$R = \begin{bmatrix} i_W \cdot i & j_W \cdot i & k_W \cdot i \\ i_W \cdot j & j_W \cdot j & k_W \cdot j \\ i_W \cdot k & j_W \cdot k & k_W \cdot k \end{bmatrix}$$

$$\Rightarrow P_C = t + R P_W$$

透视投影

- 步骤

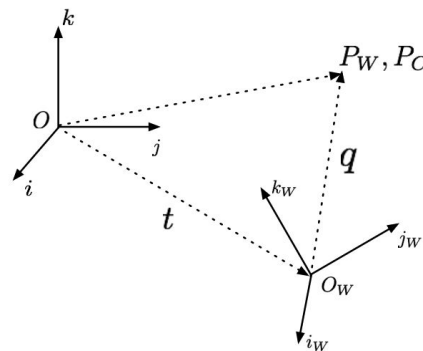
- 3. 把 P_W 变换到 P_C
 - 在齐次坐标下

$$\Rightarrow P_C = t + R P_W$$

$$\begin{pmatrix} P_C \\ 1 \end{pmatrix} = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} \begin{pmatrix} P_W \\ 1 \end{pmatrix}$$

Point P_C in homogeneous coordinates

Point P_W in homogeneous coordinates



Perspective Projection

- Steps

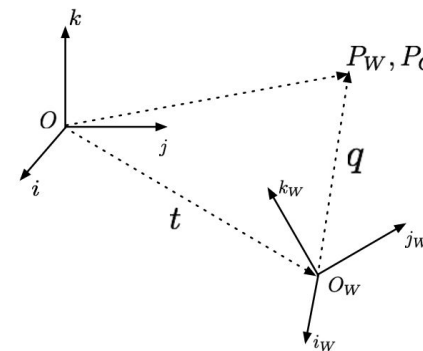
- 3. Transform P_W into P_C
 - In **homogeneous coordinates**

$$\Rightarrow P_C = t + R P_W$$

$$\begin{pmatrix} P_C \\ 1 \end{pmatrix} = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} \begin{pmatrix} P_W \\ 1 \end{pmatrix}$$

Point P_C in homogeneous coordinates

Point P_W in homogeneous coordinates



透视投影

- Collecting all results

$$p^h = [K \quad 0_{3 \times 1}] P_C^h = K [I_{3 \times 3} \quad 0_{3 \times 1}] \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} P_W^h$$

- Hence

Projection matrix M

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{K [R \quad t]}_{\text{Projection matrix } M} P_W^h \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Intrinsic parameters Extrinsic parameters

R: 旋转
t: 平移
Rt: 外参矩阵

自由度:
K: 4
Rt: 6

Perspective Projection

- Collecting all results

$$p^h = [K \quad 0_{3 \times 1}] P_C^h = K [I_{3 \times 3} \quad 0_{3 \times 1}] \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} P_W^h$$

- Hence

Projection matrix M

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{K [R \quad t]}_{\text{Projection matrix } M} P_W^h \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Degree of freedom:
K: 4
Rt: 6

透视投影

- 在 Carla 这类仿真器中，相机参数是给定的（或由用户设置）

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disal
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Lar
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for d
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

$$p^h = K[R \quad t]P_W^h$$

Intrinsic parameters Extrinsic parameters

R: 旋转
t: 平移
Rt: 外参矩阵

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disal
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Lar
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for d
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

$$p^h = K[R \quad t]P_W^h$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

透视投影

- 在 Carla 这类仿真器中，相机参数是给定的（或由用户设置）
 - 没有焦距？只有水平视场角？

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disal
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Lar
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for d
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

$$p^h = K[R \quad t]P_W^h$$

Intrinsic parameters Extrinsic parameters

R: 旋转
t: 平移
Rt: 外参矩阵

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disal
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Lar
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for d
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

$$p^h = K[R \quad t]P_W^h$$

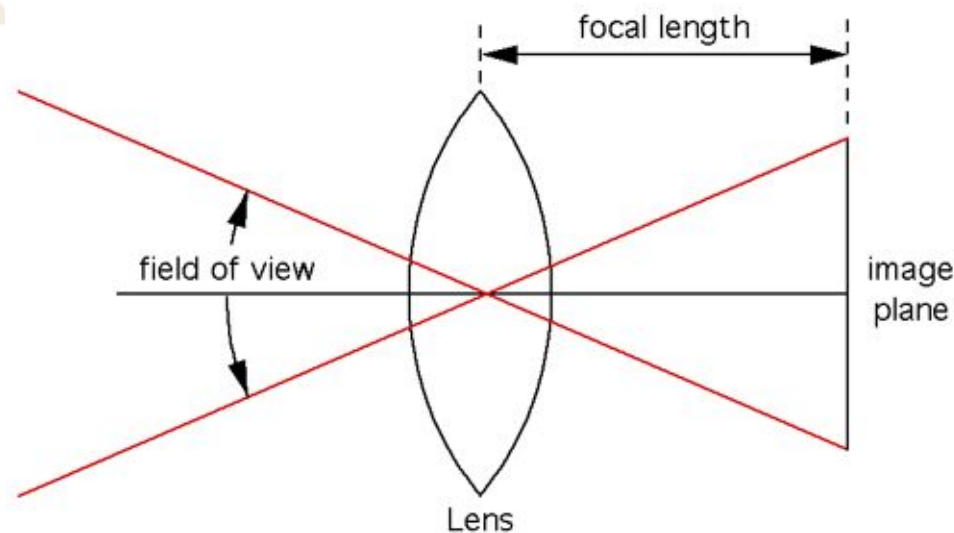
Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

透视投影

- 在 Carla 这类仿真器中，相机参数是给定的（或由用户设置）

- 没有焦距？只有水平视场角？
- 我们可以算出来



horizontal field of view = $2 \operatorname{atan}(0.5 \text{ width} / \text{focallength})$

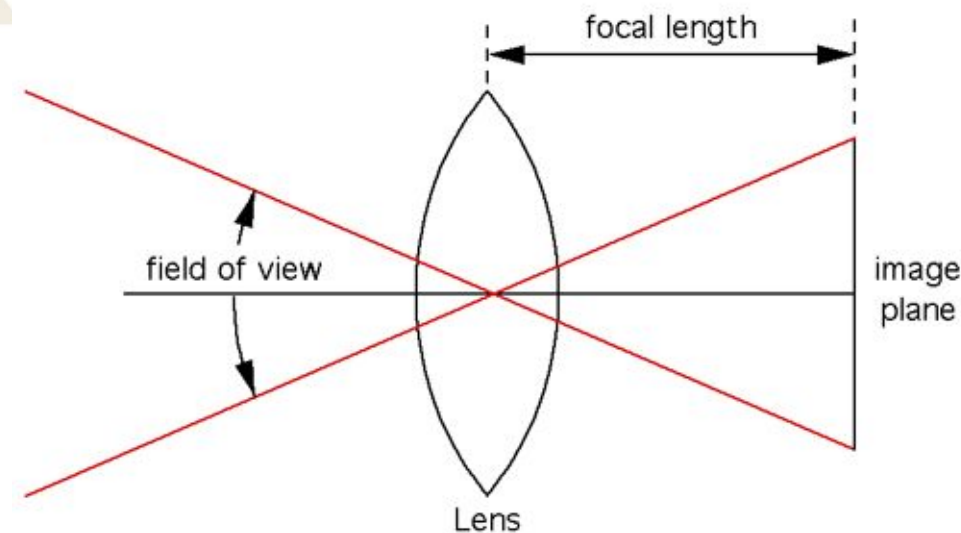
Python 代码:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)

- No focal length? Only horizontal field of view?
- We can compute that



horizontal field of view = $2 \operatorname{atan}(0.5 \text{ width} / \text{focallength})$

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

透视投影

- 在 Carla 这类仿真器中，相机参数是给定的（或由用户设置）
 - 没有焦距？只有水平视场角？
 - 这是 Carla 中完整的内参

Python 代码：

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))
Center_X = ImageSizeX / 2
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],
      [0, f, Cv],
      [0, 0, 1 ]]
```

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))
Center_X = ImageSizeX / 2
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],
      [0, f, Cv],
      [0, 0, 1 ]]
```

透视投影

- 在 Carla 这类仿真器中，相机参数是给定的（或由用户设置）
 - 没有焦距？只有水平视场角？
 - 这是 Carla 中完整的内参

Python 代码：

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))
Center_X = ImageSizeX / 2
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],
      [0, f, Cv],
      [0, 0, 1 ]]
```

如何做逆透视投影？给定 $p2d = [u, v, 1]$ ，如何得到世界点 $P = [X, Y, Z]$ ？

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))
Center_X = ImageSizeX / 2
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],
      [0, f, Cv],
      [0, 0, 1 ]]
```

How to do inverse perspective projection? Given $p2d = [u, v, 1]$, how to get world point $P = [X, Y, Z]$?

透视投影

- 在 Carla 这类仿真器中，相机参数是给定的（或由用户设置）
 - 没有焦距？只有水平视场角？
 - 这是 Carla 中完整的内参

Python 代码：

```
K = [[f, 0, Cu],  
      [0, f, Cv],  
      [0, 0, 1 ]]
```

```
def compute_intrinsic(img_width, img_height, fov):  
    """Compute intrinsic matrix."""  
    intrinsic = np.identity(3)  
    intrinsic[0, 2] = img_width / 2.0  
    intrinsic[1, 2] = img_height / 2.0  
    intrinsic[0, 0] = intrinsic[1, 1] = img_width / (2.0 * np.tan(fov *  
                                                                np.pi / 360.0))  
  
    return intrinsic
```

如何做逆透视投影？给定 $p2d = [u, v, 1]$ ，如何得到世界点 $P = [X, Y, Z]$ ？
 Rt （相机外参）在三维仿真中始终已知

$$P = (inv(K) * p2d) * depth * Rt$$

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
K = [[f, 0, Cu],  
      [0, f, Cv],  
      [0, 0, 1 ]]
```

```
def compute_intrinsic(img_width, img_height, fov):  
    """Compute intrinsic matrix."""  
    intrinsic = np.identity(3)  
    intrinsic[0, 2] = img_width / 2.0  
    intrinsic[1, 2] = img_height / 2.0  
    intrinsic[0, 0] = intrinsic[1, 1] = img_width / (2.0 * np.tan(fov *  
                                                                np.pi / 360.0))  
  
    return intrinsic
```

How to do inverse perspective projection? Given $p2d = [u, v, 1]$, how to get world point $P = [X, Y, Z]$?
 Rt (camera extrinsic) is known in 3D simulation at all times

$$P = (inv(K) * p2d) * depth * Rt$$

目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 几何计算机视觉
 - 相机模型
 - 透视投影
 - 相机标定
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

相机标定

- 相机标定的目标是得到相机的内参和外参

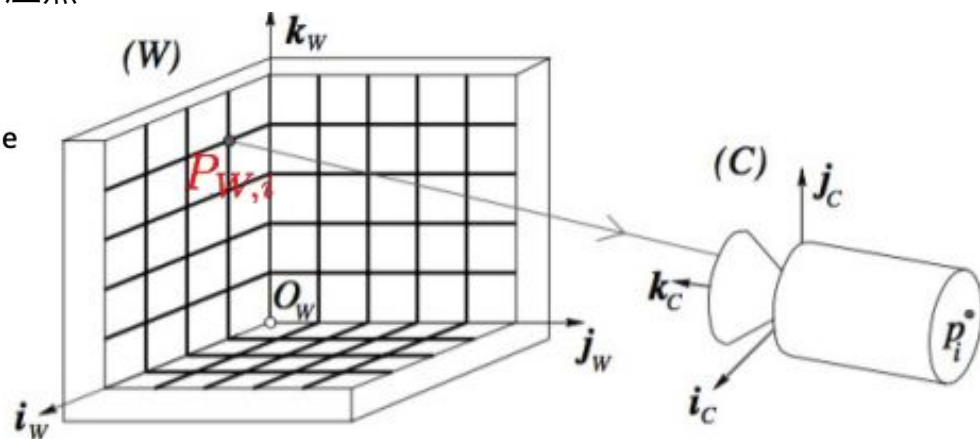
- 我们需要知道 10* 个参数 (K 有 4 个, R 有 3 个, t 有 3 个)
- 策略

■ 需要足够多 (至少 6 组) 对应点

$$p_i \leftrightarrow P_{W,i}$$

$P_{W,1}, P_{W,2}, \dots, P_{W,n}$ with **known** positions in world frame

$p_1, p_2, \dots, p_n$ with **known** positions in image frame



* 若包含偏斜 (skewness) 则为 11

Camera Calibration

- The target for camera calibration is to get the **intrinsic and extrinsic parameters** of the camera

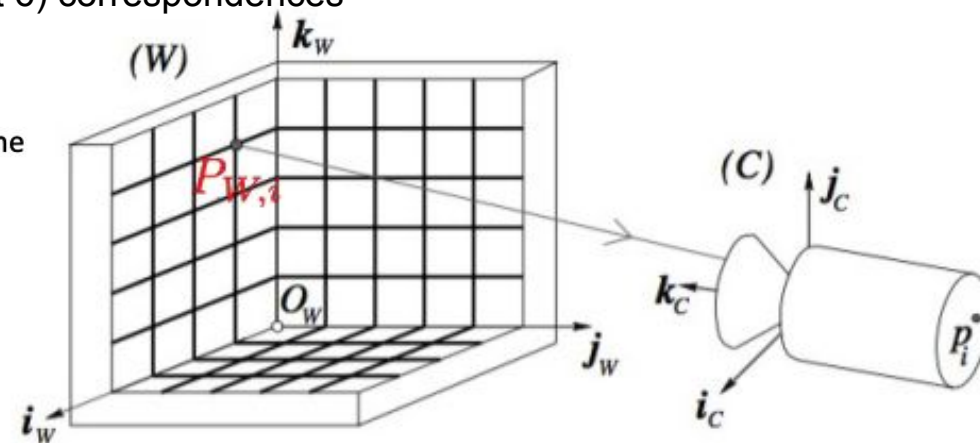
- We need to know 10* parameters (4 for K, 3 for R, 3 for t)
- Strategy

■ We need enough (at least 6) correspondences

$$p_i \leftrightarrow P_{W,i}$$

$P_{W,1}, P_{W,2}, \dots, P_{W,n}$ with **known** positions in world frame

$p_1, p_2, \dots, p_n$ with **known** positions in image frame



* or 11 if we include skewness

相机标定

- 相机标定的目标是得到相机的内参和外参

- 我们需要知道 10* 个参数 (K 有 4 个, R 有 3 个, t 有 3 个)
- 策略

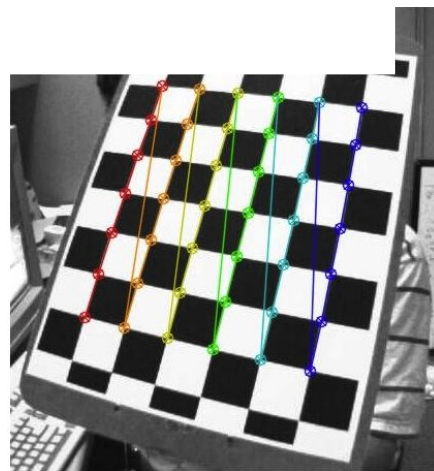
- 需要足够多 (至少 6 组) 对应点

$$p_i \leftrightarrow P_{w,i}$$

- 方法

- 借助棋盘格和 OpenCV...

- 对某些图案来说, 对应点很容易计算
- https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html



`cv.findChessboardCorners()`

`cv.calibrateCamera()`

* 若包含偏斜 (skewness) 则为 11

Camera Calibration

- The target for camera calibration is to get the **intrinsic and extrinsic parameters** of the camera

- We need to know 10* parameters (4 for K, 3 for R, 3 for t)
- Strategy

- We need enough (at least 6) correspondences

$$p_i \leftrightarrow P_{w,i}$$

- Method

- With the help of a chessboard and OpenCV...

- It is easy to compute correspondences for some patterns
- https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html



`cv.findChessboardCorners()`

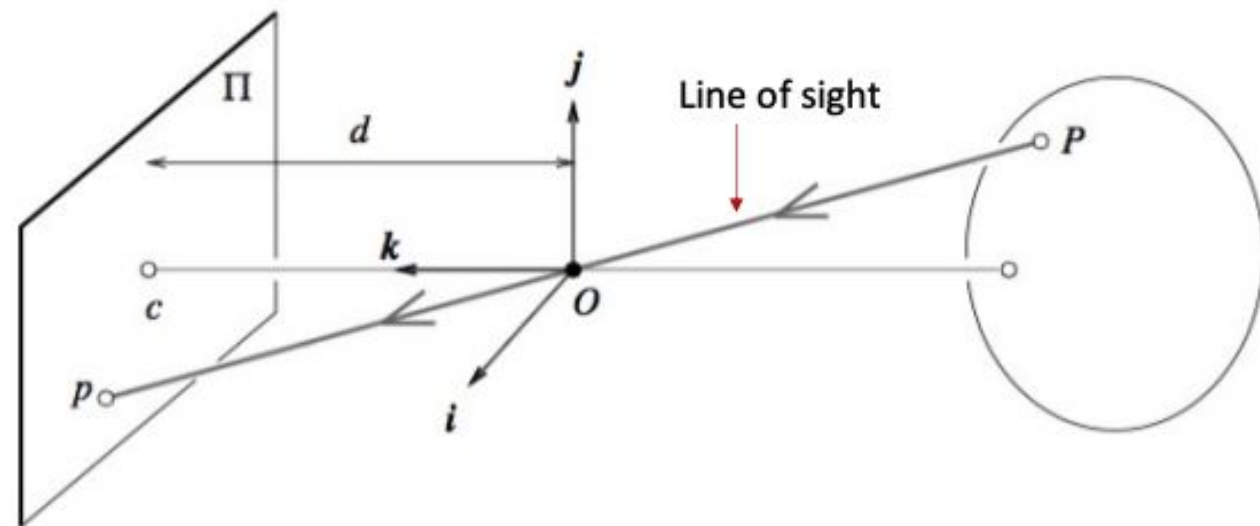
`cv.calibrateCamera()`

* or 11 if we include skewness

测量深度 —— 三维重建

• 如果相机已标定（内参和外参已知），给定图像点 p ，能否得到三维世界中的点 P ？

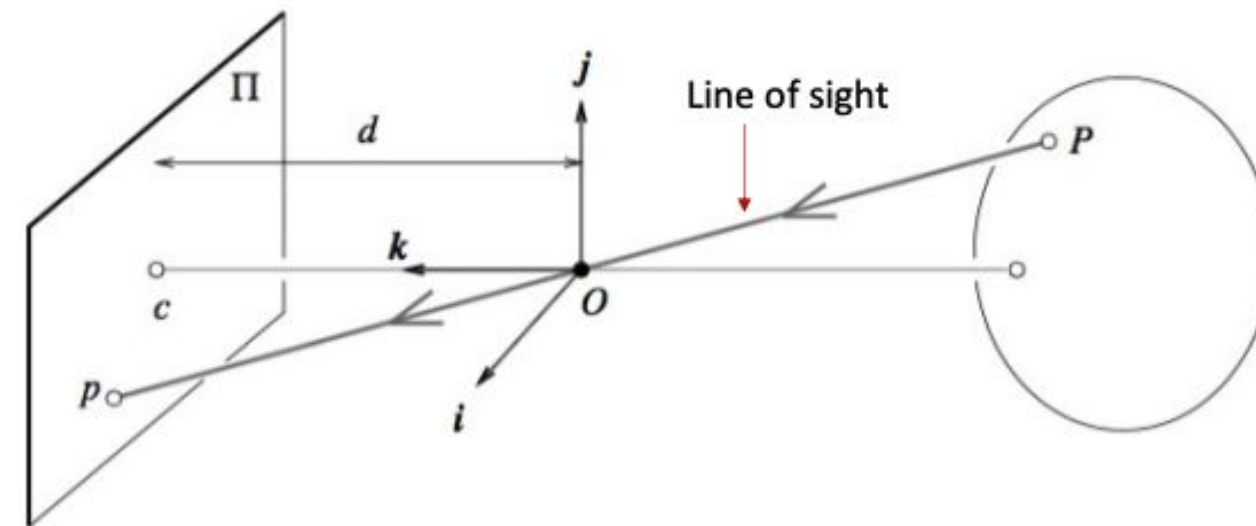
○ 不能！我们只知道 P 在直线 OP 上



Measuring depth - 3D reconstruction

• If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?

○ No! We can only know P is on the line OP



测量深度 —— 三维重建

- 如果相机已标定（内参和外参已知），给定图像点 p ，能否得到三维世界中的点 P ？

- 不能！我们只知道 P 在直线 OP 上
- 那怎么办？

- 立体视觉

- 两台相机，相对位姿已知（就像我们的眼睛）

- 运动恢复结构（SfM）

- 同一相机，多张图像

- LiDAR!

Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?

- No! We can only know P is on the line OP
 - So how?

- Stereo vision

- Two cameras, relative pose is known (like our eyes)

- Structure from motion

- Same camera, multiple images

- LiDAR!

AI 应用 —— 来自我们的本科生暑期科研项目

现在我们已经能检测物体（比如网球）。能否估计它的速度？

答案：可以！用深度相机！



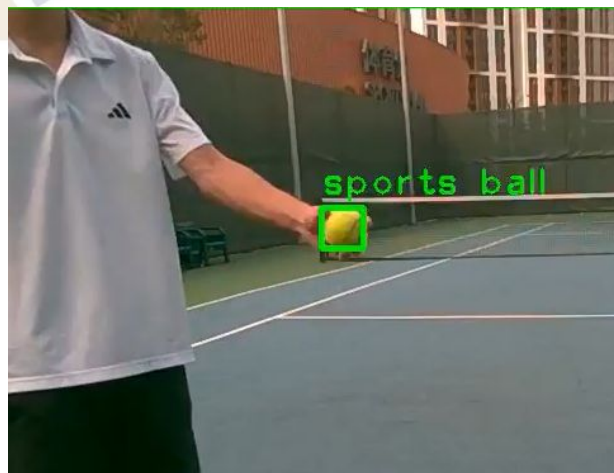
速度？



AI Applications - From Our UG Summer Research Project

Since now we can detect an object (like a tennis ball). Can we estimate its speed?

Answer: Yes! With a depth camera!



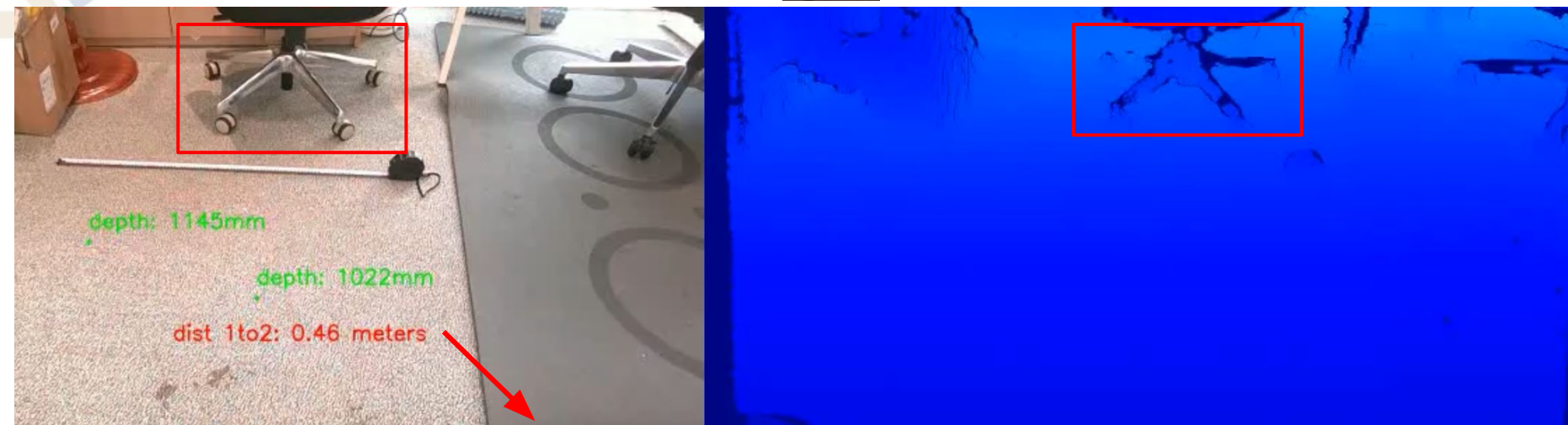
Speed?



AI 应用 —— 来自我们的本科生暑期科研项目

现在我们已经能检测物体（比如网球）。能否估计它的速度？

答案：可以！用深度相机！

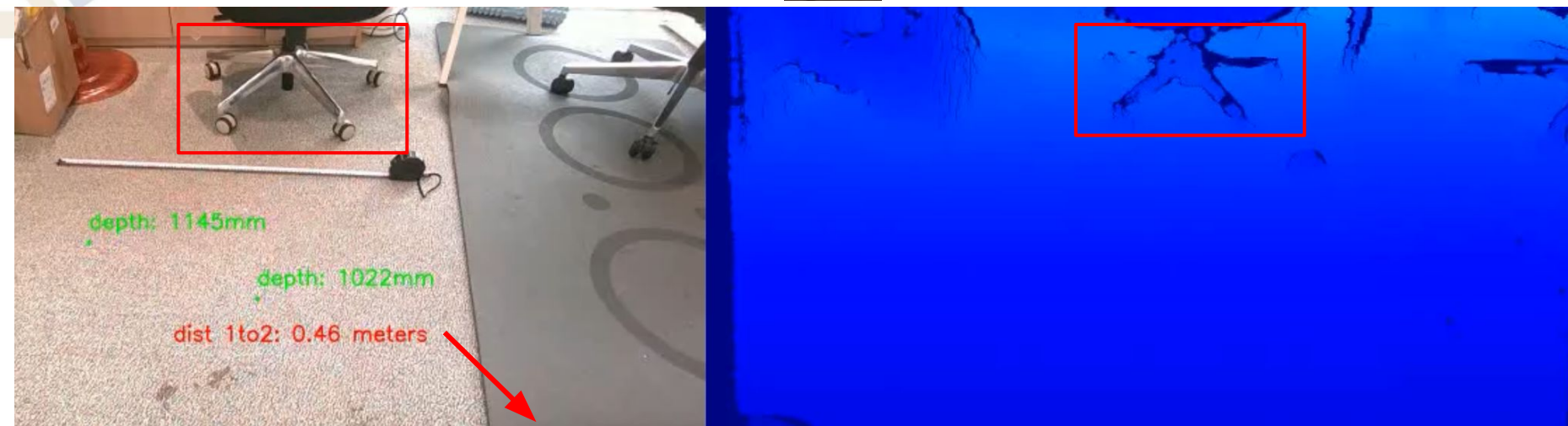


深度相机会告诉你每个像素有多远。因此可以计算真实世界中的距离。

AI Applications - From Our UG Summer Research Project

Since now we can detect an object (like a tennis ball). Can we estimate its speed?

Answer: Yes! With a depth camera!



The depth camera tells you how far away each pixel is. So you can compute real-world distance measurements.

测量深度 —— 三维重建

• 如果相机已标定（内参和外参已知），给定图像点 p ，能否得到三维世界中的点 P ？

- 不能！我们只知道 P 在直线 OP 上
- 但仍可以从像平面映射到地面（BEV!!）

- 需要知道地平面的参数
- 单应性矩阵 $\text{Homography} = K_2 * R_{21} * \text{inv}(K_1)$
- 这能让你...

需要知道：图像相机和地面相机的内参以及相对外参

Measuring depth - 3D reconstruction

• If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?

- No! We can only know P is on the line OP
- But we can still get from **image plane to the ground plane** (BEV!!)
 - Need to know the ground plane's parameters
 - Homography matrix = $K_2 * R_{21} * \text{inv}(K_1)$
 - This allow you to..

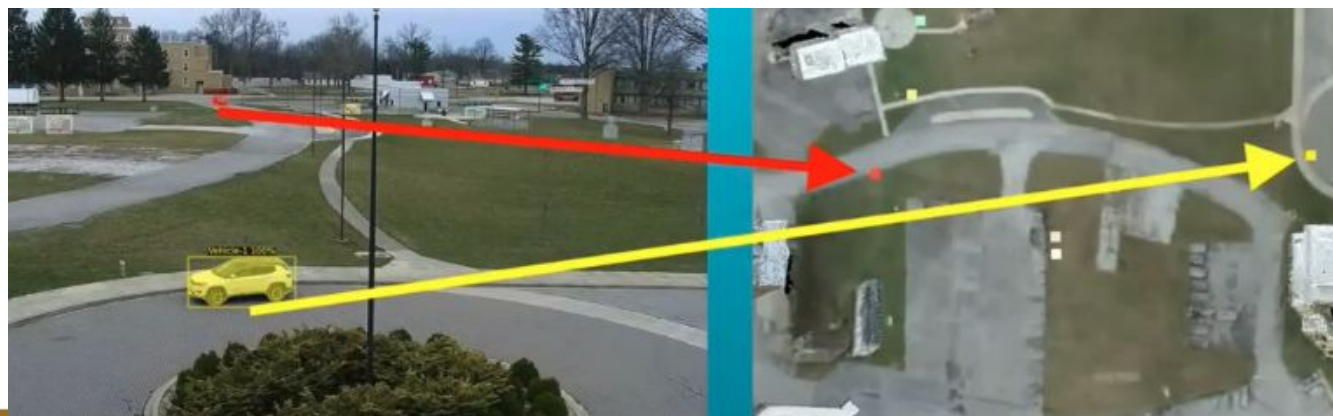
Need to know: Intrinsic of the image camera and the ground plane camera + the relative extrinsics

测量深度 —— 三维重建

• 如果相机已标定（内参和外参已知），给定图像点 p ，能否得到三维世界中的点 P ？

- 不能！我们只知道 P 在直线 OP 上
- 但仍可以从像平面映射到地面（鸟瞰图!!）

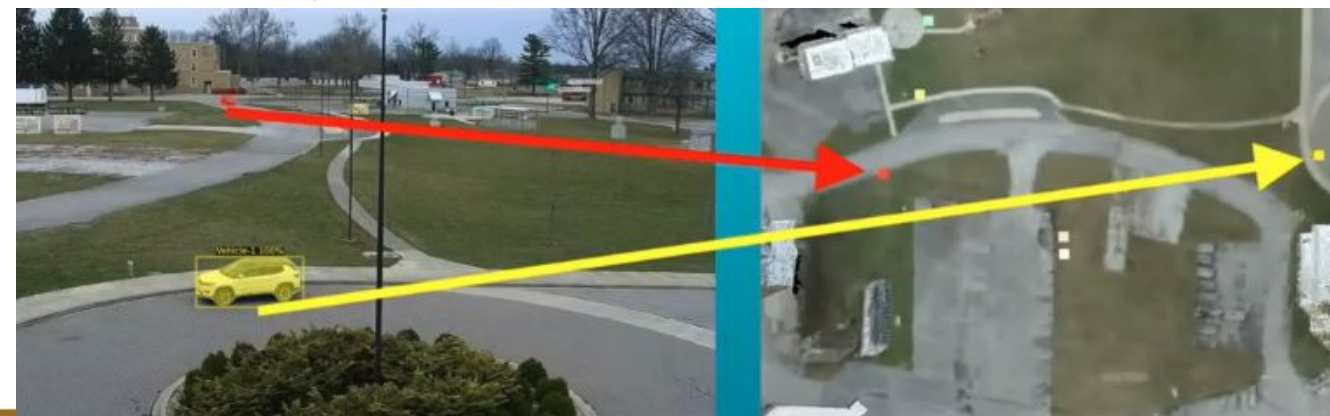
- 需要知道地平面的参数
- 单应性矩阵 Homography = $K_2 * R_{21} * \text{inv}(K_1)$
- 这能让你...



Measuring depth - 3D reconstruction

• If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?

- No! We can only know P is on the line OP
- But we can still get from **image plane to the ground plane** (Bird Eye View!!)
 - Need to know the ground plane's parameters
 - Homography matrix = $K_2 * R_{21} * \text{inv}(K_1)$
 - This allow you to..



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 几何计算机视觉
 - 相机模型
 - 透视投影
 - 相机标定
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

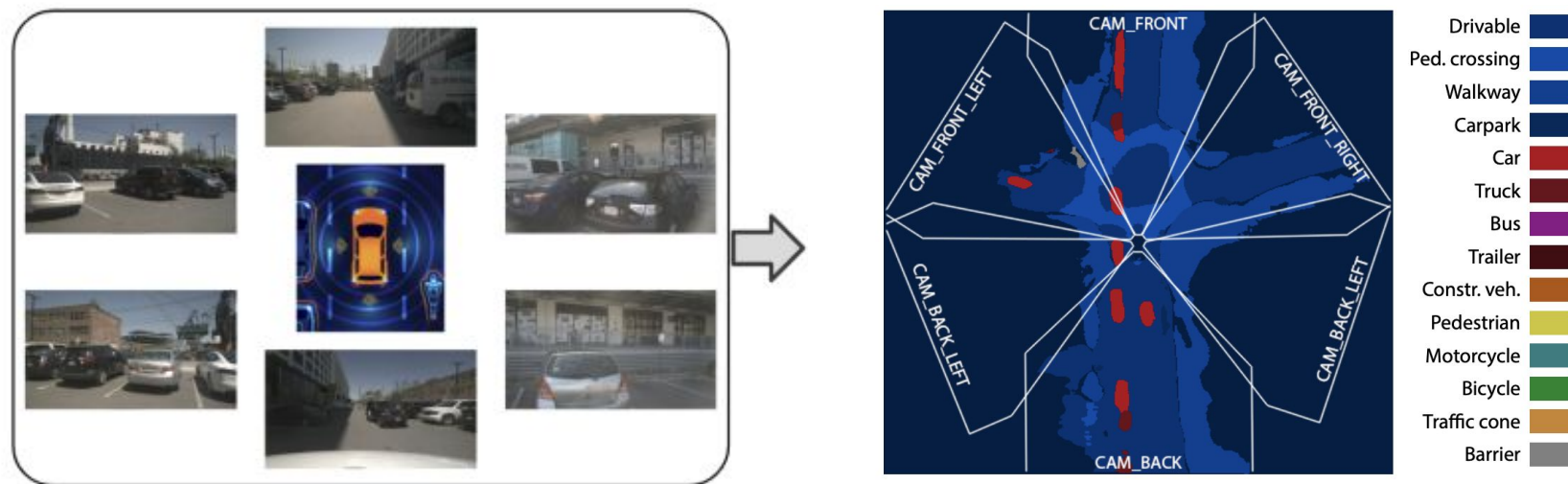
Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

鸟瞰图 (BEV) 感知

• 什么是 BEV 感知?

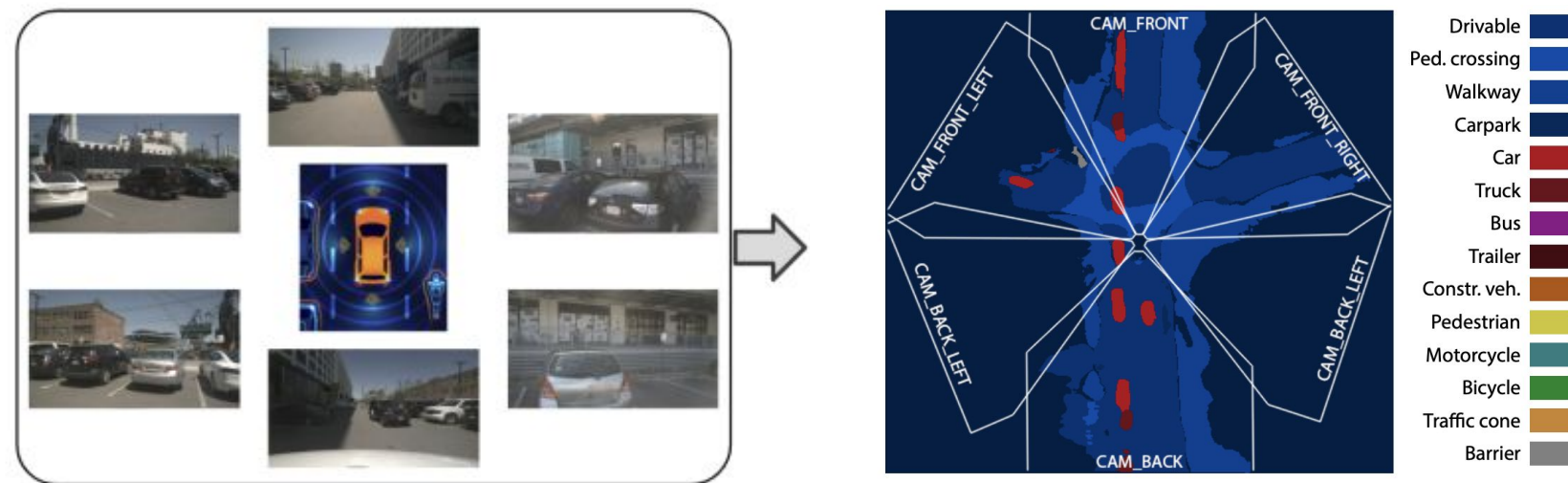
- 过去检测/分割/跟踪等是在前视图或透视视图中完成的
- 相机和传感器更多时, 需要统一视角
- 从多相机/多传感器到自上而下的统一视图 (BEV)
- 在自动驾驶算法中越来越流行



Bird-Eye-View Perception

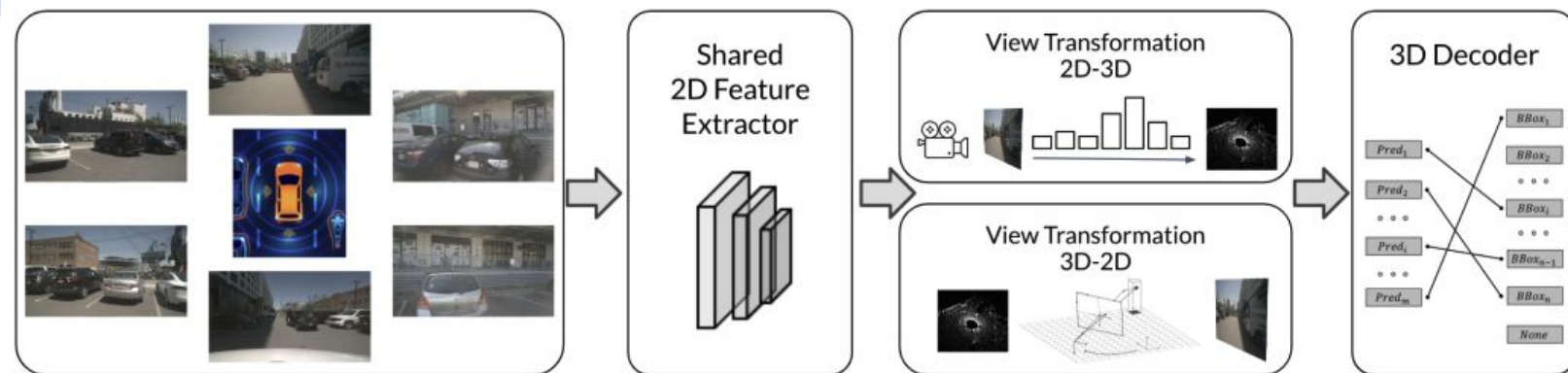
• What is BEV perception?

- Previously, detection/segmentation/tracking, etc., done in a front view or perspective view
 - With more cameras and sensors, need a unified view
- From multi-camera/sensor to a top-down unified view (BEV)
- Increasingly popular for autonomous driving algorithms



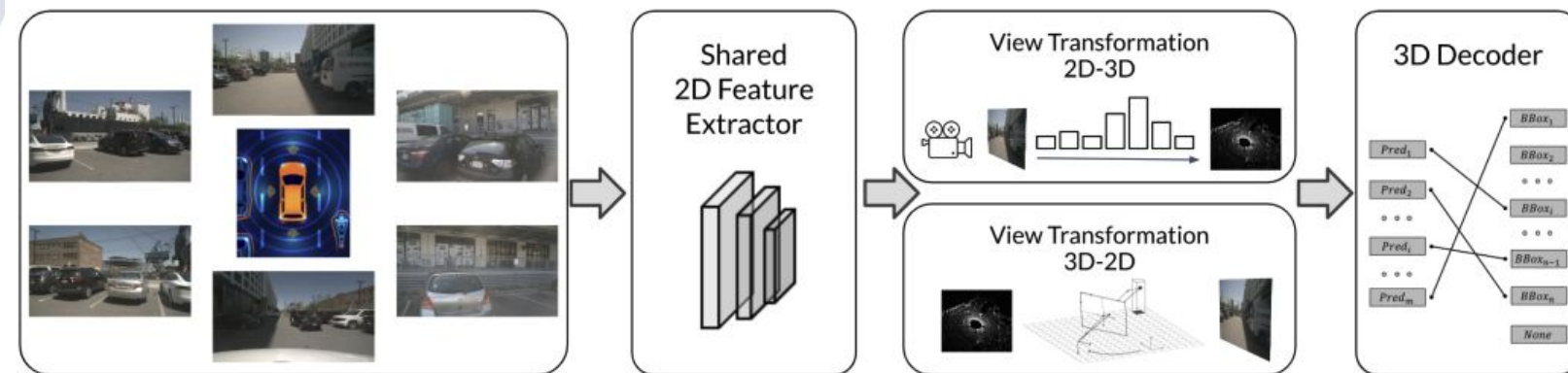
鸟瞰图 (BEV) 感知

- RGB 相机的通用 BEV 流程



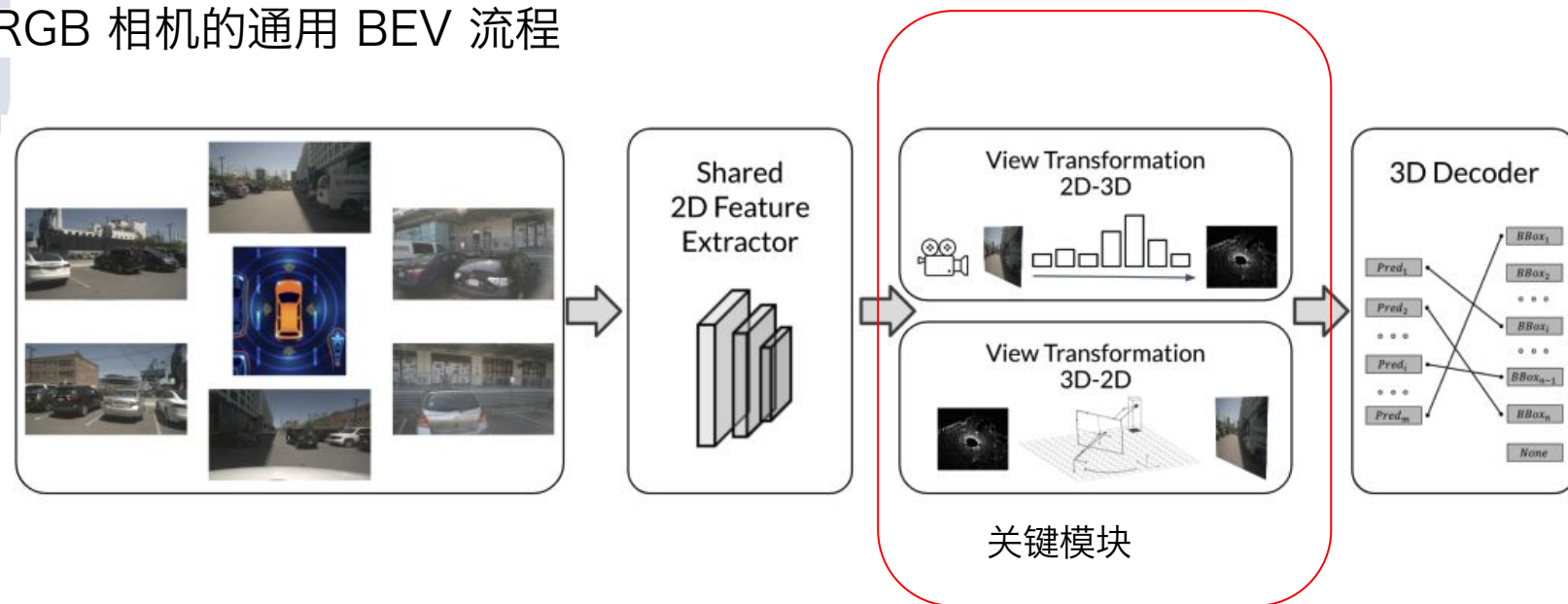
Bird-Eye-View Perception

- General BEV pipeline for RGB cameras



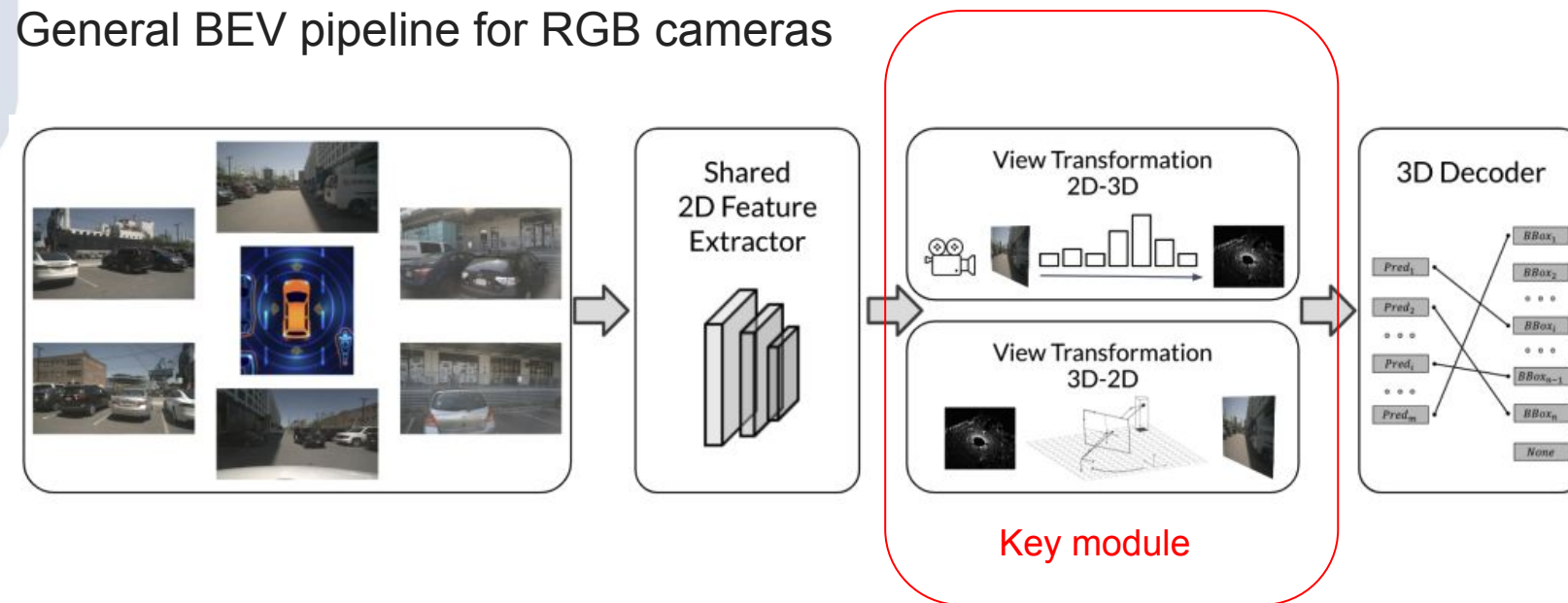
鸟瞰图 (BEV) 感知

- RGB 相机的通用 BEV 流程



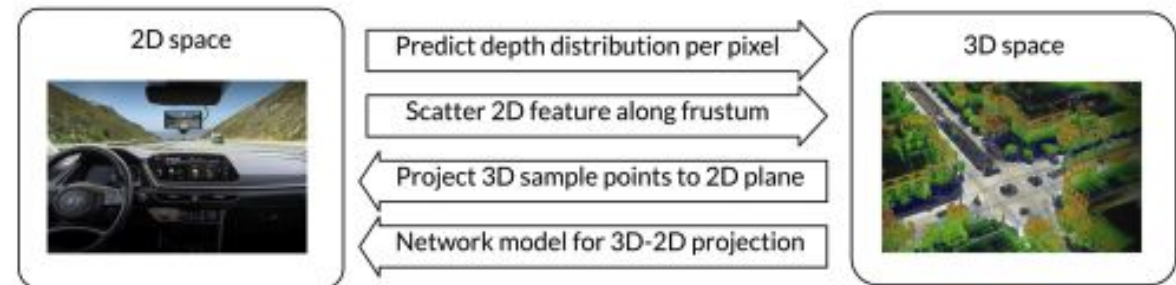
Bird-Eye-View Perception

- General BEV pipeline for RGB cameras



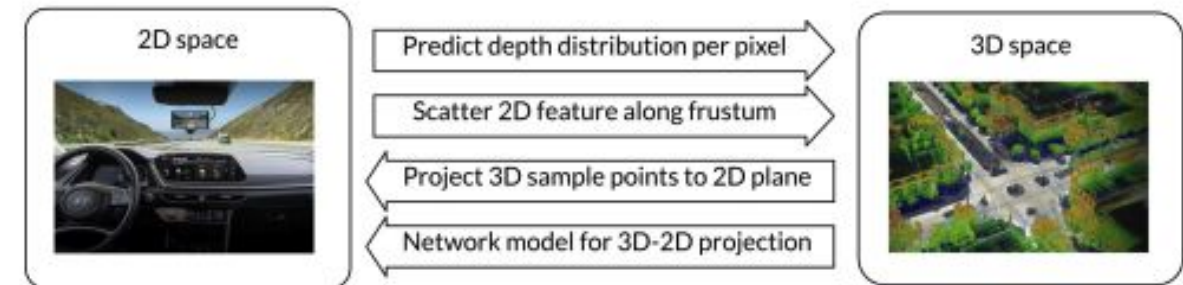
鸟瞰图（BEV）感知

- RGB 相机的通用 BEV 流程
 - 2D-3D 还是 3D-2D?



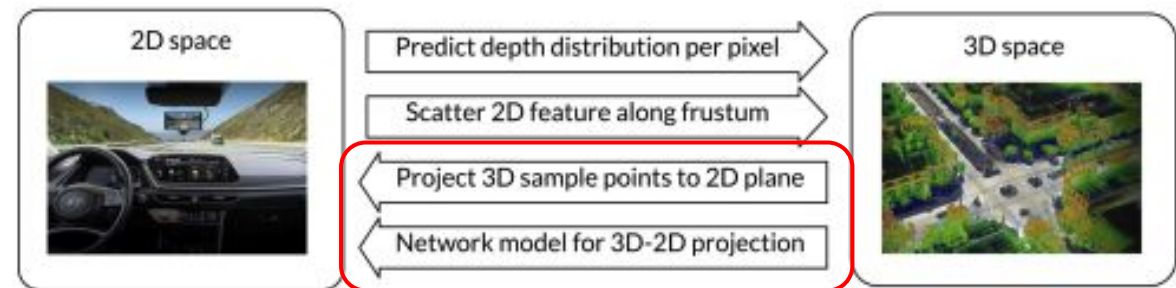
Bird-Eye-View Perception

- General BEV pipeline for RGB cameras
 - 2D-3D or 3D-2D?



鸟瞰图 (BEV) 感知

- RGB 相机的通用 BEV 流程
 - 2D-3D 还是 3D-2D?

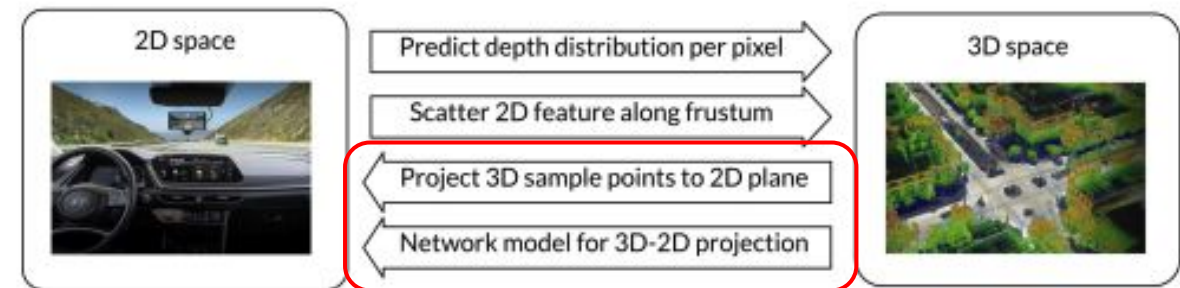


从 3D 到 2D

1. 在 BEV 地图上, 对每个 (3D) 点, 用相机参数投影到相机 2D 平面
2. 从 2D 空间采样特征

Bird-Eye-View Perception

- General BEV pipeline for RGB cameras
 - 2D-3D or 3D-2D?

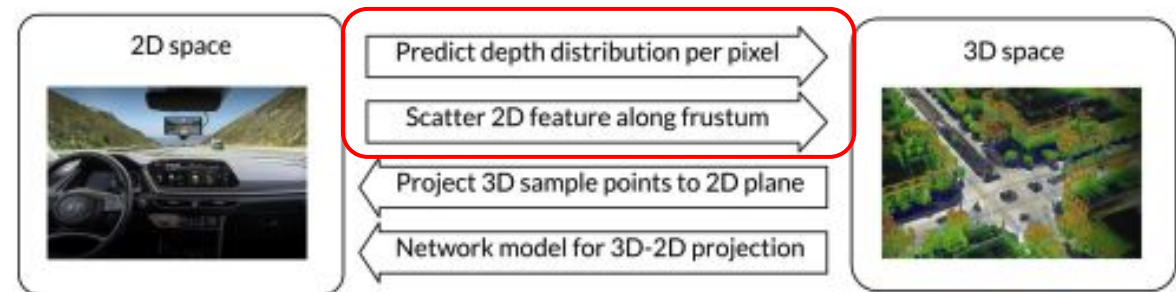


From 3D to 2D

1. On the BEV map, for each (3D) point, project to the camera 2D plane using camera parameters
2. Sample feature from 2D space

鸟瞰图 (BEV) 感知

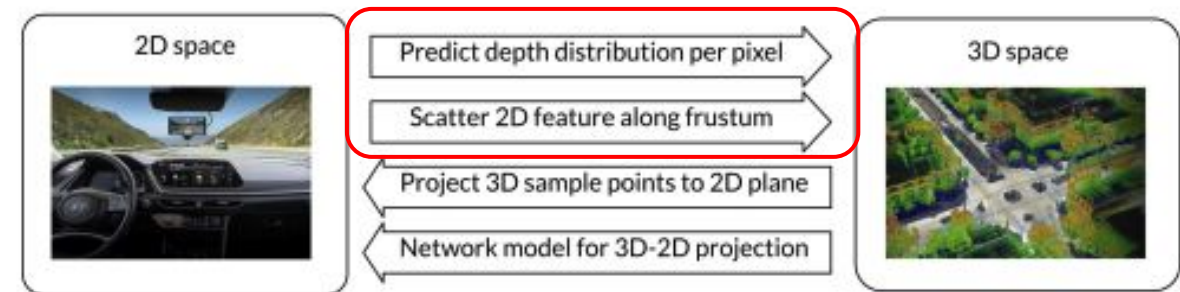
- RGB 相机的通用 BEV 流程
 - 2D-3D 还是 3D-2D?



从 2D 到 3D?
如何直接在 3D 空间得到特征?

Bird-Eye-View Perception

- General BEV pipeline for RGB cameras
 - 2D-3D or 3D-2D?



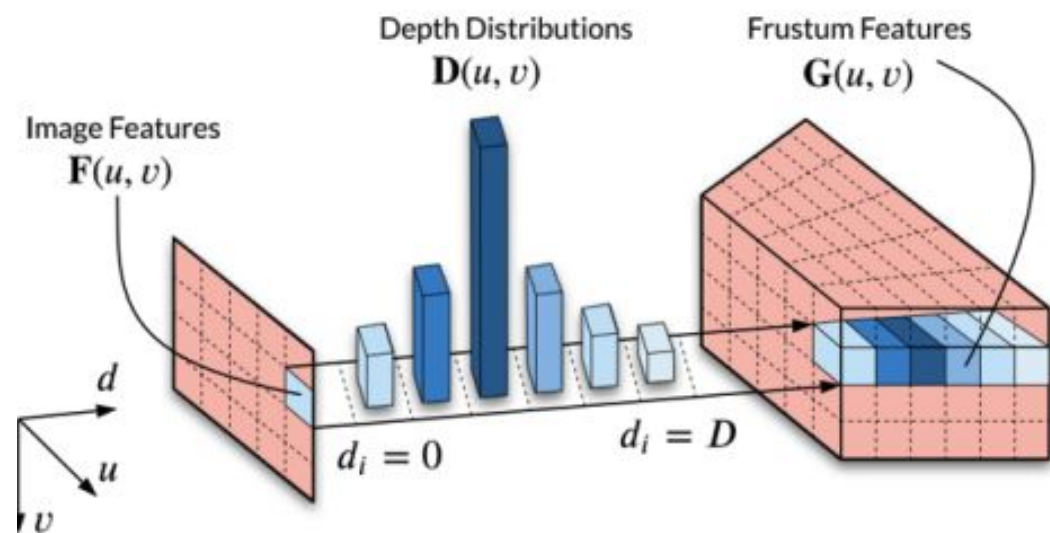
From 2D to 3D?
How do we get feature in 3D space directly?

鸟瞰图 (BEV) 感知

- RGB 相机的通用 BEV 流程

- 2D-3D

- 预测每像素深度, 作为沿投影锥的特征分布
 - 2D 特征 -> 3D 体素

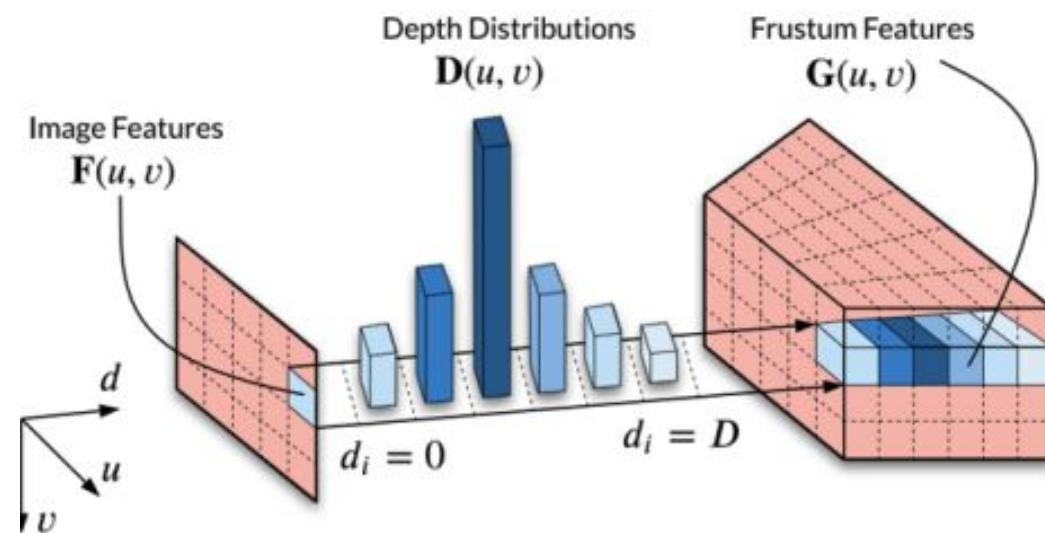


Bird-Eye-View Perception

- General BEV pipeline for RGB cameras

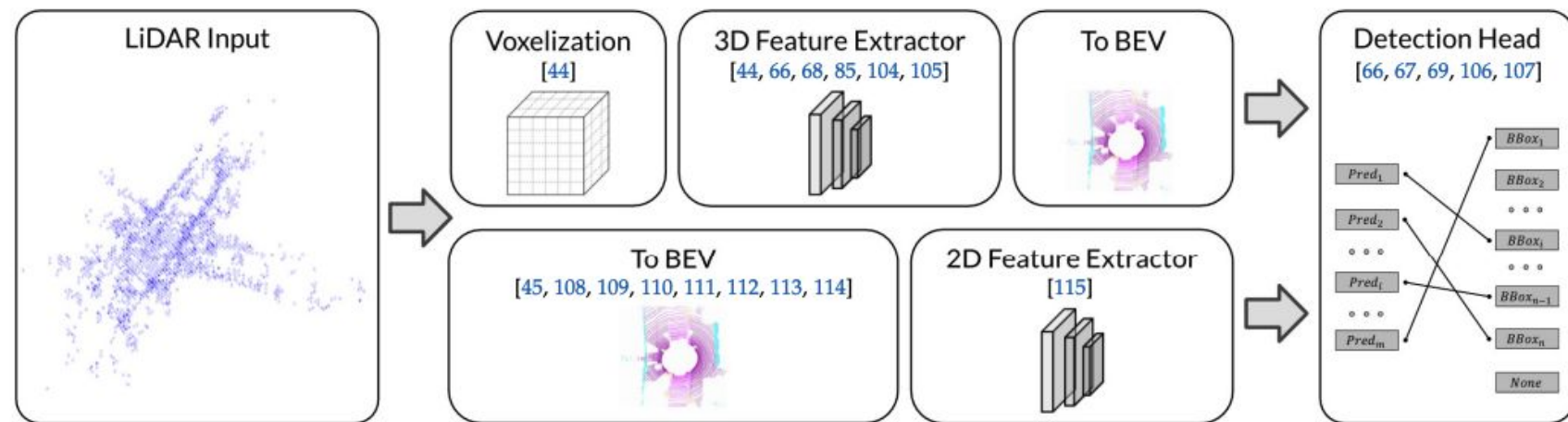
- 2D-3D

- Predict per-pixel depth as feature distribution along the projection cone
 - 2D features -> 3D voxels



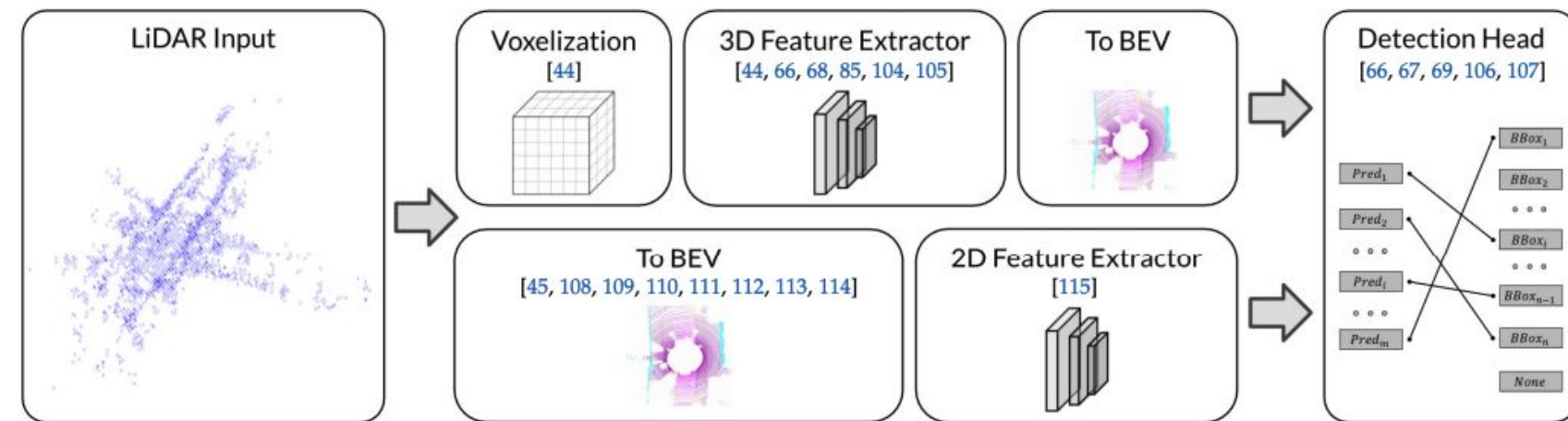
鸟瞰图 (BEV) 感知

- LIDAR / 雷达的通用 BEV 流程
 - LIDAR 输入是三维点云



Bird-Eye-View Perception

- General BEV pipeline for LIDAR / Radar
 - LIDAR inputs are point clouds in 3D

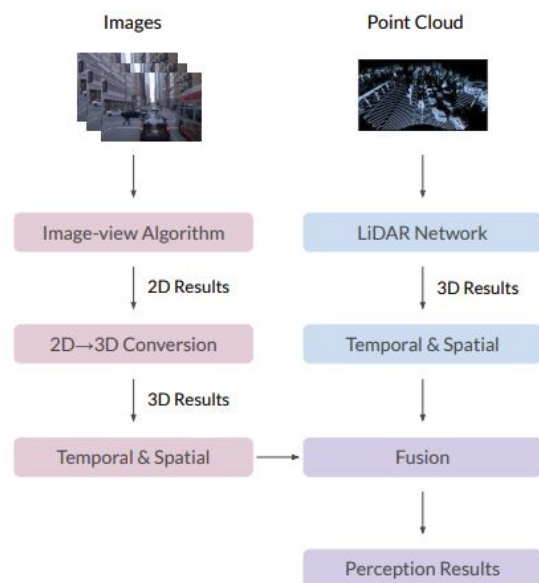


鸟瞰图 (BEV) 感知

- 融合 RGB 与 LIDAR

○ 以前

后期融合



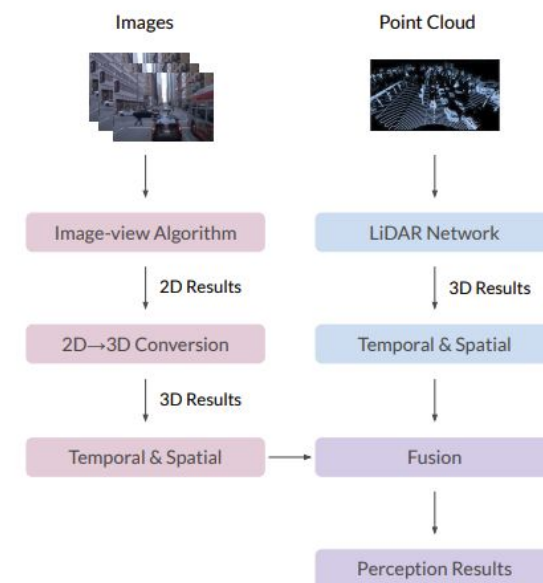
(a) Perspective view (PV) perception pipeline

Bird-Eye-View Perception

- Fusing RGB and LIDAR

○ Before

Late fusion

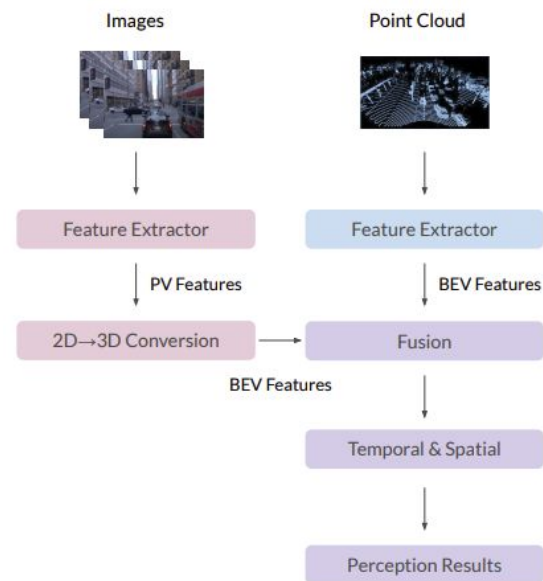


(a) Perspective view (PV) perception pipeline

鸟瞰图 (BEV) 感知

- 融合 RGB 与 LIDAR

○ 现在

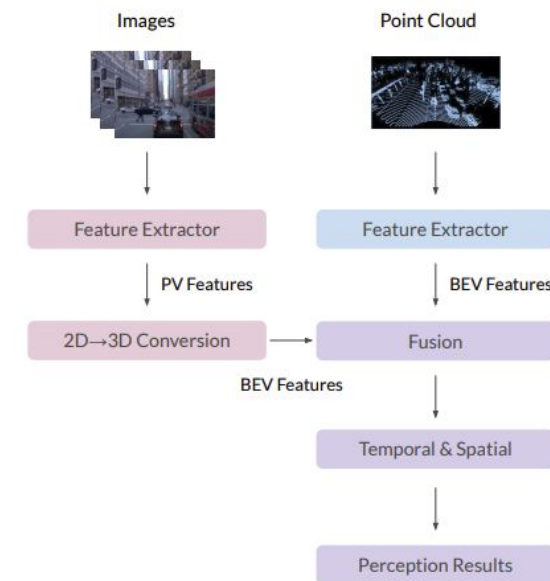


(b) BEV perception pipeline (BEV Fusion)

Bird-Eye-View Perception

- Fusing RGB and LIDAR

○ Now

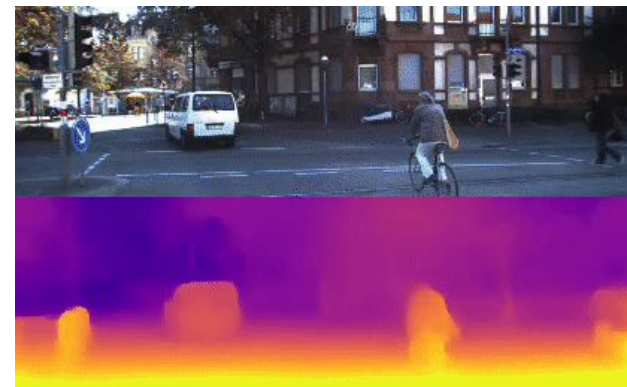
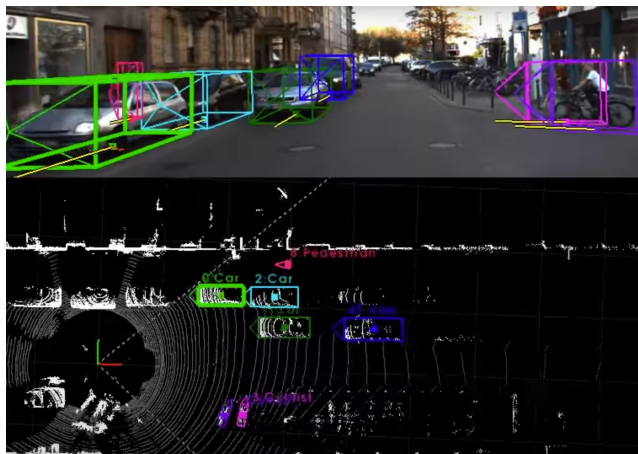


(b) BEV perception pipeline (BEV Fusion)

鸟瞰图 (BEV) 感知

- BEV 感知数据集

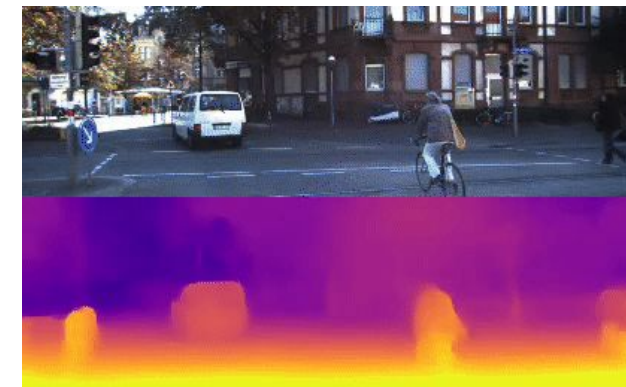
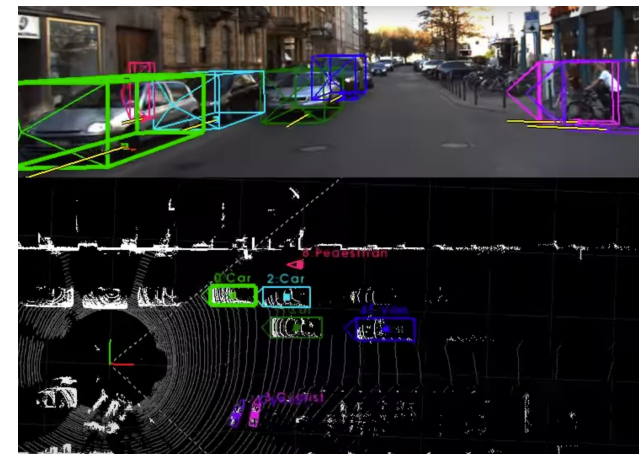
- KITTI, - RGB 帧、3D bbox、深度*



Bird-Eye-View Perception

- BEV Perception datasets

- KITTI, - RGB frames, 3D bbox, Depth*



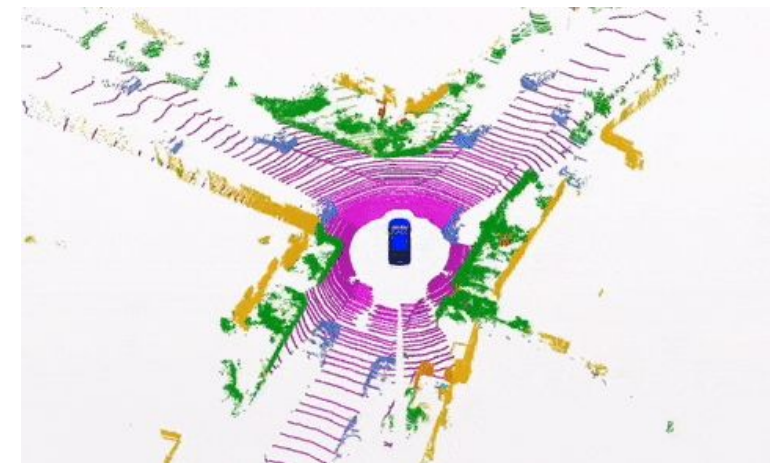
鸟瞰图 (BEV) 感知

- BEV 感知数据集
 - SemanticKITTI - RGB 帧、3D bbox、三维分割



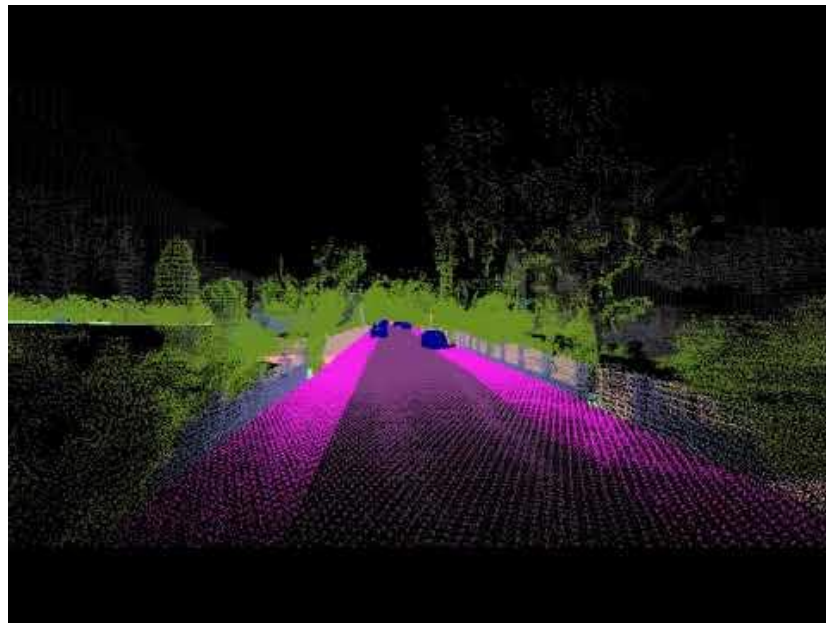
Bird-Eye-View Perception

- BEV Perception datasets
 - SemanticKITTI - RGB frames, 3D bbox, 3D segmentation



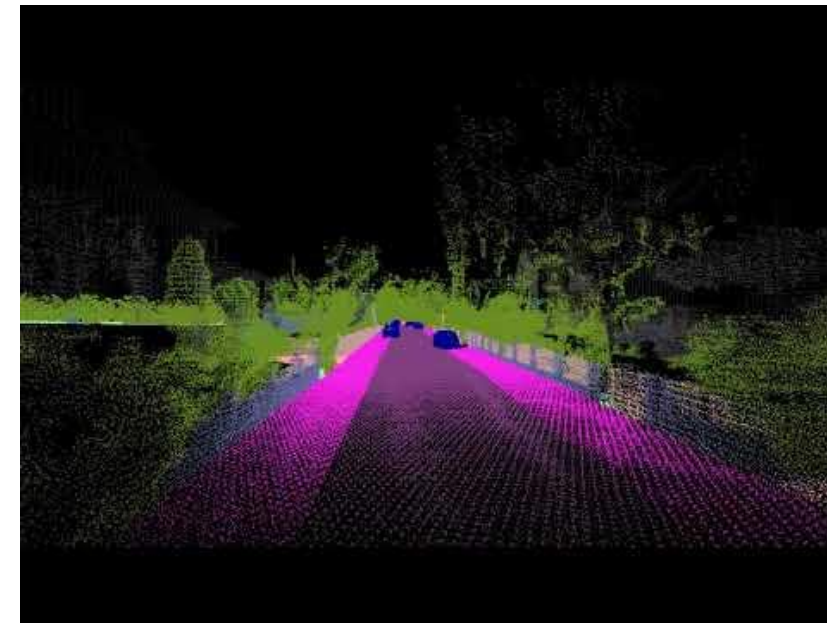
鸟瞰图 (BEV) 感知

- BEV 感知数据集
 - KITTI-360 - 2019



Bird-Eye-View Perception

- BEV Perception datasets
 - KITTI-360 - 2019



鸟瞰图 (BEV) 感知

- BEV 感知数据集

- 还有更多

- nuScenes
 - Waymo Open Dataset
 - Argo

Dataset	Year	City
KITTI [11]	2012	EU
Waymo [8]	2019	NA
nuScenes [7]	2019	NA/AS
Argo v1 [24]	2019	NA
Argo v2 [12]	2022	NA
ApolloScape [25]	2018	AS
OpenLane [26]	2022	NA
ONCE-3DLanes [27]	2022	AS
Lyft L5 [28]	2019	AF
A* 3D [29]	2019	AS
H3D [30]	2019	NA
SemanticKITTI [31]	2019	EU
A2D2 [32]	2020	EU
Cityscapes 3D [33]	2020	-
PandaSet [34]	2020	NA
KITTI-360 [35]	2020	EU
Cirrus [36]	2020	-
ONCE [37]	2021	AS
AIODrive [38]	2021	Sim
DeepAccident [39]	2022	Sim

Bird-Eye-View Perception

- BEV Perception datasets

- And more

- nuScenes
 - Waymo Open Dataset
 - Argo

Dataset	Year	City
KITTI [11]	2012	EU
Waymo [8]	2019	NA
nuScenes [7]	2019	NA/AS
Argo v1 [24]	2019	NA
Argo v2 [12]	2022	NA
ApolloScape [25]	2018	AS
OpenLane [26]	2022	NA
ONCE-3DLanes [27]	2022	AS
Lyft L5 [28]	2019	AF
A* 3D [29]	2019	AS
H3D [30]	2019	NA
SemanticKITTI [31]	2019	EU
A2D2 [32]	2020	EU
Cityscapes 3D [33]	2020	-
PandaSet [34]	2020	NA
KITTI-360 [35]	2020	EU
Cirrus [36]	2020	-
ONCE [37]	2021	AS
AIODrive [38]	2021	Sim
DeepAccident [39]	2022	Sim

鸟瞰图 (BEV) 感知

- 已有的 BEV 应用



Bird-Eye-View Perception

- Existing BEV applications



目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 几何计算机视觉
 - 相机模型
 - 透视投影
 - 相机标定
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

自我中心感知

- 或者说透视视图感知 / 第一人称视角感知
 - 用于机器人视觉与增强现实

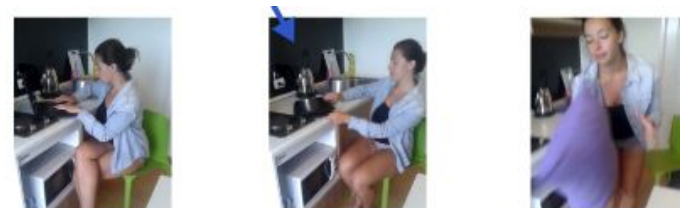


First Person



不同于...

Third Person



Egocentric Perception

- Or perspective view perception / First-person view perception
 - For robot vision and augmented reality



First Person



Different from...

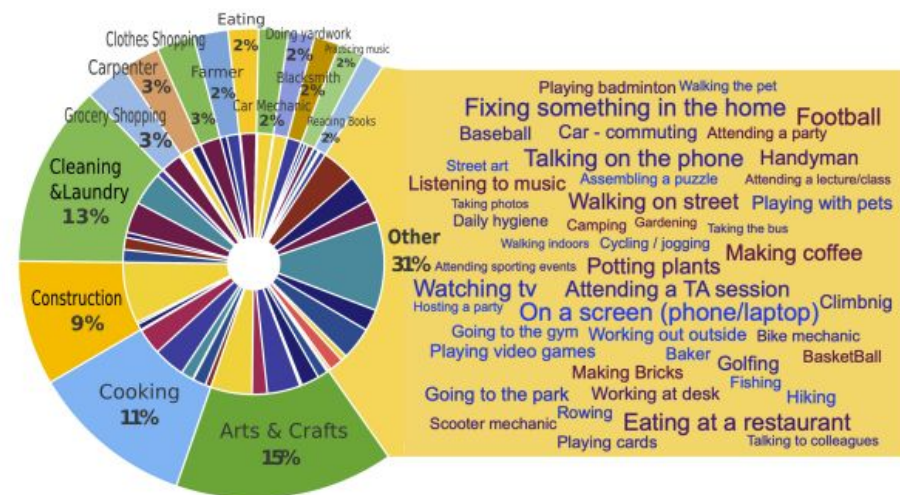
Third Person



自我中心感知

• Ego4D 数据集

- 3000+ 小时自我中心视频（头戴 GoPro 等），覆盖全球 74 个地点
- 日常活动：

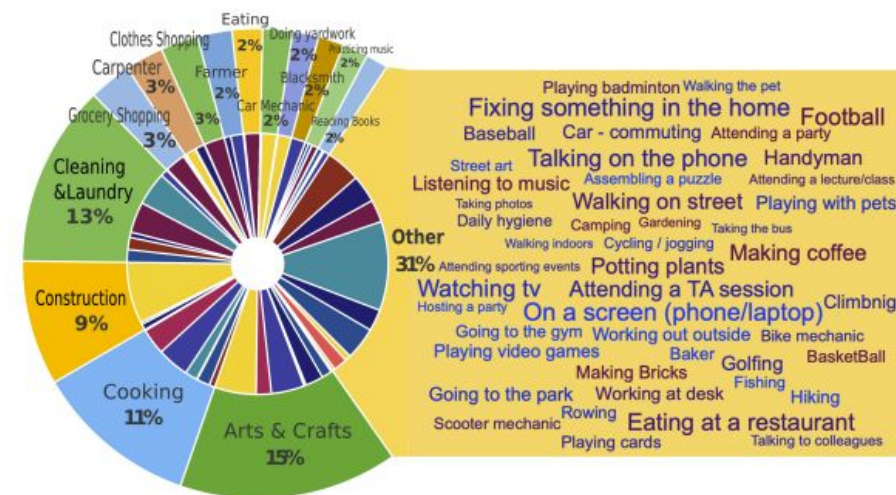


[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022
[2] <https://ego4d-data.org/>

Egocentric Perception

• Ego4D dataset

- 3000+ hours of ego-centric videos (head-mounted go-pros, etc.) over 74 locations in the world
- Daily activities:



[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022
[2] <https://ego4d-data.org/>

自我中心感知

- Ego4D 数据集

- 3000+ 小时自我中心视频（头戴 GoPro 等），覆盖全球 74 个地点
- 任务：手与物体交互

■ 可用于机器人示范学习！



State-change: Plant removed from ground

[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022
[2] <https://ego4d-data.org/>

Egocentric Perception

- Ego4D dataset

- 3000+ hours of ego-centric videos (head-mounted go-pros, etc.) over 74 locations in the world
- Task: Hands and Objects interaction
 - For robot demonstration learning!



State-change: Plant removed from ground

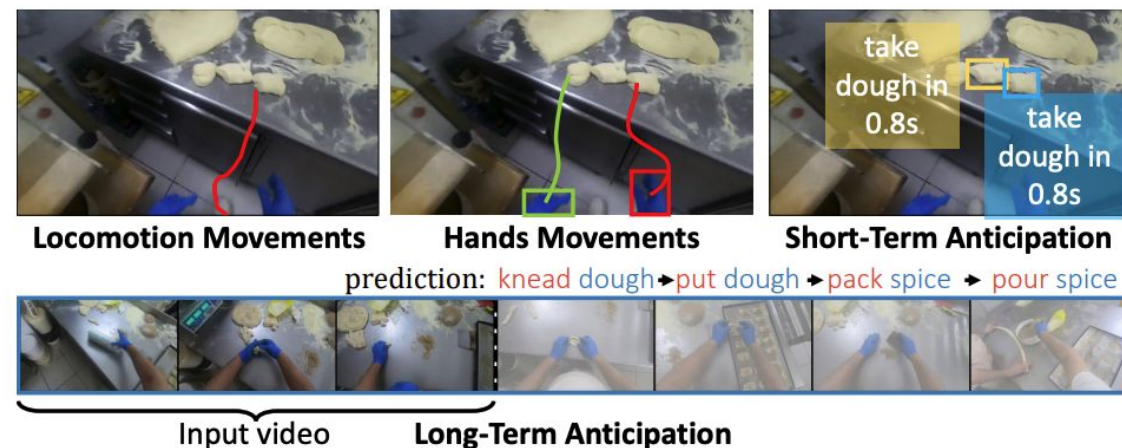
[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022
[2] <https://ego4d-data.org/>

自我中心感知

• Ego4D 数据集

- 3000+ 小时自我中心视频（头戴 GoPro 等），覆盖全球 74 个地点
- 任务：预测（Forecasting）

■ 预测这个人接下来会做什么、手会怎么动 —— 世界模型！



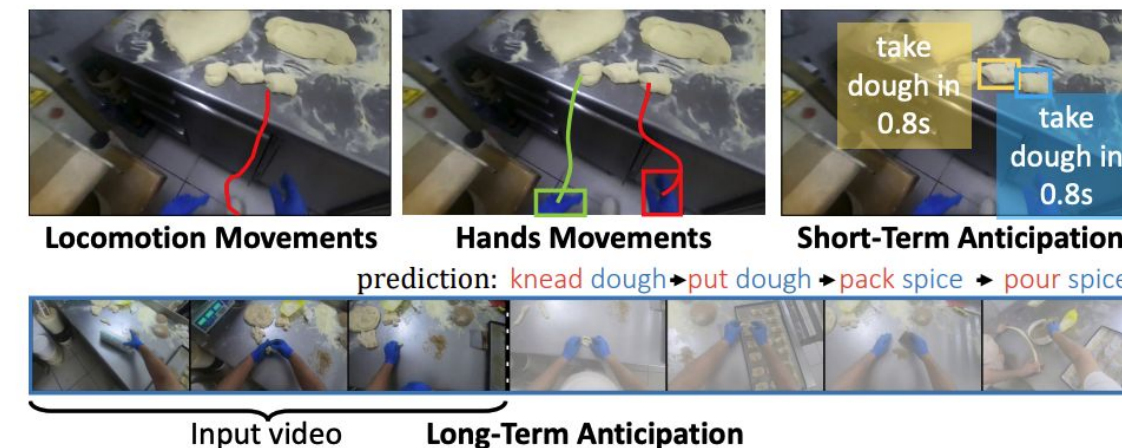
[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022
[2] <https://ego4d-data.org/>

Egocentric Perception

• Ego4D dataset

- 3000+ hours of ego-centric videos (head-mounted go-pros, etc.) over 74 locations in the world
- Task: Forecasting

■ Predict what the person will do, their hand movements - world model!

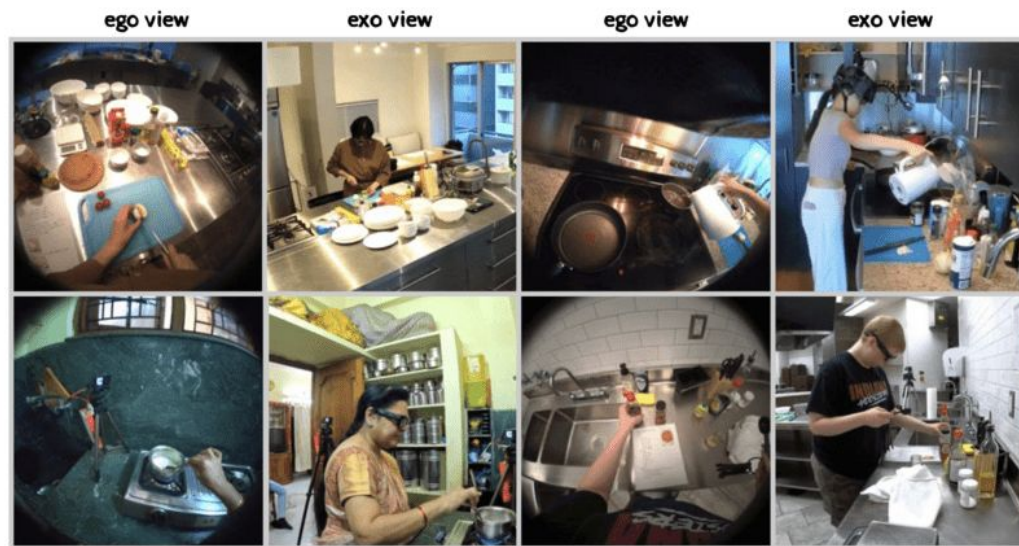


[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022
[2] <https://ego4d-data.org/>

自我中心感知

- Ego-Exo4D 数据集

- 第一人称 + 第三人称视频数据

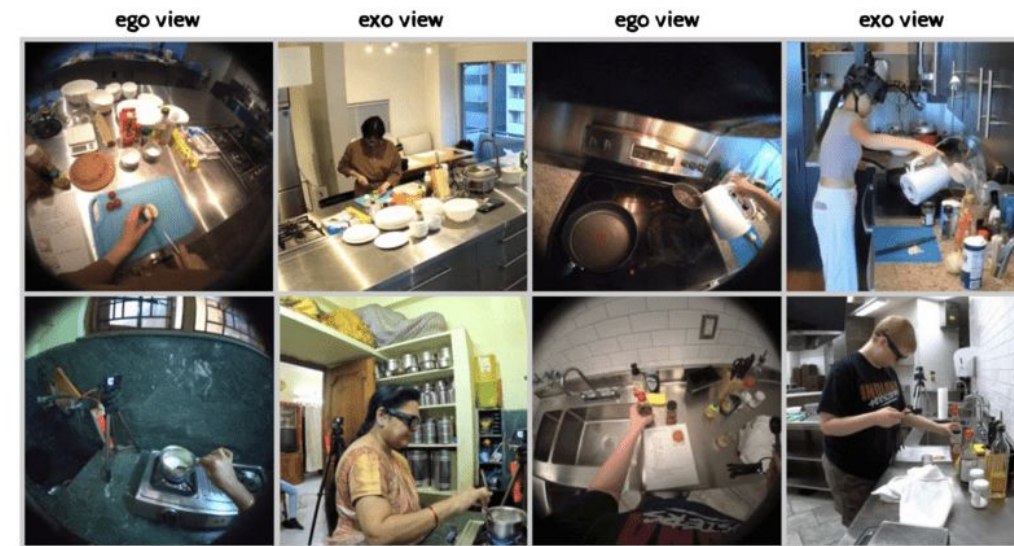


[1] Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. Dec 2023
[2] <https://ego-exo4d-data.org/>

Egocentric Perception

- Ego-Exo4D dataset

- First-person + third person video data



[1] Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. Dec 2023
[2] <https://ego-exo4d-data.org/>

自我中心感知

- Ego-Exo4D 数据集

- 第一人称 + 第三人称视频数据
- 从人类示范中训练预训练模型，学习如何完成某些任务

Bike Repair



Narrate and Act: It's nice and tight, clockwise to tighten that drive side nut, do another quick check.

Atomic Action Description: C tightens the axle nut with the bicycle skewer in his right hand.

Expert Commentary: Now the mechanic is applying pressure to the left side of the wheel and pushing the wheel towards the right. That allows to have the cog pull the chain and create a little more tension on the chain. Meanwhile, he is tightening the right side nut and making sure that there is some tension on the chain allowing a little slack but also holding it into place.

[1] Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. Dec 2023
[2] <https://ego-exo4d-data.org/>

Egocentric Perception

- Ego-Exo4D dataset

- First-person + third person video data
- Train a pretrain model for how to accomplish certain tasks from human demonstrations

Bike Repair



Narrate and Act: It's nice and tight, clockwise to tighten that drive side nut, do another quick check.

Atomic Action Description: C tightens the axle nut with the bicycle skewer in his right hand.

Expert Commentary: Now the mechanic is applying pressure to the left side of the wheel and pushing the wheel towards the right. That allows to have the cog pull the chain and create a little more tension on the chain. Meanwhile, he is tightening the right side nut and making sure that there is some tension on the chain allowing a little slack but also holding it into place.

[1] Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. Dec 2023
[2] <https://ego-exo4d-data.org/>

目录

- 目标
 - 我们将学习具身智能中的关键感知模型
- 任务内容
 - 计算机视觉基础
 - 图像与视频存储
 - CNN 之前的特征
 - 视觉识别
 - 几何计算机视觉
 - 相机模型
 - 透视投影
 - 相机标定
 - 鸟瞰图 (BEV) 感知
 - 第一人称视角感知

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

阅读材料

- BEV 感知综述

- Delving into the Devils of Bird's-eye-view Perception: A Review, Evaluation and Recipe. 2022

- Vision-Centric BEV Perception: A Survey, 2022

- 相机标定教程

- https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

- https://boofcv.org/index.php?title=Tutorial_Camera_Calibration

- https://github.com/zhengthomastang/Cal_PnP

- EPro-PnP: Generalized End-to-End Probabilistic Perspective-n-Points for Monocular Object Pose Estimation, CVPR 2022 best paper

- 标定图像中的物体

Reading Material

- BEV perception survey

- Delving into the Devils of Bird's-eye-view Perception: A Review, Evaluation and Recipe. 2022

- Vision-Centric BEV Perception: A Survey, 2022

- Camera calibration tutorial

- https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

- https://boofcv.org/index.php?title=Tutorial_Camera_Calibration

- https://github.com/zhengthomastang/Cal_PnP

- EPro-PnP: Generalized End-to-End Probabilistic Perspective-n-Points for Monocular Object Pose Estimation, CVPR 2022 best paper

- Calibrate objects in the image



谢谢！



Thank you!